

Universal Serial Bus (USB)

This chapter describes the universal serial bus (USB) controller on the microcontroller.

Topic	Page
32.1 Overview.....	1817
32.2 USB Host Controller	1817
32.3 USB Device Controller	1844
32.4 USB Connectivity.....	1938

32.1 Overview

The microcontroller provides several varieties of USB functionality, including:

- USB host: device provides a two-port *USB Specification Revision 1.1* - compliant host controller, which is based on the *OHCI Specification for USB Release 1.0a*
- USB device: device provides a full-speed USB device
- Support for up to 2 active simultaneous ports (1 Function and 1 Host, or 2 Hosts)

32.2 USB Host Controller

The USB host controller (HC) is a two-port controller that communicates with USB devices at the USB low-speed (1.5M bit-per-second maximum) and full-speed (12M bit-per-second maximum) data rates. It is compatible with the *Universal Serial Bus Specification Revision 2.0* and the *Open HCI—Open Host Controller Interface Specification for USB*, Release 1.0a, available through the Compaq Computer Corporation web site, and hereafter called the *OHCI Specification for USB*. It is assumed that users of the USB host controller are already familiar with the *USB Specification* and *OHCI Specification for USB*.

The USB host controller implements the register set and makes use of the memory data structures defined in the *OHCI Specification for USB*. These registers and data structures are the mechanism by which a USB host controller driver software package can control the USB host controller. The *OHCI Specification for USB* also defines how the USB host controller implementation must interact with those registers and data structures in system memory. The device CPU accesses these registers via the module's peripheral bus.

To reduce processor software and interrupt overhead, the USB host controller generates USB traffic based on data structures and data buffers stored in system memory. The USB host controller accesses these data structures without direct intervention by the processor using the master access (bus transaction initiator) port. These data structures and data buffers can be located in internal or external RAM.

The USB host controller provides an interrupt to the Vectored Interrupt Manager (VIM) to signal certain hardware events to the host controller driver software.

32.2.1 USB Open Host Controller Interface Functionality

32.2.1.1 OHCI Controller Overview

The *Open HCI—Open Host Controller Interface Specification for USB*, Release 1.0a, defines a set of registers and data structures stored in system memory that define how a USB host controller interfaces to system software. This specification, in conjunction with the *Universal Serial Bus Specification Version 2.0*, defines most of the USB functionality that the USB host controller provides.

The *OHCI Specification for USB* focuses on two main aspects of the hardware implementation of a USB host controller: its register set and the memory data structures that define the activity to appear on the USB bus. Also discussed are issues such as interrupt generation, USB host controller state, USB frame management, and the methods that the hardware must use to process the lists of data structures in system memory.

This document does not duplicate the information presented in the *OHCI Specification for USB* or the *USB Specification*. USB host controller users can refer to the *USB Specification* and the *OHCI Specification for USB* for detailed discussions of USB requirements and OHCI controller operation.

32.2.2 USB Host Controller Differences From OHCI Specification for USB

The USB host controller implementation does not implement every aspect of the functionality defined in the *OHCI Specification for USB*. The differences focus on the OHCI ownership change interrupt. Other restrictions are imposed by the effects of the device pin multiplexing options.

32.2.2.1 Top-Level Pin Multiplexing and OHCI Registers

The RM4x microcontroller supports two USB Host ports and one USB Device port. The USB device port interface signals are multiplexed with the second USB Host port interface signals. All USB interface signals are also multiplexed with other functions on the RM4x microcontrollers. The configuration of these multiplexors is done by configuring the I/O Multiplexing Module control registers. Refer to the *I/O Multiplexing and Control Module (IOMM)* chapter for more information on the sequence for configuring these control registers.

The OHCI RHDESCRIPTORA register always reports two available USB host ports, regardless of the top-level pin multiplexing settings.

Table 32-1. USB Host / Device Interface Signal Multiplexing and Control

USB Host / Device Interface Signal Name	Control for Selecting USB Host Signal	Control for Selecting USB Device Signal
USB1.OVERCURRENT	PINMMR14 [7:0] = 08h	-
USB1.RCV	PINMMR13 [31:24] = 08h	-
USB1.VM	PINMMR12 [31:24] = 08h	-
USB1.VP	PINMMR12 [23:16] = 08h	-
USB1.PORTPOWER	PINMMR21 [15:8] = 02h	-
USB1.SPEED	PINMMR19 [15:8] = 04h	-
USB1.SUSPEND	PINMMR20 [23:16] = 08h	-
USB1.TXDAT	PINMMR18 [31:24] = 02h	-
USB1.TXEN	PINMMR17 [7:0] = 04h	-
USB1.TXSE0	PINMMR18 [15:8] = 02h	-
USB2.OVERCURRENT / USB_FUNC.VBUSI	PINMMR1 [15:8] = 08h	PINMMR1 [15:8] = 10h
USB2.RCV / USB_FUNC.RXDI	PINMMR0 [7:0] = 02h	PINMMR0 [7:0] = 04h
USB2.VM / USB_FUNC.RXDMI	PINMMR1 [7:0] = 02h	PINMMR1 [7:0] = 04h
USB2.VP / USB_FUNC.RXDPI	PINMMR0 [15:8] = 02h	PINMMR0 [15:8] = 04h
USB2.PORTPOWER / USB_FUNC.GZO	PINMMR6 [7:0] = 02h	PINMMR6 [7:0] = 04h
USB2.SPEED / USB_FUNC.PUENON	PINMMR4 [31:24] = 04h	PINMMR4 [31:24] = 08h
USB2.SUSPEND / USB_FUNC.SUSPENDO	PINMMR6 [23:16] = 04h	PINMMR6 [23:16] = 08h
USB2.TXDAT / USB_FUNC.TXDO	PINMMR2 [7:0] = 02h	PINMMR2 [7:0] = 04h
USB2.TXEN / USB_FUNC.PUENO	PINMMR4 [23:16] = 04h	PINMMR4 [23:16] = 08h
USB2.TXSE0 / USB_FUNC.SE0O	PINMMR3 [15:8] = 02h	PINMMR3 [15:8] = 04h

32.2.2.2 No Ownership Change Interrupt

The USB host controller does not implement the OHCI ownership change interrupt.

32.2.3 Implementation of OHCI Specification for USB

32.2.3.1 Isochronous Transmit Descriptor (TD) OFFSETX/PSWX Values

The USB host controller implements the *OHCI Specification for USB* optional feature of checking isochronous OFFSETX/PSWX values. If either OFFSETX or OFFSET(X+1) does not have a condition code of *Not Accessed*, or if the OFFSET(X+1) value is not greater than or equal to OFFSETX, then an unrecoverable error is reported. Unrecoverable errors issued for these reasons do not cause an update of the HOSTUEADDR, HOSTUESTATUS, or HOSTTIMEOUTCTRL registers.

32.2.3.2 USB Host Controller Endpoint Descriptor (ED) List Head Pointers

The *OHCI Specification for USB* provides a specific sequence of operations for the host controller driver to perform when setting up the host controller. Failure to follow that sequence can result in malfunction. As a specific example, the HCCONTROLHEADED and HCBULKHEADED pointer registers and the 32 HCCAINERRUPTABLE pointers must all point to valid physical addresses of valid endpoint descriptors.

The USB host controller does not check HCCONTROLHEADED registers, HCBULKHEADED registers, or the values in the 32 HCCAINERRUPTABLE pointers before using them to access EDs. If any of these pointers are NULL when the corresponding list enable bit is set, the USB host controller attempts to access using the physical address of 0, which causes an unrecoverable error to be signaled. HOSTUEADDR, HOSTUESTATUS, and HOSTTIMEOUTCTRL registers are updated in this case.

32.2.3.3 OHCI USB Suspend State

The USB host controller ignores upstream traffic from downstream devices for about 3 ms after the host controller state (HCCONTROL.HCFS) changes from USB resume state to USB operational state. If any TDs cause generation of downstream packets during that time, the downstream packets are sent, but any response provided by the downstream device is ignored. Any such TDs are aborted with completion codes marked as *Device Not Responding*. TDs on any of the lists (periodic, control, bulk, and isochronous) can cause such an occurrence.

The USB specification requires that system software must provide a 10-ms resume recovery time (T_{RSMRCY}) after a bus segment transitions from resume signaling to normal operational mode. During that time, only start of frame packets are to be sent on the bus segment. It is recommended that system software disable all list enable bits (HCCONTROL.PLE, HCCONTROL.IE, HCCONTROL.CLE, and HCCONTROL.BLE) and then wait for at least 1 ms before setting the host controller into USB suspend state (via HCCONTROL.HCFS). When restoring from suspend, system software must set the host controller into USB resume state, and wait for the host controller to transition into USB operational state. System software must then wait 10 ms before enabling the host controller list enable bits.

When the host controller has been placed into the USB suspend state under software control, but is brought out by a remote wake-up, system software must monitor the HCRHPORTSTATUS[x].PSS and HCRHPORTSTATUS[x].PSSC bits. The HCRHPORTSTATUS[x].PSS bit changes to 0 only after completion of resume signaling on the bus segment completes and completion of the 3-ms period where packets from downstream devices are ignored.

When using port-specific suspend, it is not necessary to disable the host controller lists so long as there are no active EDs and TDs directed toward devices that are downstream of the suspended port. For port-specific suspend operations, the host controller does not issue a root hub status change interrupt with the HCRHPORTSTATUS[n].PSSC bit = 1 and HCRHPORTSTATUS[n].PSS = 0 until after the approximately 3-ms delay after resume signaling completes.

When using port-specific suspend, system software must ensure that there are no active EDs for devices that are downstream of the suspended port before setting the port into suspend mode. While the port is in suspend or being resumed, system software must not enable any EDs for any devices downstream of the suspended port. Once the root hub status change interrupt occurs as a result of the suspended port PSS bit changing to 0, EDs can be enabled for devices downstream of the port that is now operational.

32.2.4 USB Host Controller Registers

Most of the host controller (HC) registers are the OHCI operational registers, which are defined by the *OHCI Specification for USB*. Four additional registers not specified by the *OHCI Specification for USB* provide additional information about the USB host controller state. USB host controller registers can be accessed in user and supervisor modes.

The USB host controller registers are listed in [Table 32-2](#). [Section 32.2.4.1](#) through [Section 32.2.4.27](#) describe specific register bits. The base address for the USB Host controller registers is FCF7 8B00h.

Table 32-2. USB Host Controller Registers

Address	Register ⁽¹⁾	Description	Section
FCF7 8B00h	HCREVISION	OHCI revision number	Section 32.2.4.1
FCF7 8B04h	HCCONTROL	HC operating mode	Section 32.2.4.2
FCF7 8B08h	HCCOMMANDSTATUS	HC command and status	Section 32.2.4.3
FCF7 8B0Ch	HCINTERRUPTSTATUS	HC interrupt status	Section 32.2.4.4
FCF7 8B10h	HCINTERRUPTENABLE	HC interrupt enable	Section 32.2.4.5
FCF7 8B14h	HCINTERRUPTDISABLE	HC interrupt disable	Section 32.2.4.6
FCF7 8B18h	HCHCCA	Physical address of HCCA ⁽²⁾	Section 32.2.4.7
FCF7 8B1Ch	HCPERIODCURRENTED	Physical address of current periodic endpoint descriptor ⁽²⁾	Section 32.2.4.8
FCF7 8B20h	HCCONTROLHEADED	Physical address of head of control endpoint descriptor list ⁽²⁾	Section 32.2.4.9
FCF7 8B24h	HCCONTROLCURRENTED	Physical address of current control endpoint descriptor ⁽²⁾	Section 32.2.4.10
FCF7 8B28h	HCBULKHEADED	Physical address of head of bulk endpoint descriptor list ⁽²⁾	Section 32.2.4.11
FCF7 8B2Ch	HCBULKCURRENTED	Physical of current bulk endpoint descriptor ⁽²⁾	Section 32.2.4.12
FCF7 8B30h	HCDONEHEAD	Physical address of head of list of retired transfer descriptors ⁽²⁾	Section 32.2.4.13
FCF7 8B34h	HCFMINTERVAL	HC frame interval	Section 32.2.4.14
FCF7 8B38h	HCFMREMAINING	HC frame remaining	Section 32.2.4.15
FCF7 8B3Ch	HCFMNUMBER	HC frame number	Section 32.2.4.16
FCF7 8B40h	HCPERIODICSTART	HC periodic start	Section 32.2.4.17
FCF7 8B44h	HCLSTHRESHOLD	HC low-speed threshold	Section 32.2.4.18
FCF7 8B48h	HCRHDESCRIPTORA	HC root hub A	Section 32.2.4.19
FCF7 8B4Ch	HCRHDESCRIPTORB	HC root hub B	Section 32.2.4.20
FCF7 8B50h	HCRHSTATUS	HC root hub status	Section 32.2.4.21
FCF7 8B54h	HCRHPORTSTATUS0	HC port 0 control and status	Section 32.2.4.22
FCF7 8B58h	HCRHPORTSTATUS1	HC port 1 control and status	Section 32.2.4.23
FCF7 8BE0h	HOSTUEADDR	Host UE address	Section 32.2.4.24
FCF7 8BE4h	HOSTUESTATUS	Host UE status	Section 32.2.4.25
FCF7 8BE8h	HOSTTIMEOUTCTRL	Host time-out control	Section 32.2.4.26
FCF7 8BECh	HOSTREVISION	Host revision	Section 32.2.4.27

⁽¹⁾ Access to these registers must be by 32-bit reads or 32-bit writes. Use of other access sizes may result in undefined operation.

⁽²⁾ Restrictions apply to the physical addresses used in these registers. See [Section 32.2.9](#).

32.2.4.1 OHCI Revision Number Register (HCREVISION)

The OHCI revision number ([Figure 32-1](#)) register reports the revision number of the *OHCI Specification for USB* upon which the USB host controller is based.

Figure 32-1. OHCI Revision Number Register (HCREVISION) [address = FCF78B00h]

31	Reserved															16	
R-0																	
15	Reserved							8	7	REV							0
R-0								R-10h									

LEGEND: R = Read only; -n = value at reset

Table 32-3. OHCI Revision Number Register (HCREVISION) Bit Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reserved
7-0	REV	10h	OHCI specification revision. The OHCI revision number upon which the USB host controller is based. Write has no effect.

32.2.4.2 HC Operating Mode Register (HCCONTROL)

The HC operating mode register ([Figure 32-2](#)) controls the operating mode of the USB host controller.

Figure 32-2. HC Operating Mode Register (HCCONTROL) [address = FCF78B04h]

31	Reserved															16
R-0																
15	Reserved										11	10	9	8		
R-0											RWE		RWC		IR	
R-0											R/W-0		R/W-0		R/W-0	
7	6	5	4	3	2	1	0									
HCFS		BLE		CLE		IE		PLE		CBSR						
R/W-00		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0						

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 32-4. HC Operating Mode Register (HCCONTROL) Bit Field Descriptions

Bit	Field	Value	Description
31-11	Reserved	0	Reserved
10	RWE	0	Remote wake-up enable. This bit has no effect in the device. The USB host controller does not provide a processor wake-up mechanism.
9	RWC	0	Remote wake-up connected. This bit has no effect in the device. The USB host controller does not provide a processor wake-up mechanism.
8	IR	0	Interrupt routing. The USB host controller does not provide an SMI interrupt. This bit must be 0 to allow the USB host controller interrupt to propagate to the Vectored Interrupt Manager (VIM).

Table 32-4. HC Operating Mode Register (HCCONTROL) Bit Field Descriptions (continued)

Bit	Field	Value	Description
7-6	HCFS	0 1h 2h 3h	Host controller functional state: USB reset USB resume USB operational USB suspend A transition to USB operational causes SOF generation to begin in 1 ms. The USB host controller can automatically transition from USB suspend to USB resume if a downstream resume is received. The USB host controller enters USB suspend after a software reset. The USB host controller enters USB reset after a hardware reset. The USB reset state resets the root hub and causes downstream signaling of USB reset.
5	BLE	0 1	Bulk list enable: 0 Bulk ED list not processed in the next 1-ms frame. Host controller driver can modify the list. If driver removes the ED pointed to by the HCBULKCURRENTED from the ED list, it must update HCBULKCURRENTED to point to an ED still on the list before it re-enables the bulk list. 1 Enables processing of bulk ED list. HCBULKHEADED must be 0 or point to a valid ED before setting this bit. HCBULKCURRENTED must point to a valid ED or be 0 before setting this bit.
4	CLE	0 1	Control list enable: 0 Control ED list is not processed in the next 1-ms frame. Host controller driver can modify the control ED list. If driver removes the ED pointed to by the HCCONTROLCURRENTED from the ED list, it must update HCCONTROLCURRENTED to point to an ED still on the list before it re-enables the control list. 1 Enables processing of the control ED list. HCCONTROLHEADED must be 0 or point to a valid ED before setting this bit. HCCONTROLCURRENTED must be 0 or point to a valid ED before setting this bit.
3	IE	0 1	Isochronous enable 0 Isochronous EDs are not processed. The USB host controller checks this bit every time it finds an isochronous ED in the periodic list. 1 Enables processing of isochronous EDs. When this bit is written to 1, processing of isochronous EDs can be enabled in the next frame, if not in the current frame.
2	PLE	0 1	Periodic list enable 0 The periodic ED lists are not processed. When written to 0, periodic list processing is disabled beginning with the next frame. 1 Enables processing of the periodic ED lists. When written to 1, periodic list processing begins in the next frame.
1-0	CBSR	0 1h 2h 3h	Control/bulk service ratio. Specifies the ratio between control and bulk EDs processed in a frame. 0 One control ED per bulk ED. 1h Two control EDs per bulk ED. 2h Three control EDs per bulk ED. 3h Four control EDs per bulk ED.

32.2.4.3 HC Command and Status Register (HCCOMMANDSTATUS)

The HC command and status register shows the current state of the host controller and accepts commands from the host controller driver.

Figure 32-3. HC Command and Status Register (HCCOMMANDSTATUS) [address = FCF78B08h]

31					18	17	16		
Reserved					SOC				
R-0					R-0				
15					4	3	2	1	0
Reserved					OCR	BLF	CLF	HCR	
R-0					R/W-0	R/W-0	R/W-0	R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-5. HC Command and Status Register (HCCOMMANDSTATUS) Bit Field Descriptions

Bit	Field	Value	Description
31-18	Reserved	0	Reserved
17-16	SOC	0-3h	Scheduling overrun count Counts the number of times a scheduling overrun occurs. This count is incremented even if the host controller driver has not acknowledged any previous pending scheduling overrun interrupt.
15-4	Reserved	0	Reserved
3	OCR	0-1	Ownership change request This bit is set by the host controller driver to gain ownership of the host controller. The device does not support SMI interrupts, so no ownership change interrupt occurs.
2	BLF	0-1	Bulk list filled The host controller driver must set this bit if it modifies the bulk list to include new TDs. If HCBULKCURRENTED is 0, the USB host controller does not begin processing bulk list EDs unless this bit is set. When the USB host controller sees this bit set and begins processing the bulk list, it clears this bit.
1	CLF	0-1	Control list filled The host controller driver must set this bit if it modifies the control list to include new TDs. If HCCONTROLHEADED is 0, the USB host controller does not begin processing control list EDs unless this bit is set. When the USB host controller sees this bit set and begins processing the control list, it clears this bit.
0	HCR	0-1	Host controller reset. Write of 0 has no effect. Value of 1 initiates a software reset of the USB host controller. This transitions the USB host controller to the USB suspend state. This resets most USB host controller OHCI registers. OHCI register accesses must not be attempted until a read of this bit returns a 0. A write of 1 to this bit does not reset the root hub and does not signal USB reset to downstream USB functions.

32.2.4.4 HC Interrupt Status Register (HCINTERRUPTSTATUS)

The HC interrupt status register reports the status of the USB host controller internal interrupt sources.

Figure 32-4. HC Interrupt Status Register (HCINTERRUPTSTATUS) [address = FCF78B0Ch]

31		30		29										16									
Reserved		OC		Reserved																			
R-0		R-0		R-0																			
15								7		6		5		4		3		2		1		0	
Reserved										RHSC		FNO		UE		RD		SF		WDH		SO	
R-0										R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0		R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-6. HC Interrupt Status Register (HCINTERRUPTSTATUS) Bit Field Descriptions

Bit	Field	Value	Description
31	Reserved	0	Reserved
30	OC	0-1	Ownership change The USB host controller does not implement ownership change interrupts.
29-7	Reserved	0	Reserved
6	RHSC	0 1	Root hub status change When 1, indicates a root hub status change has occurred. Write of 0 has no effect. Write of 1 clears this bit.
5	FNO	0 1	Frame number overflow When 1, indicates a frame number overflow has occurred. Write of 0 has no effect. Write of 1 clears this bit.
4	UE	0 1	Unrecoverable error When 1, indicates that an unrecoverable error has occurred on the OCPI bus or that an isochronous TD PSW field condition code was not set to <i>Not Accessed</i> when the USB host controller attempted to perform a transfer using that PSW/offset pair. Write of 0 has no effect. Write of 1 clears this bit.
3	RD	0 1	Resume detected When 1, indicates that a downstream device has issued a resume request. Write of 0 has no effect. Write of 1 clears this bit.
2	SF	0 1	Start of frame When 1, indicates that a SOF has been issued. Write of 0 has no effect. Write of 1 clears this bit.
1	WDH	0 1	Write done head When 1, indicates that the USB host controller has updated the HCDONEHEAD register. Write of 0 has no effect. Write of 1 clears this bit. The host controller driver must read the value from HCDONEHEAD before writing 1 to this bit.
0	SO	0 1	Scheduling overrun When 1, indicates that a scheduling overrun has occurred. Write of 0 has no effect. Write of 1 clears this bit.

32.2.4.5 HC Interrupt Enable Register (HCINTERRUPTENABLE)

The HC interrupt enable register (Figure 32-5) enables various OHCI interrupt sources to generate interrupts to the device level 2 interrupt handler.

Figure 32-5. HC Interrupt Enable Register (HCINTERRUPTENABLE) [address = FCF78B10h]

31	30	29											16					
MIE	OC	Reserved																
R/W-0	R-0	R-0																
15											7	6	5	4	3	2	1	0
Reserved											RHSC	FNO	UE	RD	SF	WDH	SO	
R-0											R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-7. HC Interrupt Enable Register (HCINTERRUPTENABLE) Bit Field Descriptions

Bit	Field	Value	Description
31	MIE	0 1	Master interrupt enable When 1, allows other enabled OHCI interrupt sources to propagate to the Vectored Interrupt Manager (VIM). When 0, OHCI interrupt sources are ignored and no USB host controller interrupts are propagated to the Vectored Interrupt Manager (VIM). Write of 0 has no effect. Write of 1 sets this bit.
30	OC	0	Ownership change This bit has no effect on the device.
29-7	Reserved	0	Reserved
6	RHSC	0 1	Root hub status change When 1 and MIE is 1, allows root hub status change interrupts to propagate to the Vectored Interrupt Manager (VIM). When 0, or when MIE is 0, root hub status change interrupts do not propagate. Write of 0 has no effect. Write of 1 sets this bit.
5	FNO	0 1	Frame number overflow When 1 and MIE is 1, allows frame number overflow interrupts to propagate to the Vectored Interrupt Manager (VIM). When 0, or when MIE is 0, frame number overflow interrupts do not propagate. Write of 0 has no effect. Write of 1 sets this bit.
4	UE	0 1	Unrecoverable error When 1 and MIE is 1, allows unrecoverable error interrupts to propagate to the Vectored Interrupt Manager (VIM). When 0, or when MIE is 0, unrecoverable error interrupts do not propagate. Write of 0 has no effect. Write of 1 sets this bit.
3	RD	0 1	Resume detected When 1 and MIE is 1, allows resume detected interrupts to propagate to the Vectored Interrupt Manager (VIM). When 0, or when MIE is 0, resume detected interrupts do not propagate. Write of 0 has no effect. Write of 1 sets this bit.

Table 32-7. HC Interrupt Enable Register (HCINTERRUPTENABLE) Bit Field Descriptions (continued)

Bit	Field	Value	Description
2	SF	0 1	Start of frame When 1 and MIE is 1, allows start of frame interrupts to propagate to the Vectored Interrupt Manager (VIM). When 0, or when MIE is 0, start of frame interrupts do not propagate. Write of 0 has no effect. Write of 1 sets this bit.
1	WDH	0 1	Write done head When 1 and MIE is 1, allows write done head interrupts to propagate to the Vectored Interrupt Manager (VIM). When 0, or when MIE is 0, write done head interrupts do not propagate. Write of 0 has no effect. Write of 1 sets this bit.
0	SO	0 1	Scheduling overrun When 1 and MIE is 1, allows scheduling overrun interrupts to propagate to the Vectored Interrupt Manager (VIM). When 0, or when MIE is 0, scheduling overrun interrupts do not propagate. Write of 0 has no effect. Write of 1 sets this bit.

32.2.4.6 HC Interrupt Disable Register (HCINTERRUPTDISABLE)

The HC interrupt disable register is used to clear bits in the HCINTERRUPTENABLE register.

Figure 32-6. HC Interrupt Disable Register (HCINTERRUPTDISABLE) [address = FCF78B14h]

31	30	29											16	
MIE	OC	Reserved												
R/W-0	R-0	R-0												
15							7	6	5	4	3	2	1	0
Reserved							RHSC	FNO	UE	RD	SF	WDH	SO	
R-0							R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-8. HC Interrupt Disable Register (HCINTERRUPTDISABLE) Bit Field Descriptions

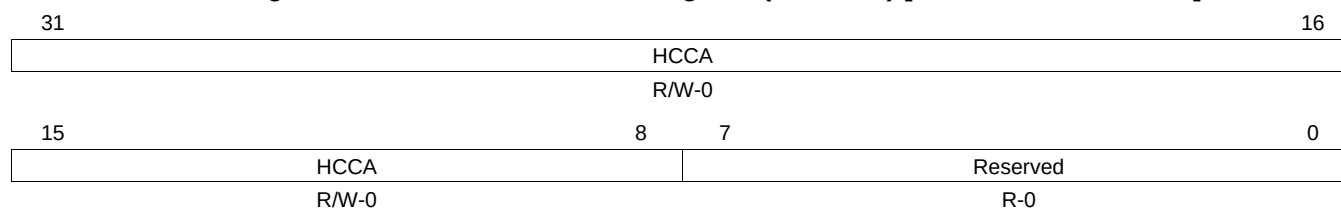
Bit	Field	Value	Description
31	MIE	0 1	Master interrupt enable Read always returns 0. Write of 0 has no effect. Write of 1 clears the HCINTERRUPTENABLE MIE bit.
30	OC	0	Ownership change This bit has no effect on the device.
29-7	Reserved	0	Reserved
6	RHSC	0 1	Root hub status change Read always returns 0. Write of 0 has no effect. Write of 1 clears the HCINTERRUPTENABLE RHSC bit.

Table 32-8. HC Interrupt Disable Register (HCINTERRUPTDISABLE) Bit Field Descriptions (continued)

Bit	Field	Value	Description
5	FNO	0 1	Frame number overflow Read always returns 0. Write of 0 has no effect. Write of 1 clears the HCINTERRUPTENABLE FNO bit.
4	UE	0 1	Unrecoverable error Read always returns 0. Write of 0 has no effect. Write of 1 clears the HCINTERRUPTENABLE UE bit.
3	RD	0 1	Resume detected Read always returns 0. Write of 0 has no effect. Write of 1 clears the HCINTERRUPTENABLE RD bit.
2	SF	0 1	Start of frame Read always returns 0. Write of 0 has no effect. Write of 1 clears the HCINTERRUPTENABLE SF bit.
1	WDH	0 1	Write done head Read always returns 0. Write of 0 has no effect. Write of 1 clears the HCINTERRUPTENABLE WDH bit.
0	SO	0 1	Scheduling overrun Read always returns 0. Write of 0 has no effect. Write of 1 clears the HCINTERRUPTENABLE SO bit.

32.2.4.7 HC HCCA Address Register (HCHCCA)

The HCCA address register defines the physical address of the beginning of the HCCA.

Figure 32-7. HC HCCA Address Register (HCHCCA) [address = FCF78B18h]


LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

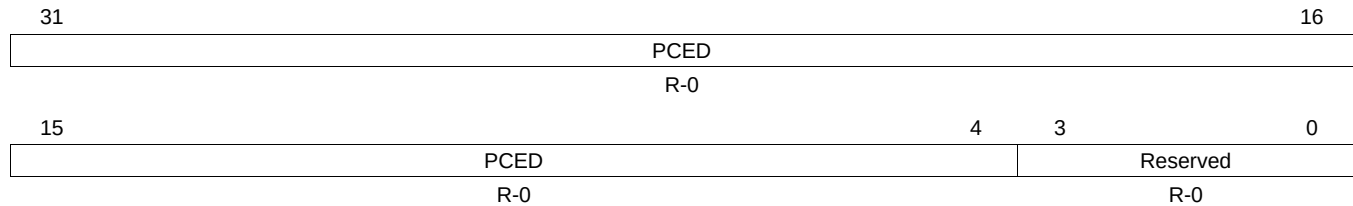
Table 32-9. HC HCCA Address Register (HCHCCA) Bit Field Descriptions

Bit	Field	Value	Description
31-8	HCCA	0-FF FFFFh	Physical address of the beginning of the HCCA.
7-0	Reserved	0	Reserved

32.2.4.8 HC Current Periodic Register (HCPERIODCURRENTED)

The HC current periodic register defines the physical address of the next endpoint descriptor (ED) on the periodic ED List.

Figure 32-8. HC Current Periodic Register (HCPERIODCURRENTED) [address = FCF78B1Ch]



LEGEND: R = Read only; -n = value at reset

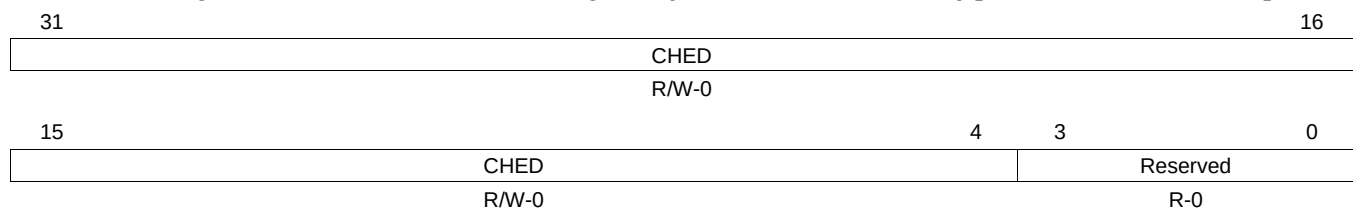
Table 32-10. HC Current Periodic Register (HCPERIODCURRENTED) Bit Field Descriptions

Bit	Field	Value	Description
31-4	PCED	0-FFF FFFFh	Physical address of current ED on the periodic ED list. This field represents bits 31:4 of the physical address of the next ED on the periodic ED List. EDs are assumed to begin at 16-byte-aligned address, so bits 3:0 of this pointer are assumed to be 0. See Section 32.2.9 for the restrictions on physical addresses.
3-0	Reserved	0	Reserved

32.2.4.9 HC Head Control Register (HCCONTROLHEADED)

The HC head control register defines the physical address of the head ED of the control ED list.

Figure 32-9. HC Head Control Register (HCCONTROLHEADED) [address = FCF78B20h]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

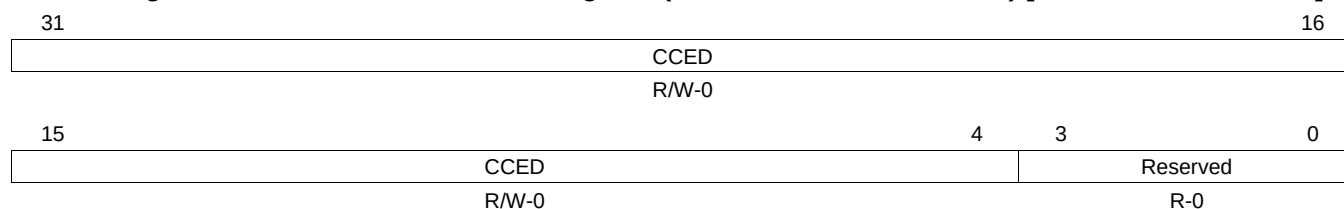
Table 32-11. HC Head Control Register (HCCONTROLHEADED) Bit Field Descriptions

Bit	Field	Value	Description
31-4	CHED	0-FFF FFFFh	Physical address of current ED on the periodic ED list. This field represents bits 31:4 of the physical address of the next ED on the periodic ED List. EDs are assumed to begin at 16-byte-aligned address, so bits 3:0 of this pointer are assumed to be 0. See Section 32.2.9 for the restrictions on physical addresses.
3-0	Reserved	0	Reserved

32.2.4.10 HC Current Control Register (HCCONTROLCURRENTED)

The HC current control register defines the physical address of the next ED on the control ED list.

Figure 32-10. HC Current Control Register (HCCONTROLCURRENTED) [address = FCF78B24h]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-12. HC Current Control Register (HCCONTROLCURRENTED) Bit Field Descriptions

Bit	Field	Value	Description
31-4	CCED	0-FFF FFFh	Physical address of current ED on the periodic ED list. This field represents bits 31:4 of the physical address of the next ED on the periodic ED List. EDs are assumed to begin at 16-byte-aligned address, so bits 3:0 of this pointer are assumed to be 0. See Section 32.2.9 for the restrictions on physical addresses. A value of 0x0000000 indicates that the USB host controller has reached the end of the control ED list without finding any transfers to process. This register is automatically updated by the USB host controller.
3-0	Reserved	0	Reserved

32.2.4.11 HC Head Bulk Register (HCBULKHEADED)

The head bulk register defines the physical address of the head ED on the bulk ED list.

Figure 32-11. HC Head Bulk Register (HCBULKHEADED) [address = FCF78B28h]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

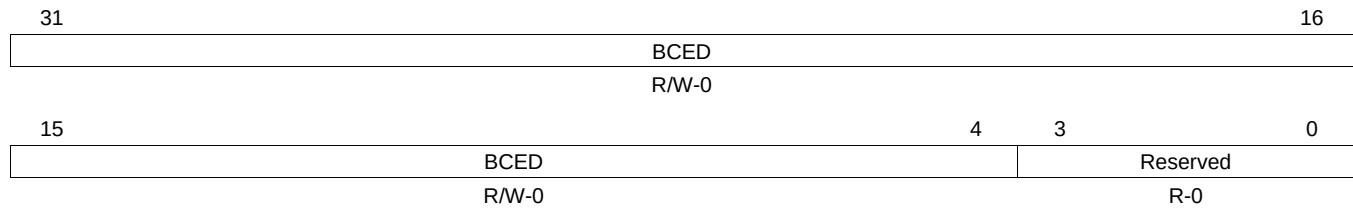
Table 32-13. HC Head Bulk Register (HCBULKHEADED) Bit Field Descriptions

Bit	Field	Value	Description
31-4	BHED	0-FFF FFFh	Physical address of current ED on the periodic ED list. This field represents bits 31:4 of the physical address of the next ED on the periodic ED List. EDs are assumed to begin at 16-byte-aligned address, so bits 3:0 of this pointer are assumed to be 0. See Section 32.2.9 for the restrictions on physical addresses. This register is automatically updated by the USB host controller.
3-0	Reserved	0	Reserved

32.2.4.12 HC Current Bulk Register (HCBULKCURRENTED)

The current bulk register defines the physical address of the next ED on the bulk ED list.

Figure 32-12. HC Current Bulk Register (HCBULKCURRENTED) [address = FCF78B2Ch]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

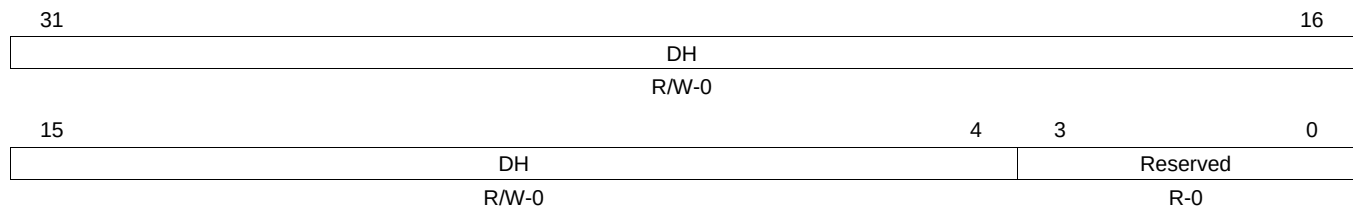
Table 32-14. HC Current Bulk Register (HCBULKCURRENTED) Bit Field Descriptions

Bit	Field	Value	Description
31-4	BCED	0-FFF FFFFh	Physical address of current ED on the periodic ED list. This field represents bits 31:4 of the physical address of the next ED on the periodic ED List. EDs are assumed to begin at 16-byte-aligned address, so bits 3:0 of this pointer are assumed to be 0. See Section 32.2.9 for the restrictions on physical addresses. A value of 0x0000000 indicates that the USB host controller has reached the end of the bulk ED list without finding any transfers to process. This register is automatically updated by the USB host controller.
3-0	Reserved	0	Reserved

32.2.4.13 HC Head Done Register (HCDONEHEAD)

The head done register defines the physical address of the current head of the done TD queue.

Figure 32-13. HC Head Done Register (HCDONEHEAD) [address = FCF78B30h]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-15. HC Head Done Register (HCDONEHEAD) Bit Field Descriptions

Bit	Field	Value	Description
31-4	DH	0-FFF FFFFh	Physical address of current ED on the periodic ED list. This field represents bits 31:4 of the physical address of the next ED on the periodic ED List. EDs are assumed to begin at 16-byte-aligned address, so bits 3:0 of this pointer are assumed to be 0. See Section 32.2.9 for the restrictions on physical addresses. This register is automatically updated by the USB host controller.
3-0	Reserved	0	Reserved

32.2.4.14 HC Frame Interval Register (HCFMINTERVAL)

The frame interval register defines the number of 12-MHz clock pulses in each USB frame.

Figure 32-14. HC Frame Interval Register (HCFMINTERVAL) [address = FCF78B34h]

31	30		16
FIT		FSMPS	
R/W-0		R/W-0	
15	14	13	0
Reserved		FI	
R-0		R/W-2EDFh	

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-16. HC Frame Interval Register (HCFMINTERVAL) Bit Field Descriptions

Bit	Field	Value	Description
31	FIT	0-1	Frame interval toggle The host controller driver must toggle this bit any time it changes the frame interval field.
30-16	FSMPS	0-7FFFh	Largest data packet Largest data packet size allowed for full-speed packets, in bit times.
15-14	Reserved	0	Reserved
13-0	FI	0-3FFFh	Frame interval Number of 12-MHz clocks in the USB frame. Nominally, this is set to 11,999 to give a 1-ms frame. The host controller driver can make minor changes to this field to attempt to manually synchronize with another clock source.

32.2.4.15 HC Frame Remaining Register (HCFMREMAINING)

The HC frame remaining register reports the number of full-speed bit times remaining in the current frame.

Figure 32-15. HC Frame Remaining Register (HCFMREMAINING) [address = FCF78B38h]

31	30		16
FRT		Reserved	
R-0		R-0	
15	14	13	0
Reserved		FR	
R-0		R-0	

LEGEND: R = Read only; -n = value at reset

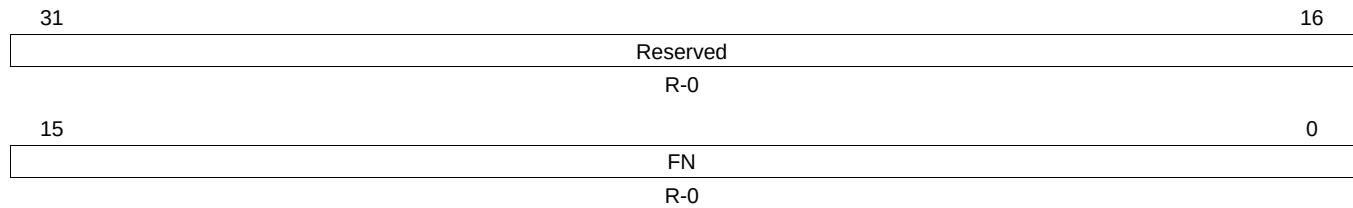
Table 32-17. HC Frame Remaining Register (HCFMREMAINING) Bit Field Descriptions

Bit	Field	Value	Description
31	FRT	0-1	Frame interval toggle The host controller driver must toggle this bit any time it changes the frame interval field.
30-14	Reserved	0	Reserved
13-0	FR	0-3FFFh	Frame remaining The number of full-speed bit times remaining in the current frame. This field is automatically reloaded with the frame interval field value at the beginning of every frame.

32.2.4.16 HC Frame Number Register (HCFMNUMBER)

The HC frame number register reports the current USB frame number.

Figure 32-16. HC Frame Number Register (HCFMNUMBER) [address = FCF78B3Ch]



LEGEND: R = Read only; -n = value at reset

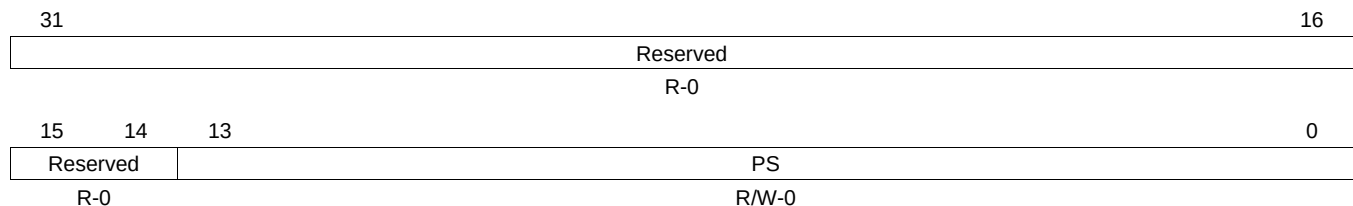
Table 32-18. HC Frame Number Register (HCFMNUMBER) Bit Field Descriptions

Bit	Field	Value	Description
31-14	Reserved	0	Reserved
13-0	FN	0-3FFFh	Frame number This field reports the current USB frame number. It is incremented when the frame remaining field is reloaded with the frame interval field value. Frame number automatically rolls over from FFFFh to 0. After frame number is incremented, its new value is written to the HCCA and the USB host controller sets the SOF interrupt status bit and begins processing the ED lists.

32.2.4.17 HC Periodic Start Register (HCPERIODICSTART)

The HC periodic start register defines the position within the USB frame where EDs on the periodic list have priority over EDs on the bulk and control lists.

Figure 32-17. HC Periodic Start Register (HCPERIODICSTART) [address = FCF78B40h]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-19. HC Periodic Start Register (HCPERIODICSTART) Bit Field Descriptions

Bit	Field	Value	Description
31-14	Reserved	0	Reserved
13-0	PS	0-3FFFh	Periodic start The host controller driver must program this value to be about 10% less than the frame interval field value so that control and bulk EDs have priority for the first 10% of the frame; then periodic EDs have priority for the remaining 90% of the frame.

32.2.4.18 HC Low-Speed Threshold Register (HCLSTHRESHOLD)

The HC low-speed threshold register defines the latest time in a frame that the USB host controller can begin a low-speed packet.

Figure 32-18. HC Low-Speed Threshold Register (HCLSTHRESHOLD) [address = FCF78B44h]

31												16			
Reserved															
R-0															
15		14		13										0	
Reserved		LST													
R-0				R/W-628h											

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-20. HC Low-Speed Threshold Register (HCLSTHRESHOLD) Bit Field Descriptions

Bit	Field	Value	Description
31-14	Reserved	0	Reserved
13-0	LST	0-3FFFh	Low-speed threshold This field defines the number of full-speed bit times in the frame after which the USB host controller cannot start an 8-byte low-speed packet. The USB host controller only begins a low-speed transaction if the frame remaining field is greater than the low-speed threshold. The host controller driver must set this field to a value that ensures that an 8-byte low-speed TD completes before the end of the frame. When set, the host controller driver must not change the value.

32.2.4.19 HC Root Hub A Register (HCRHDESCRIPTORA)

The HC root hub A register defines several aspects of the USB host controller root hub functionality.

Figure 32-19. HC Root Hub A Register (HCRHDESCRIPTORA) [address = FCF78B48h]

31							24		23		16	
POTPG							Reserved					
R/W-2h							R-0					
15		13		12	11	10	9	8	7		0	
Reserved		NOCP	OCPM	DT	NPS	PSM	NDP					
R/W-0		R/W-1	R/W-0	R-0	R/W-1	R/W-0	R-2h					

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-21. HC Root Hub A Register (HCRHDESCRIPTORA) Field Descriptions

Bit	Field	Value	Description
31-24	POTPG	0-FFh	Power-on to power-good time This field defines the minimum amount of time (2 ms × POTPG) between the USB host controller turning on power to a downstream port and when the USB host can access the downstream device. The default value is 0x2, so that the time is 4 ms. This field has no effect on USB host controller operation. After turning on power to a port, the USB host controller driver must delay the amount of time implied by POTPG before attempting to reset an attached downstream device. The required amount of time is system implementation-specific and must be calculated based on the amount of time the VBUS supply takes to provide valid VBUS to a worst-case downstream USB function controller. The implementation-specific value must be computed and then written to this register before the USB host controller driver is initialized.
23-13	Reserved	0	Reserved

Table 32-21. HC Root Hub A Register (HCRHDESCRIPTORA) Field Descriptions (continued)

Bit	Field	Value	Description
12	NOCP	0 1	No overcurrent protection When 0, the USB Host ports support over-current protection inputs. The USB Host ports must not be in the Reset state for the over-current protection inputs to take effect. When 1, this bit indicates that the USB host controller does not implement overcurrent protection inputs. This bit defaults to 1 and can be cleared by the application software.
11	OCPM	0 1	Overcurrent protection mode Over-current status is reported collectively for all downstream ports. This is the default value. Over-current status is reported on a per-port basis. This field is valid only if the NoOverCurrentProtection bit is cleared.
10	DT	0	Device type This bit is always 0, which indicates that the USB host controller implemented is not a compound device.
9	NPS	0 1	No power switching Indicates that VBUS power switching is supported and is either per-port or all-port switched per the power switching mode field. Indicates that VBUS power switching is not supported and that power is available to all downstream ports when the USB host controller is powered. This is the default value. The RM4x microcontroller USB Host ports support over-current protection inputs and automatic power switching upon indication of an over-current condition. The USB Host ports must not be in the Reset state for the over-current protection inputs to take effect.
8	PSM	0 1	Power switching mode Indicates that all ports are powered at the same time. This is the default value. Indicates that each port is powered individually. This mode allows port power to be controlled by either the global switch or per-port switching. If the PortPowerControlMask (PPCM) bit in the HC Root Hub B Register (HCRHDESCRIPTORB) is set, the USB host port only responds to port power commands (Set/ClearPortPower). If the port mask is cleared, then the USB host port is controlled by the global power switch (Set/ClearGlobalPower).
7-0	NDP	2h	Number of downstream ports This register defaults to 2 to indicate two downstream ports. The USB signal multiplexing mode and top-level pin multiplexing features can place the device in a mode where 0, 1, or 2 of the USB host controller downstream ports are usable. This register reports two ports, regardless of top-level pin multiplexing mode.

32.2.4.20 HC Root Hub B Register (HCRHDESCRIPTORB)

The HC root hub B register defines several aspects of the USB host controller root hub functionality.

Figure 32-20. HC Root Hub B Register (HCRHDESCRIPTORB) [address = FCF78B4Ch]

31	16
PPCM[15-0]	
R/W-0	
15	0
DR[15-0]	
R/W-0	

LEGEND: R/W = Read/Write; -n = value at reset

Table 32-22. HC Root Hub B Register (HCRHDESCRIPTORB) Bit Field Descriptions

Bit	Field	Value	Description
31-16	PPCM	0-FFFFh	Port power control mask Each bit defines whether a corresponding downstream port has port power controlled by the global power control. If set, per-port power control is implemented for the corresponding port. If clear, global power control is implemented for the corresponding port. PPCM bit 0 is reserved. PPCM bit 1 is the port power control mask for downstream port 0. PPCM bit 2 is the port power control mask for downstream port 1. PPCM bits 3 through 15 are reserved.
15-0	DR	0-FFFFh	Device removable Each bit defines whether a corresponding downstream port has a removable or non-removable device. A cleared bit indicates the corresponding port may have a removable device attached. A set bit indicates that the corresponding port has a non-removable device attached. DR bit 0 is reserved.. DR bit 1 is the device removable bit for downstream port 0. DR bit 2 is the device removable bit for downstream port 1. DR bits 3 through 15 are reserved.

32.2.4.21 HC Root Hub Status Register (HCRHSTATUS)

The HC root hub status register reports the USB host controller root hub status.

Figure 32-21. HC Root Hub Status Register (HCRHSTATUS) [address = FCF78B50h]

31	30	18	17	16
CRWE	Reserved	OCIC	LPSC	
R/W-0	R-0	R/W-0	R/W-0	
15	14	2	1	0
DRWE	Reserved	OCI	LPS	
R/W-0	R-0	R-0	R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-23. HC Root Hub Status Register (HCRHSTATUS) Bit Field Descriptions

Bit	Field	Value	Description
31	CRWE	0-1	Clear remote wake-up enable Write of 0 has no effect. Write of 1 clears the device remote wake-up enable bit.
30-18	Reserved	0	Reserved
17	OCIC	0 1	Overcurrent indication change This bit is automatically set when the overcurrent indicator bit changes. Write of 0 has no effect. Write of 1 clears this bit.
16	LPSC	0 1	Local power status change Write of 0 has no effect. Write of 1 sets the port power status bits for all ports if power switching mode is 0. A write of 1 sets port power status bits for ports with their corresponding port power control mask bits cleared if power switching mode is 1.
15	DRWE	0 1	Device remote wake-up enable When 1, this bit enables a connect status change event to be treated as a resume event, which causes a transition from USB suspend to USB resume state and sets the resume detected interrupt status bit. When 0, connect status change events do not cause a transition from USB suspend to USB resume state and the resume detected interrupt is not changed. Write of 0 has no effect. Write of 1 sets the device remote wake-up enable bit.
14-2	Reserved	0	Reserved
1	OCI	0 1	Overcurrent indicator. This bit reports global overcurrent indication if global overcurrent reporting is selected. When 0, no overcurrent condition has been sensed. When 1, this bit indicates that an overcurrent condition has been sensed.
0	LPS	0 1	Local power status Write of 0 has no effect. Write of 1 when in global power mode (power switching mode = 0), turns off power to all ports. If in per-port power mode (power switching mode = 1), a write of 1 turns off power to those ports whose corresponding port power control mask bit is 0.

32.2.4.22 HC Port 0 Status and Control Register (HCRHPORTSTATUS0)

The HC port 0 status and control register reports and controls the state of USB host port 0.

Figure 32-22. HC Port 0 Status and Control Register (HCRHPORTSTATUS0) [address = FCF78B54h]

31													24
Reserved													
R-0													
23	21			20	19	18	17	16					
Reserved				PRSC	OCIC	PSSC	PESC	CSC					
R-0				R/W-0	R/W-0	R/W-0	R/W-0	R/W-0			R/W-0		
15							10	9	8				
Reserved								LSDA/CPP		PPS/SPP			
R-0								R/W-0		R/W-1			
7	5			4	3	2	1	0					
Reserved				PRS/SPR	POCI/CSS	PSS/SPS	PES/SPE	CCS/CPE					
R-0				R/W-0	R/W-0	R/W-0	R/W-0	R/W-0			R/W-0		

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 32-24. HC Port 0 Status and Control Register (HCRHPORTSTATUS0) Field Descriptions

Bit	Field	Value	Description
31-21	Reserved	0	Reserved
20	PRSC	0 1	Port 0 reset status change This bit indicates, when 1, that the port 0 port reset status bit has changed. Write of 0 has no effect. Write of 1 clears this bit.
19	OCIC	0 1	Port 0 overcurrent indicator change This bit indicates, when 1, that the port 0 port overcurrent indicator has changed. Write of 0 has no effect. Write of 1 clears this bit.
18	PSSC	0 1	Port 0 suspend status change This bit indicates, when 1, that the port 0 port suspend status has changed. Suspend status is considered to have changed only after the resume pulse, low-speed EOP, and 3-ms synchronization delays have been completed. Write of 0 has no effect. Write of 1 clears this bit.
17	PESC	0 1	Port 0 enable status change This bit indicates, when 1, that the port 0 port enable status changed. Write of 0 has no effect. Write of 1 clears this bit.
16	CSC	0 1	Port 0 connect status change This bit indicates, when 1, that the port 0 current connect status has changed due to a connect or disconnect event. If current connect status is 0 when a set port reset, set port enable, or set port suspend write occurs, then this bit is set. Write of 0 has no effect. Write of 1 clears this bit. If the HCRHDESCRIPTORB.DR[1] bit is set to indicate a non-removable USB device on port 0, this bit is set only after a root hub reset to inform the system that the device is attached.
15-10	Reserved	0	Reserved

Table 32-24. HC Port 0 Status and Control Register (HCRHPORTSTATUS0) Field Descriptions (continued)

Bit	Field	Value	Description
9	LSDA/PPP	0 1	Port 0 low-speed device attached/clear port power This bit indicates, when read as 1, that a low-speed device is attached to port 0. A 0 in this bit indicates a full-speed device. This bit is valid only when port 0 current connect status is 1. Write of 0 has no effect. The host controller driver can write a 1 to this bit to clear the port 0 port power status.
8	PPS/SPP	0 1	Port 0 port power status/set port power This bit indicates, when read as 1, that the port 0 power is enabled. When read as 0, port 0 power is not enabled. Write of 0 has no effect. Write of 1 to this bit sets the port 0 port power status bit.
7-5	Reserved	0	Reserved
4	PRS/SPR	0 1	Port 0 port reset status/set port reset When read as 1, indicates that port 0 is signaling the USB reset. When read as 0, USB reset is not being sent to port 0. Write of 0 has no effect. Write of 1 to this bit sets the port 0 port reset status bit and causes the USB host controller to begin signaling USB reset to port 0.
3	POCI/CSS	0 1	Port 0 port overcurrent indicator/clear suspend status When read as 1, indicates a port 0 port overcurrent condition has occurred. When 0, no port 0 port overcurrent condition has occurred. Write of 0 has no effect. Write of 1 to this bit when port 0 port suspend status is 1 causes resume signaling on port 0. A write of 1 when port 0 port suspend status is 0 has no effect.
2	PSS/SPS	0 1	Port 0 port suspend status/set port suspend When read as 0, indicates that port 0 is not in the USB suspend state. This bit is cleared automatically at the end of the USB resume sequence and also at the end of the USB reset sequence. When read as 1, indicates that port 0 is in the USB suspend state or is in the resume sequence. Write of 0 has no effect. If port 0 current connect status is 1, a write of 1 to this bit sets the port 0 port suspend status bit and places port 0 in USB suspend state. If current connect status is 0, a write of 1 instead sets connect status change to inform the USB host controller driver software of an attempt to suspend a disconnected device.
1	PES/SPE	0 1	Port 0 port enable status/set port enable When read as 0, this bit indicates that port 0 is not enabled. When read as 1, indicates that port 0 is enabled. This bit is automatically set at completion of port 0 USB reset if it was not already set before the USB reset completed, and is automatically set at the end of a USB suspend if the port was not enabled when the USB resume completed. Write of 0 has no effect. Write of 1 to this bit when port 0 current connect status is 1 sets the port 0 port enable status bit. A write of 1 when port 0 current connect status is 0 has no effect.
0	CCS/CPE	0 1	Port 0 current connection status/clear port enable When read as 1, indicates that port 0 currently has a USB device attached. When 0, indicates that no USB device is attached to port 0. This bit is set to 1 after root hub reset if the HCRHDESCRIPTORB.DR[1] bit is set to indicate a non-removable device on port 0. Write of 0 has no effect. Write of 1 to this bit clears the port 0 port enable bit.

32.2.4.23 HC Port 1 Status and Control Register (HCRHPORTSTATUS1)

The HC port 1 status register reports and controls the state of USB host port 1.

Figure 32-23. HC Port 1 Status and Control Register (HCRHPORTSTATUS1) [address = FCF78B58h]

31											24
Reserved											
R-0											
23	21				20	19	18	17		16	
Reserved					PRSC	OCIC	PSSC	PESC		CSC	
R-0					R/W-0	R/W-0	R/W-0	R/W-0		R/W-0	
15							10	9		8	
Reserved								LSDA/CPP		PPS/SPP	
R-0								R/W-0		R/W-1	
7	5				4	3	2	1		0	
Reserved					PRS/SPR	POCI/CSS	PSS/SPS	PES/SPE		CCS/CPE	
R-0					R/W-0	R/W-0	R/W-0	R/W-0		R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value after reset

Table 32-25. HC Port 1 Status and Control Register (HCRHPORTSTATUS1) Field Descriptions

Bit	Field	Value	Description
31-21	Reserved	0	Reserved
20	PRSC	0 1	Port 1 reset status change This bit indicates, when 1, that the port 1 port reset status bit has changed. Write of 0 has no effect. Write of 1 clears this bit.
19	OCIC	0 1	Port 1 overcurrent indicator change This bit indicates, when 1, that the port 1 port overcurrent indicator has changed. Write of 0 has no effect. Write of 1 clears this bit.
18	PSSC	0 1	Port 1 suspend status changed This bit indicates, when 1, that the port 1 port suspend status has changed. Suspend status is considered to have changed only after the resume pulse, low-speed EOP, and 3-ms synchronization delays have been completed. Write of 0 has no effect. Write of 1 clears this bit.
17	PESC	0 1	Port 1 enable status change This bit indicates, when 1, that the port 1 port enable status changed. Write of 0 has no effect. Write of 1 clears this bit.
16	CSC	0 1	Port 1 connect status change This bit indicates, when 1, that the port 1 current connect status has changed due to a connect or disconnect event. If current connect status is 0 when a set port reset, set port enable, or set port suspend write occurs, then this bit is set. Write of 0 has no effect. Write of 1 clears this bit. If the HCRHDESCRIPTORB.DR[2] bit is set to indicate a nonremovable USB device on port 1, this bit is set only after a root hub reset to inform the system that the device is attached.
15-10	Reserved	0	Reserved

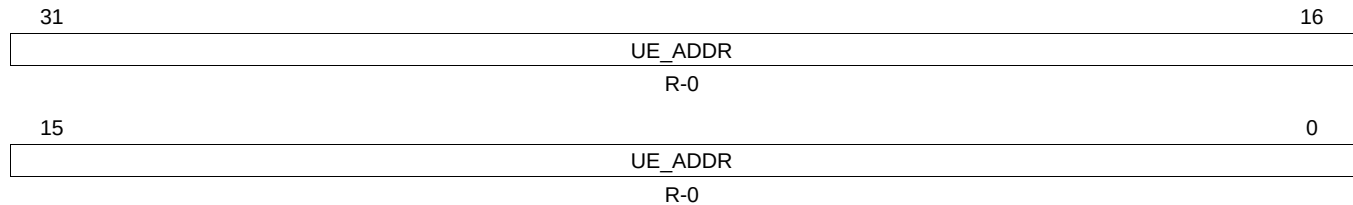
Table 32-25. HC Port 1 Status and Control Register (HCRHPORTSTATUS1) Field Descriptions (continued)

Bit	Field	Value	Description
9	LSDA/CPD	0 Write of 0 has no effect. 1 The host controller driver can write a 1 to this bit to clear the port 1 port power status.	Port 1 low-speed device attached/clear port power This bit indicates, when read as 1, that a low-speed device is attached to port 1. A 0 in this bit indicates a full-speed device. This bit is valid only when port 1 current connect status is 1.
8	PPS/SPP	0 Write of 0 has no effect. 1 Write of 1 to this bit sets the port 1 port power status bit.	Port 1 port power status/set port power This bit indicates, when read as 1, that the port 1 power is enabled. When read as 0, port 1 power is not enabled.
7-5	Reserved	0	Reserved
4	PRS/SPR	0 Write of 0 has no effect. 1 Write of 1 to this bit sets the port 1 port reset status bit and causes the USB host controller to begin signaling USB reset to port 1.	Port 1 port reset status/set port reset When read as 1, indicates that port 1 is sending a USB reset. When read as 0, USB reset is not being sent to port 1.
3	POCI/CSS	0 Write of 0 has no effect. 1 Write of 1 to this bit when port 1 port suspend status is 1 causes resume signaling on port 1. A write of 1 when port 1 port suspend status is 0 has no effect.	Port 1 port overcurrent indicator/clear suspend status When read as 1, indicates that a port 1 port overcurrent condition has occurred. When 0, no port 1 port overcurrent condition has occurred.
2	PSS/SPS	0 Write of 0 has no effect. 1 If port 1 current connect status is 1, a write of 1 to this bit sets the port 1 port suspend status bit and places port 1 in USB suspend state. If current connect status is 0, a write of 1 instead sets connect status change to inform the USB host controller driver software of an attempt to suspend a disconnected device.	Port 1 port suspend status/set port suspend When read as 0, indicates that port 1 is not in the USB suspend state. This bit is cleared automatically at the end of the USB resume sequence and also at the end of the USB reset sequence. When read as 1, indicates that port 1 is in the USB suspend state, or is in the resume sequence.
1	PES/SPE	0 Write of 0 has no effect. 1 Write of 1 to this bit when port 1 current connect status is 1 sets the port 1 port enable status bit. A write of 1 when port 1 current connect status is 0 has no effect.	Port 1 port enable status/set port enable When read as 0, this bit indicates that port 1 is not enabled. When read as 1, indicates that port 1 is enabled. This bit is automatically set at completion of port 1 USB reset if it was not already set before the USB reset completed and is automatically set at the end of a USB suspend if the port was not enabled when the USB resume completed.
0	CCS/CPE	0 Write of 0 has no effect. 1 Write of 1 to this bit clears the port 1 port enable bit.	Port 1 current connection status/clear port enable When read as 1, indicates that port 1 currently has a USB device attached. When 0, indicates that no USB device is attached to port 1. This bit is set to 1 after root hub reset if the HCRHDESCRIPTORB.DR[2] bit is set to indicate a non-removable device on port 1.

32.2.4.24 Host UE Address Register (HOSTUEADDR)

The host UE address register reports the physical address of the last OCPI bus access that caused an unrecoverable error (UE). This register has no meaning until an unrecoverable error has occurred. It also has no meaning if the USB host controller issues an unrecoverable error because the offset checking fault occurred while processing an isochronous TD. This register is not defined by the OHCI specification.

Figure 32-24. Host UE Address Register (HOSTUEADDR) [address = FCF78BE0h]



LEGEND: R = Read only; -n = value at reset

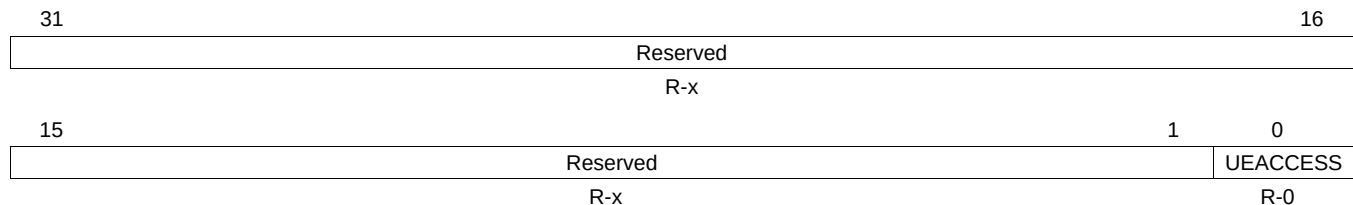
Table 32-26. Host UE Address Register (HOSTUEADDR) Field Descriptions

Bit	Field	Value	Description
31-0	UE_ADDR	0-FFFF FFFFh	Unrecoverable error address This register captures the physical address of any bus transaction that is started by the USB host controller that encounters an unrecoverable error condition. This information, along with the information in HOSTUESTATUS, can help a developer determine why the USB host issued an access to a physical address that resulted in an unrecoverable error.

32.2.4.25 Host UE Status Register (HOSTUESTATUS)

The host UE status register reports the bus cycle-type for the last unrecoverable error that occurred. This register has no meaning until an unrecoverable error has occurred. It also has no meaning if the USB host controller issues an unrecoverable error because the offset checking fault occurred while processing an isochronous TD. This register is not defined by the OHCI specification.

Figure 32-25. Host UE Status Register (HOSTUESTATUS) [address = FCF78BE4h]



LEGEND: R = Read only; -n = value at reset; -x = value is indeterminate after reset

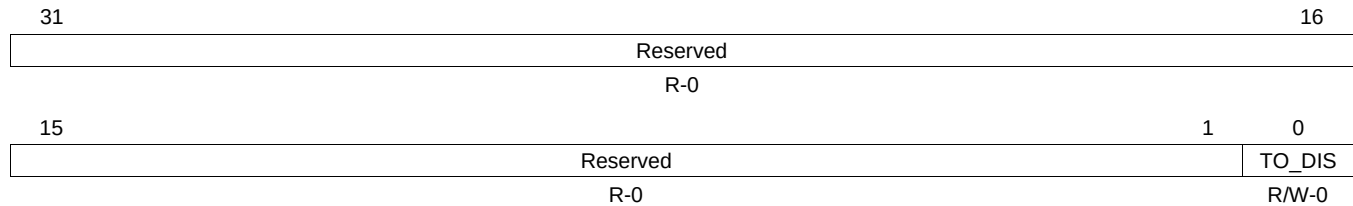
Table 32-27. Host UE Status Register (HOSTUESTATUS) Bit Field Descriptions

Bit	Field	Value	Description
31-1	Reserved	x	Reserved
0	UEACCESS	0-1	Access type when unrecoverable error occurred When an unrecoverable error occurs because of time-out of a bus write, this bit is set. When an unrecoverable error occurs because of time-out of a bus read, this bit is cleared. This bit has no meaning before an unrecoverable error occurs. This information, along with the information in HOSTUEADDR, can help a developer determine why the USB host issued a bus access that resulted in an unrecoverable error.

32.2.4.26 Host Time-out Control Register (HOSTTIMEOUTCTRL)

The host time-out control register controls the USB host controller bus time-out mechanism. This register is not defined by the OHCI specification.

Figure 32-26. Host Time-out Control Register (HOSTTIMEOUTCTRL) [address = FCF78BE8h]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

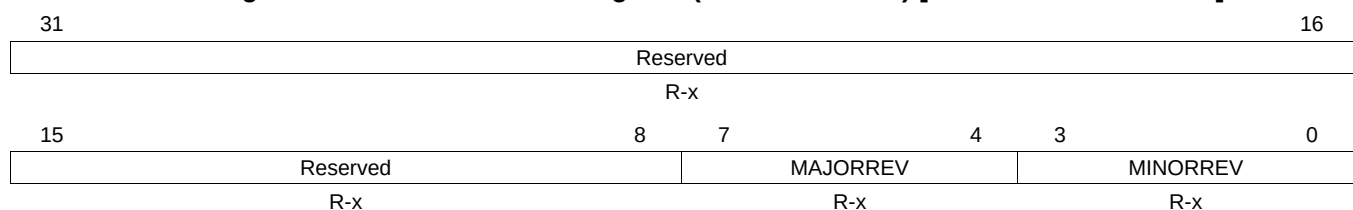
Table 32-28. Host Time-out Control Register (HOSTTIMEOUTCTRL) Bit Field Descriptions

Bit	Field	Value	Description
31-1	Reserved	0	Reserved
0	TO_DIS	0	Bus time-out disable When 0 (the default state), the USB host controller waits no more than 4096 VCLK3 clocks for completion of a bus access to system memory. If the bus cycle does not complete in that time, the USB host controller signals an unrecoverable error.
		1	When 1, the USB host controller bus time-out counter is disabled and the host controller waits indefinitely for completion of a USB host controller access to system memory.

32.2.4.27 Host Revision Register (HOSTREVISION)

The host revision register returns the revision number for the USB host controller. This register is not defined by the OHCI specification.

Figure 32-27. Host Revision Register (HOSTREVISION) [address = FCF78BECh]



LEGEND: R = Read only; -n = value at reset; -x = value is indeterminate after reset

Table 32-29. Host Revision Register (HOSTREVISION) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	x	Reserved
7-4	MAJORREV	0-Fh	Major revision number MAJORREV indicates the major revision number of the USB host controller. The original USB host controller version implements a major revision number of 0.
3-0	MINORREV	0-Fh	Minor revision number MINORREV indicates the minor revision number of the USB host controller. The original USB host controller version implements a minor revision number of 0.

32.2.5 USB Host Controller Reserved Registers and Reserved Bit Fields

To enhance code reusability with possible future versions of the USB host controller, reads and writes to reserved USB host controller register addresses are to be avoided. Unless otherwise specified, when writing registers that have reserved bits, read-modify-write operations must be used so that the reserved bits are written with their previous values.

32.2.6 USB Host Controller Registers, USB Reset, and USB Clocking

The Global Clock Module (GCM) provides VCLK3 to the USB host controller for the bus accesses. The GCM also provides the 48MHz clock (VCLKA3) and 12MHz clock (VCLKA3_DIVR) to the USB host controller. If the application disables the VCLK3 domain or the VCLKA3 domain from the GCM, reads from and writes to the USB host controller registers do not occur correctly. To properly access the USB host controller registers, the USB host controller must be clocked and must be out of reset.

The USB host controller hardware reset is controlled by the system reset signal from the SYS module. The USB host controller is held in reset whenever the system reset signal is low or the HostControllerReset (HCR) field of the HcCommandStatus Register is 1. The USB host controller can transition out of reset whenever the clock is enabled and system reset is high and HcCommandStatus.HCR is 0.

The USB host controller completes its reset within about 72 cycles of the 48-MHz clock after the host controller clock transitions out of reset. After system software turns on the clock to the USB host controller and removes it from reset, it is necessary to wait until the USB host controller internal reset completes. To ensure that the USB host controller has completely reset, system software must wait until reads of both the HCREVISION register and the HCHCCA register return their correct reset default values.

32.2.7 OHCI Interrupts

The USB host controller provides an interrupt output to the Vectored Interrupt Manager (VIM). This is a level-sensitive interrupt signal.

32.2.7.1 OHCI Scheduling Overrun Interrupt

The OHCI scheduling overrun interrupt is supported as described in the *OHCI Specification for USB*.

32.2.7.2 OHCI HCDONEHEAD Writeback Interrupt

The OHCI HCDONEHEAD writeback interrupt is supported as described in the *OHCI Specification for USB*.

32.2.7.3 OHCI Start Of Frame Interrupt

The OHCI start of frame interrupt is supported as described in the *OHCI Specification for USB*.

32.2.7.4 OHCI Resume Detect Interrupt

The OHCI resume detect interrupt is supported as described in the *OHCI Specification for USB*.

32.2.7.5 OHCI Unrecoverable Error Interrupt

The OHCI unrecoverable error interrupt is supported as described in the *OHCI Specification for USB*. This interrupt occurs if the USB host controller is unable to complete a bus read or write within 4096 VCLK3 clocks when the USB host bus time-out feature is enabled. When a bus time-out causes an unrecoverable error, HOSTUEADDR and HOSTUESTATUS are updated. When an isochronous TD is processed with an OFFSET/PSW field that is not set for *Not Accessed*, an unrecoverable error interrupt is generated, but HOSTUEADDR and HOSTUESTATUS are not updated.

32.2.7.6 OHCI Frame Number Overflow

The OHCI frame number overflow interrupt is supported as described in the *OHCI Specification for USB*.

32.2.7.7 OHCI Root Hub Status Change

The OHCI root hub status change interrupt is supported as described in the *OHCI Specification for USB*.

32.2.7.8 OHCI Ownership Change Interrupt

The optional OHCI ownership change interrupt is not supported.

32.2.8 USB Host Controller Access to System Memory

The USB host controller has access to system memory to read and write the OHCI data structures and data buffers associated with USB traffic.

32.2.9 Physical Addressing

All system memory accesses initiated by the USB host controller use physical addresses.

32.2.10 USB Host Controller Bus Addressing and OHCI Data Structure Pointers

The USB host controller OHCI registers that point to the HCCA and the ED lists must be programmed with values that correspond to the physical addresses of the particular data structure. System software must use physical addresses when manipulating the OHCI control registers that point to the HCCA, and to the ED and TD lists. System software must also use physical addresses for the ED and TD pointers that are stored within ED and TD entries.

The USB host controller driver software must also be able to examine the list of completed transfer descriptors that the host controller creates as it retires transfer descriptors. This list is pointed to by the HCDONEHEAD register, which contains a physical address that points to the most recent transfer descriptor that has been retired.

32.2.11 NULL Pointers

The *OHCI Specification for USB* uses NULL pointers to indicate the end of a list. The USB host controller compares the ED and TD pointers against the value 0x00000000 to determine if the pointer is a null pointer. Address conversion routines must understand this usage.

32.2.12 USB Host Controller Power Management

Power management of the USB host controller is limited to disabling the clock to the USB host by clearing UHOST_EN. When UHOST_EN is 0, the USB host controller clocks are disabled and the USB host controller is held in reset. The USB signal multiplexing controlled by register in the I/O Multiplexing Module (IOMM).

When the host controller 48-MHz clock is disabled or UHOST_EN is 0, all USB host controller OHCI registers and the HOSTUEADDR, HOSTUESTATUS, HOSTTIMEOUTCTRL, and HOSTREVISION registers are inaccessible.

32.3 USB Device Controller

The USB device controller supports the implementation of a full-speed device fully compliant with the USB 1.1 standard.

It provides an interface between the host processor and the USB wire and handles USB transactions with minimal CPU software intervention.

The USB device controller module supports one control endpoint (EP0), up to 15 IN endpoints, and up to 15 OUT endpoints. The exact endpoint configuration is software programmable. The specific items of a configuration are for each endpoint, the size in bytes, the direction (IN, OUT), the type (bulk/interrupt or ISO), and the associated number.

The USB device controller module also supports three DMA channels for IN endpoints and three DMA channels for OUT endpoints for either bulk/interrupt or ISO transactions.

32.3.1 USB Device Controller Registers

Table 32-30 lists the USB device controller registers. These registers are aligned at a 16-bit boundary, so that the memory address offsets for the registers are 0, 2h, 4h, 6h, and so on. The registers support 8-bit and 16-bit read/write accesses. A 32-bit read access will return the same contents on the upper 16 bits as on the lower 16 bits. Section 32.3.1.1 through Section 32.3.1.23 describe the register bits.

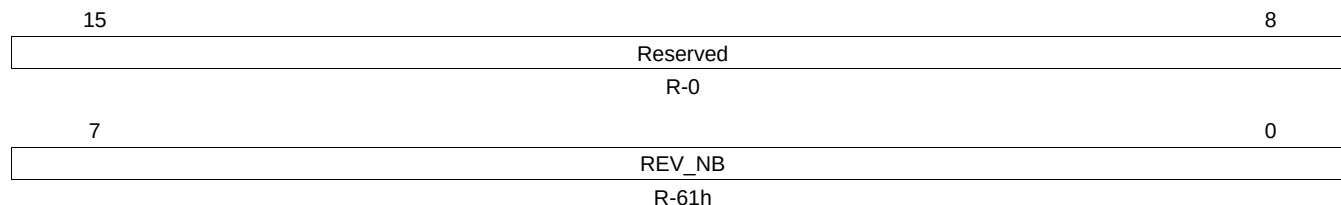
Table 32-30. USB Device Controller Registers

Address	Register	Description	Section
FCF7 8A00h	REV	Revision	Section 32.3.1.1
Endpoint			
FCF7 8A02h	EP_NUM	Endpoint selection	Section 32.3.1.2
FCF7 8A04h	DATA	Data	Section 32.3.1.3
FCF7 8A06h	CTRL	Control	Section 32.3.1.4
FCF7 8A08h	STAT_FLG	Status	Section 32.3.1.5
FCF7 8A0Ah	RXFSTAT	Receive FIFO status	Section 32.3.1.6
FCF7 8A0Ch	SYSCON1	System configuration 1	Section 32.3.1.7
FCF7 8A0Eh	SYSCON2	System configuration 2	Section 32.3.1.8
FCF7 8A10h	DEVSTAT	Device status	Section 32.3.1.9
FCF7 8A12h	SOF	Start of frame	Section 32.3.1.10
FCF7 8A14h	IRQ_EN	Interrupt enable	Section 32.3.1.11
FCF7 8A16h	DMA_IRQ_EN	DMA interrupt enable	Section 32.3.1.12
FCF7 8A18h	IRQ_SRC	Interrupt source	Section 32.3.1.13
FCF7 8A1Ah	EPN_STAT	Non-ISO endpoint interrupt enable	Section 32.3.1.14
FCF7 8A1Ch	DMAN_STAT	Non-ISO DMA interrupt enable	Section 32.3.1.15
DMA Configuration			
FCF7 8A20h	RXDMA_CFG	DMA receive channels configuration	Section 32.3.1.16
FCF7 8A22h	TXDMA_CFG	DMA transmit channels configuration	Section 32.3.1.17
FCF7 8A24h	DATA_DMA	DMA FIFO data	Section 32.3.1.18
FCF7 8A26h	TXDMA0	Transmit DMA control 0	Section 32.3.1.19
FCF7 8A28h	TXDMA1	Transmit DMA control 1	Section 32.3.1.19
FCF7 8A2Ah	TXDMA2	Transmit DMA control 2	Section 32.3.1.19
FCF7 8A30h	RXDMA0	Receive DMA control 0	Section 32.3.1.20
FCF7 8A32h	RXDMA1	Receive DMA control 1	Section 32.3.1.20
FCF7 8A34h	RXDMA2	Receive DMA control 2	Section 32.3.1.20
Endpoint Configuration			
FCF7 8A40h	EP0	Endpoint configuration 0	Section 32.3.1.21
FCF7 8A42h- FCF7 8A5Eh	EP1_RX to EP15_RX	Receive endpoint configuration 1 to 15	Section 32.3.1.22
FCF7 8A62h- FCF7 8A7Eh	EP1_TX to EP15_TX	Transmit endpoint configuration 1 to 15	Section 32.3.1.23

32.3.1.1 Revision Register (REV)

This read-only register contains the revision number of the module. A write to this register is denied.
Device reset and USB controller reset have no effect on this register.

Figure 32-28. Revision Register (REV) [address = FCF78A00h]



LEGEND: R = Read only; -n = value at reset

Table 32-31. Revision Register (REV) Field Descriptions

Bit	Field	Value	Description
15-8	Reserved	0	Reserved. Reads return zeros, writes have no effect.
7-0	REV_NB	61h	This 8-bit field indicates revision number of current USB device controller module—value fixed by hardware: Revision 6.1

32.3.1.2 Endpoint Selection Register (EP_NUM)

This read/write register selects and enables the endpoint that can be accessed by the USB device controller.

Figure 32-29. Endpoint Selection Register (EP_NUM) [address = FCF78A02h]

15															8																											
Reserved																																										
R-0																																										
7							6							5							4							3							0							
Reserved							SETUP_SEL							EP_SEL							EP_DIR							EP_NUM														
R-0							RW-0							RW-0							RW-0							RW-0														

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

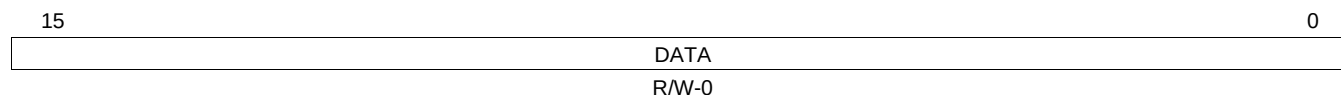
Table 32-32. Endpoint Selection Register (EP_NUM) Field Descriptions

Bit	Field	Value	Description
15-7	Reserved	0	Reserved. Reads return zeros, writes have no effect.
6	SETUP_SEL	0 1	Set by the CPU in response to a Setup general USB Interrupt, to access the EP0 read-only Setup FIFO when reading the DATA register. Setting this bit will clear the IRQ_SRC.Setup bit. When this bit is set, other EP_NUM register bits must be 0. Note: After having read the Setup FIFO, the CPU must clear this bit by writing 0 to it. No access Access permitted The value after system reset or USB reset is low.
5	EP_SEL	0 1	Set by the CPU to access the status (STAT_FLG, RXFSTAT) and data (DATA) registers for the endpoint selected. If the EP_NUM.EP_DIR bit is set to 0, the CPU can read data from endpoint RX FIFO by reading the DATA register. If the EP_NUM.EP_DIR bit is set to 1, the CPU can write data into endpoint TX FIFO by writing into the DATA register. After each access to an endpoint during interrupt handling, the CPU must absolutely clear this bit. Note: When the CPU sets this bit, it must set the SETUP_SEL bit to 0. After having accessed the endpoint's FIFO for either read or write access, the CPU must clear this bit by writing a 0 to it. No access Access permitted. Value after system reset or USB reset is low.
4	EP_DIR	0 1	The endpoint direction bit gives the direction associated with the endpoint number selected in EP_NUM.EP_NUM: 0 OUT endpoint 1 IN endpoint Value after system reset or USB reset is low.
3-0	EP_NUM	0 1h : Fh	The endpoint number binary encoded in these four bits, associated with the direction given by EP_NUM.EP_DIR bit, is the current endpoint selected. All reads and writes to the endpoint status and the control and data locations are for this endpoint. EP0 EP1 : EP15 Value after system reset or USB reset is low.

32.3.1.3 Data Register (DATA)

This register is the entry point to write into a selected TX endpoint or to read data from a selected RX endpoint, or to read data from setup FIFO. If selected endpoint direction is OUT, this register is read-only and a write into it is denied. If selected endpoint direction is IN, this register is write-only and a read to this register is denied.

Figure 32-30. Data Register (DATA) [address = FCF78A04h]



LEGEND: R/W = Read/Write; -n = value after reset

Table 32-33. Data Register (DATA) Field Descriptions

Bit	Field	Description
15-0	DATA	Transmit/receive FIFO data: EP_NUM.EP_DIR = 1: This register contains the data written by the USB device controller to be sent to the USB host during the next IN transaction. Data can be written successfully only if the EP_NUM.EP_SEL bit is asserted. EP_NUM.EP_DIR = 0: This register contains the data received by the USB core from USB host OUT or SETUP transactions. Data can be read successfully only if the EP_NUM.EP_SEL bit is asserted, or if EP_NUM.SETUP_SEL bit is asserted (for setup data).

32.3.1.4 Control Register (CTRL)

This set-only register controls the FIFO and status of the selected endpoint. A read access to this register always returns 0.

Endpoint 0 setup FIFO is always enabled and ready to accept setup data. No control register CTRL is implemented for this FIFO because the USB device controller cannot control it.

Figure 32-31. Control Register (CTRL) [address = FCF78A06h]

15															8																																	
Reserved																																																
R-0																																																
7							6							5							3							2							1							0						
CLR_HALT							SET_HALT							Reserved							SET_FIFO_EN							CLR_EP							RESET_EP													
RW-0							RW-0							R-0							RW-0							RW-0							RW-0													

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-34. Control Register (CTRL) Field Descriptions

Bit	Field	Value	Description
15-8	Reserved	0	Reserved
7	CLR_HALT	0 1	The clear halt endpoint (non-ISO) bit only concerns non-ISO endpoints—used by the USB device controller to clear an endpoint halt condition: No action Clear halt condition Always read 0.
6	SET_HALT	0 1	The set halt endpoint (non-ISO) only concerns non-ISO endpoints—used by the USB device controller to halt the selected endpoint. The halted endpoint returns STALL handshakes to the USB host. The USB device controller can disable the endpoint interrupt if it does not want to be informed of STALL handshakes. Note: If the endpoint to halt is used by a DMA channel, the USB device controller must disable the DMA channel before setting halt condition for this endpoint. No action Halt endpoint Always read 0.
5-3	Reserved	0	Reserved. Reads return zeros, writes have no effect.
2	SET_FIFO_EN	0 1	The set FIFO enable (non-ISO) bit only concerns non-ISO endpoints. If the selected endpoint direction is IN, the USB device controller uses this bit to enable the USB device to transmit data from the FIFO at the next valid IN token. If the selected endpoint direction is OUT, the USB device controller uses this bit to enable the USB device to receive data from the USB host at the next valid OUT transaction. If not, setting the device returns a NAK handshake. ISO endpoints FIFO are always enabled. Note: The USB device controller must never enable endpoint 0 FIFO out of control transfers. For bulk and interrupt endpoints, FIFO must never be enabled when the halt feature is set or when RX FIFO is not empty. Furthermore, during EP interrupts handling, the USB device controller must have cleared the interrupt bit before setting CTRL.SET_FIFO_EN bit (to avoid masked ACK interrupts). No action FIFO enabled Always read 0.
1	CLR_EP	0 1	Clear endpoint: the USB device controller sets this bit to clear the selected endpoint FIFO pointers and flags. This bit resets the FIFO pointers, the FIFO empty status bit is set, and the FIFO enable bit and other FIFO flags are cleared upon completion of the FIFO reset. It also clears the previous transaction handshake status. For ISO endpoints or non-ISO double-buffered endpoints, both foreground and background FIFO are cleared. No action Clear endpoint Always read 0.

Table 32-34. Control Register (CTRL) Field Descriptions (continued)

Bit	Field	Value	Description
0	RESET_EP		The endpoint reset (non-control) bit only concerns non-control endpoints. The USB device controller sets this bit to reset the selected endpoint. It forces an interrupt or a bulk endpoint data PID to DATA0, clears the halt condition, and clears the FIFO (both foreground and background if endpoint is double-buffered) and previous transactions handshake status. For an ISO endpoint, it only clears the FIFO (both foreground and background).
		0	No action
		1	Reset endpoint
			Always read 0.

32.3.1.5 Status Register (STAT_FLG)

This read-only register provides a status of the FIFO and the results of the transactions handshakes for the selected endpoint. The 8 MSB are reserved for ISO endpoints, whereas the 8 LSB are reserved for non-ISO endpoints. This register cannot be read if EP_NUM.EP_SEL bit is not asserted for the endpoint. No status flag exists for the read-only set-up FIFO, which is always enabled.

NOTE: The updates for non-ISO transactions are done at the end of each non-transparent and valid transaction to a given endpoint, if no non-handled interrupt is pending on the endpoint.

The definition of a non-transparent, non-ISO IN transaction is one that responds with an ACK handshake or a STALL handshake, or optionally a NAK handshake if SYSCON1.NAK_EN is asserted to 1. An ERR handshake or a NAK handshake when SYSCON1.NAK_EN is 0 is considered transparent.

A write to this register has no effect.

Figure 32-32. Status Register (STAT_FLG) [address = FCF78A08h]

15	14	13	12	11	10	9	8
NO_RX_PACKET	MISS_IN	DATA_FLUSH	ISO_ERR	Reserved		ISO_FIFO_EMPTY	ISO_FIFO_FULL
R-0	R-0	R-0	R-0	R-0		R-0	R-0
7	6	5	4	3	2	1	0
Reserved	EP_HALTED	STALL	NAK	ACK	FIFO_EN	NON_ISO_FIFO_EMPTY	NON_ISO_FIFO_FULL
R-0	R-0	R-0	R-0	R-0	R-0	R-0	R-0

LEGEND: R = Read only; -n = value at reset

Table 32-35. Status Register (STAT_FLG) Field Descriptions

Bit	Field	Value	Description
15	NO_RX_PACKET		The isochronous no packet received (ISO OUT) bit only concerns ISO OUT endpoints. This bit notifies the USB device controller that the core has not received any isochronous packet on the endpoint. This bit is updated on a SOF (start of frame).
		0	The endpoint received a packet on the previous frame.
		1	The endpoint did not receive a packet on the previous frame.
			Value after system reset or USB reset is high.

Table 32-35. Status Register (STAT_FLG) Field Descriptions (continued)

Bit	Field	Value	Description
14	MISS_IN	0 1	The isochronous missed IN token for the previous frame (ISO IN) bit only concerns ISO IN endpoints. This bit notifies the USB device controller that the core missed a valid ISO IN token during the previous frame and that TX data were flushed from the FIFO instead of being transmitted to the USB host. This bit is updated on a SOF (start of frame). The endpoint received an IN token the previous frame. The endpoint did not receive an IN token the previous frame and TX data were flushed. Value after system reset or USB reset is low.
13	DATA_FLUSH	0 1	The isochronous receive data flush (ISO OUT) bit only concerns ISO OUT endpoints. When set, this bit indicates that data were flushed from the isochronous FIFO that was moved from the foreground to the background. This happens when the USB device controller does not read all of the data from the foreground FIFO in a frame. This bit is updated in every frame. Not significant. Data was flushed. Value after system reset or USB reset is low.
12	ISO_ERR	0 1	The isochronous receive data error (ISO OUT) bit only concerns ISO OUT endpoints. When set, this bit indicates that the ISO data packet was received incorrectly. This happens when the core detects an error in the data packet (CRC, bit stuffing, PID check) or when there is an overrun condition in the FIFO. When this bit is set, the FIFO contents are automatically flushed by the core and the FIFO status is empty. This bit is updated in every frame. Not significant. ISO packet received with errors. Value after system reset or USB reset is low.
11-10	Reserved	0	Reserved
9	ISO_FIFO_EMPTY	0 1	The ISO FIFO empty bit only concerns ISO endpoints. This bit is set when the FIFO for the selected ISO endpoint is empty, either via an appropriate CTRL.CLR_EP bit or CTRL.RESET_EP bit or after successful reads from the selected FIFO. ISO FIFO is not empty. ISO FIFO is empty. Value after system reset or USB reset is high (FIFO empty).
8	ISO_FIFO_FULL	0 1	The ISO FIFO full bit only concerns ISO endpoints. This bit is set when the FIFO for the selected ISO endpoint is full. This condition is cleared by setting the CTRL.CLR_EP bit or CTRL.RESET_EP bit, or after one successful read (by the USB device controller or the USB host). ISO FIFO is not full. ISO FIFO is full. Value after system reset or USB reset is low (FIFO empty).
7	Reserved	0	Reserved
6	EP_HALTED	0 1	The endpoint halted flag (non-ISO) bit only concerns non-ISO endpoints. This bit when asserted 1 indicates that the selected endpoint is halted. The endpoint can be put into the halt state only by the USB device controller writing the endpoint halt control bit (in response to a SET_FEATURE request, for instance). The selected endpoint is not halted. The selected endpoint is halted. Value after system reset or USB reset is low.

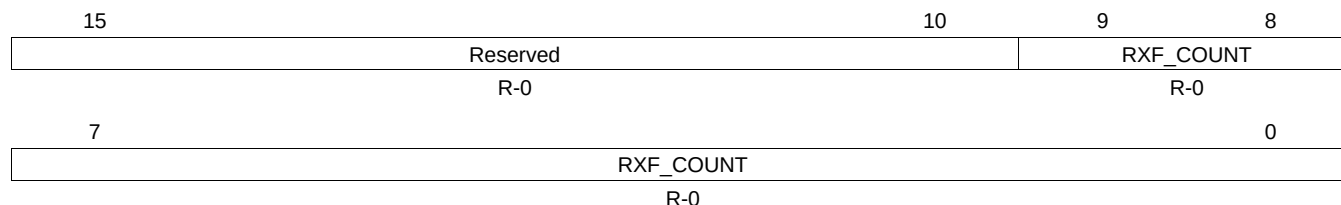
Table 32-35. Status Register (STAT_FLG) Field Descriptions (continued)

Bit	Field	Value	Description
5	STALL	0 1	The transaction stall (non-ISO) bit only concerns non-ISO endpoints. This status bit is set at the end of a transaction if a STALL handshake packet was returned to the USB host, and if no non-handled interrupt is pending on the current buffer (see Section 32.3.11). The core automatically returns a STALL packet if a valid IN token is received by a halted TX endpoint, if a valid OUT transaction is received by a halted RX endpoint, or if there is a request error (endpoint 0). The bit is cleared when the USB device controller has finished handling the corresponding interrupt (at EP_NUM.EP_SEL bit deselection). No STALL handshake was returned. A STALL handshake packet was returned. Value after system reset or USB reset is low.
4	NAK	0 1	The transaction non-acknowledge (non-ISO) bit only concerns non-ISO endpoints with SYSCON1.NAK_EN bit asserted. This status bit is set at the end of a transaction if a NAK handshake is returned to the USB host, and if no non-handled interrupt is pending on the current buffer. The USB core automatically returns a NAK handshake to the USB host if a valid IN token is received by a TX endpoint or if a valid OUT transaction is received by an RX endpoint, and the STAT_FLG.FIFO_EN bit is not set for the endpoint. The bit is cleared when the USB device controller has finished handling the corresponding interrupt (at EP_NUM.EP_SEL bit deselection). No NAK handshake was returned (SYSCON1.NAK_EN bit is set). A NAK handshake packet was returned and SYSCON1.NAK_EN bit is set. Value after system reset or USB reset is low.
3	ACK	0 1	The transaction acknowledge (non-ISO) bit only concerns non-ISO endpoints. This bit is set at the end of a non-transparent valid IN transaction if the data packet was sent successfully to the USB host and the ACK handshake was received, or at the end of a non-transparent valid OUT transaction if the data packet was received successfully by the USB device and the ACK handshake was returned. The bit is cleared when the USB device controller has finished handling the corresponding interrupt (at EP_NUM.EP_SEL bit deselection). No ACK handshake packet was returned. An ACK handshake packet was returned. Value after system reset or USB reset is low.
2	FIFO_EN	0 1	The FIFO enable status (non-ISO) bit only concerns non-ISO endpoints. This bit is asserted when CTRL.SET_FIFO_EN is set to 1 and is cleared automatically after a transaction completes with an ACK, STALL. The non-ISO endpoint FIFO is disabled. The non-ISO endpoint FIFO is enabled. Value after system reset or USB reset is low.
1	NON_ISO_FIFO_EMPTY	0 1	The non-ISO FIFO empty bit only concerns non-ISO endpoints. This bit is set when the FIFO for the selected non-ISO endpoint is empty, either via an appropriate CTRL.CLR_EP bit or CTRL.RESET_EP bit or after successful reads from the selected FIFO. Non-ISO FIFO is not empty. Non-ISO FIFO is empty. Value after system reset or USB reset is high (FIFO empty).
0	NON_ISO_FIFO_FULL	0 1	The non-ISO FIFO full bit only concerns non-ISO endpoints. This bit is set when the FIFO for the selected non-ISO endpoint is full. This condition is cleared by setting the CTRL.CLR_EP bit or CTRL.RESET_EP bit, or after one successful read (by the USB device controller or the USB host). Non-ISO FIFO is not full. Non-ISO FIFO is full. Value after system reset or USB reset is low (FIFO empty).

32.3.1.6 Receive FIFO Status Register (RXFSTAT)

This read-only register indicates how many bytes are in the receive FIFO for the selected endpoint. A write to this register has no effect. The USB device controller cannot read this register if the EP_NUM.EP_SEL bit is not set for the endpoint. No receive FIFO status exists for the setup FIFO, because 8 bytes are always expected.

Figure 32-33. Receive FIFO Status Register (RXFSTAT) [address = FCF78A0Ah]



LEGEND: R = Read only; -n = value at reset

Table 32-36. Receive FIFO Status Register (RXFSTAT) Field Descriptions

Bit	Field	Value	Description
15-10	Reserved	0	Reserved
9-0	RXF_COUNT	0-3FFh	The receive FIFO byte count 10-bit field indicates the number of bytes currently in the receive FIFO. Values after system reset or USB reset is low (all 10 bits).

32.3.1.7 System Configuration Register 1 (SYSCON1)

This read/write register provides control functions for power management and miscellaneous control for the core.

Values depend on whether the reset action comes from the USB device controller (CPU) or the USB host.

Figure 32-34. System Configuration Register 1 (SYSCON1) [address = FCF78A0Ch]

15				9				8
Reserved							CFG_LOCK	
R-0							R/W-0	
7	6	5	4	3	2	1	0	
DATA_ENDIAN	DMA_ENDIAN	Reserved	NAK_EN	AUTODEC_DIS	SELF_PWR	SOFF_DIS	PULLUP_EN	
R/W-0	R/W-0	R-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-37. System Configuration Register 1 (SYSCON1) Field Descriptions

Bit	Field	Value	Description
15-9	Reserved	0	Reserved
8	CFG_LOCK	0 1	Device configuration locked bit: after the USB device controller has entered the device configuration (registers 0x20 to 0x3F), it must set the CFG_LOCK bit so that the device can be used. When the device configuration is not locked, the device is not ready to be used. Device configuration is not locked. Device is not ready Device configuration is locked. The value after the USB device controller hardware reset is low; after USB reset, it is unchanged (keep previous configuration).
7	DATA_ENDIAN	0 1	The data endian bit can be set by the USB device controller to select little- or big-endian format on data access (read or write). See Table 32-38 . Little endian Big endian The value after USB device controller hardware reset is low; after USB reset, it is unchanged (keep previous configuration).
6	DMA_ENDIAN	0 1	The DMA data endian bit can be set by the USB device controller to select little- or big-endian format on data access (read or write). See Table 32-38 . Little endian Big endian The value after USB device controller hardware reset is low; after USB reset, it is unchanged (keep previous configuration).
5	Reserved	0	Reserved
4	NAK_EN	0 1	The NAK enable bit can be set by the USB device controller to be signaled for NAK transaction handshake response. When this bit is set, STAT_FLG.NAK is set on a NAK handshake if no non-handled interrupt is pending on the endpoint, and the endpoint interrupt is asserted. In the normal mode, when cleared, NAK handshake response to the USB host is made transparent to the USB device controller and no interrupt is asserted. NAK is disabled. NAK is enabled. The value after system reset or USB reset is low. Note: If the USB device controller sets this bit, it must wait for a NAK interrupt before selecting TX endpoint to write TX data.
3	AUTODEC_DIS	0 1	The autodecode process disabled can be set to force software to handle all EP0 transactions (except the SET_ADDRESS transaction). Autodecode process is activated (see Table 32-55). Autodecode process is deactivated. The value after USB device controller hardware reset is low; after USB reset, it is unchanged.

Table 32-37. System Configuration Register 1 (SYSCON1) Field Descriptions (continued)

Bit	Field	Value	Description
2	SELF_PWR	0 Bus powered 1 Self-powered The value after USB device controller hardware reset is low; after USB reset, it is unchanged.	The self-powered bit indicates to the USB host whether the device is bus-powered or self-powered. This information is needed for an autodecoded request. The USB device controller must update this bit after a SET_CONFIGURATION according to the self-powered bit D6 given in the configuration descriptor (see <i>USB 1.1 Specification</i>).
1	SOFF_DIS	0 Power shutoff circuitry is enabled. 1 Power shutoff circuitry is disabled. The value after USB device controller hardware reset is low; after USB reset, it is unchanged.	When the shutoff disable bit is set, it disables the power shutoff circuitry.
0	PULLUP_EN	0 Pull-up is disabled. USB host cannot detect the device. In this mode, the 48-MHz USB clock is forced off. 1 Pull-up is enabled. The value after USB device controller hardware reset is low; after USB reset, it is high; and after detach, it is unchanged.	The external pullup enable bit allows the device to disconnect itself from the USB bus, forcing the host to reset and reconfigure the device. This bit can be used to prevent USB traffic when the device is not ready.

Table 32-38. Little-Endian and Big-Endian Formats
Little-Endian Format (16 bits):

15 8 7

Byte 1	Byte 0		Byte 0	Byte 1
--------	--------	--	--------	--------

Big-Endian Format (16 bits):

15 8 7 0

	Byte 0		Byte 1	
--	--------	--	--------	--

32.3.1.8 System Configuration Register 2 (SYSCON2)

This set-only register provides miscellaneous controls for the function. A read to this register always returns 0.

Figure 32-35. System Configuration Register 2 (SYSCON2) [address = FCF78A0Eh]

15							8	
Reserved								
R-0								
7	6	5	4	3	2	1	0	
Reserved	RMT_WKP	STALL_CMD	Reserved	DEV_CFG	CLR_CFG	Reserved		
R-0	W-0	W-0	R-0	W-0	W-0	R-0		

LEGEND: R = Read only; W = Write only; -n = value at reset

Table 32-39. System Configuration Register 2 (SYSCON2) Field Descriptions

Bit	Field	Value	Description
15-7	Reserved	0	Reserved
6	RMT_WKP	0 No action. 1 Initiates the remote wake-up sequence. Always read 0.	The set-only remote wake-up bit, when written with a 1, initiates the remote wake-up sequence even if DEVSTAT.R_WK_OK bit was not previously set to 1 by the USB host. Reading this bit always returns 0. Writing 0 into this bit has no effect. To generate a resume, the software must check the remote wake-up enable value before initiating any wake-up sequence.
5	STALL_CMD	0 No action. 1 Stall current USB command. Always read 0.	The set-only stall command bit only concerns non-autodecoded requests on control endpoint (EP0). This is asserted in response to a USB command where either the command itself or its data is invalid. Asserting this bit forces the non-autodecoded command to complete with a STALL handshake. It has no effect for autodecoded requests.
4	Reserved	0	Reserved
3	DEV_CFG	0 No action. 1 Allows DEVSTAT.CFG to be set. Always read 0.	If the USB device controller receives a SET_CONFIGURATION with a valid configuration value, and the device is in addressed state, it must write a 1 to the device configured (DEV_CFG) bit to inform the command decode that the device moves to configured state. The core sets the DEVSTAT.CFG bit to 1. If the device is already configured when the SET_CONFIGURATION request is received, the USB device controller does not have to set this bit. If the new configuration value is 0, USB device controller must set the SYSCON2.CLR_CFG bit to move to addressed state. Reading this bit always returns 0. Writing 0 into this bit has no effect.
2	CLR_CFG	0 No action. 1 Allows DEVSTAT.CFG to be cleared. Always read 0.	If the USB device controller receives a SET_CONFIGURATION with a configuration value of 0, and if the device is configured, it must write a 1 to the clear configured (CLR_CFG) bit to inform the command decode that the device becomes deconfigured (moves to addressed state). The core clears the DEVSTAT.CFG bit. Reading this bit always returns 0. Writing 0 into this bit has no effect.
1-0	Reserved	0	Reserved

32.3.1.9 Device Status Register (DEVSTAT)

This read-only register provides a status reflecting the visible device states as defined in *USB1.1 Specification*. A write to this register has no effect.

NOTE: This register is double buffered. If IRQ_EN.DS_CHG_IE is set (interrupt enabled), the background register is moved into foreground position only after clearing any pending DS_CHG interrupt. Therefore, if there is a state change and a pending DS_CHG interrupt has not been serviced yet, then the recent state change is not visible, because the background register was only updated and not moved into the foreground position.

The values depend on whether the reset action comes from the USB device controller (CPU) or USB host. They also depend on whether the device is attached or not when the USB device controller hardware reset event occurs.

Figure 32-36. Device Status Register (DEVSTAT) [address = FCF78A10h]

15				10		9	8
Reserved						B_HNP_ENABLE	A_HNP_SUPPORT
R-0						R-0	R-0
7	6	5	4	3	2	1	0
A_ALT_HNP_SUPPORT	R_WK_OK	USB_RESET	SUS	CFG	ADD	DEF	ATT
R-0	R-0	R-0	R-0	R-0	R-0	R-0	R-0

LEGEND: R = Read only; -n = value at reset

Table 32-40. Device Status Register (DEVSTAT) Field Descriptions

Bit	Field	Value	Description
15-10	Reserved	0	Reserved
9	B_HNP_ENABLE	0 1	The HNP enable for B-device bit is used for On-The-Go feature selection purposes. See the <i>On-The-Go Supplement</i> to the <i>USB 2.0 Specification</i> . Not significant. HNP enable for B-device. Value after system reset or USB reset is low.
8	A_HNP_SUPPORT	0 1	The B-device is directly connected to an A-device port that supports HNP. This A_HNP_SUPPORT bit is used for On-The-Go feature selection purposes. See the <i>On-The-Go Supplement</i> to the <i>USB 2.0 Specification</i> . Not significant. B-device connection to HNP capable A-device. Value after system reset or USB reset is low.
7	A_ALT_HNP_SUPPORT	0 1	The B-device is connected to an A-device port that is not capable of HNP, but the A-device has an alternate port that is capable of HNP. This A_ALT_HNP_SUPPORT bit is used for On-The-Go feature selection purposes. See the <i>On-The-Go Supplement</i> to the <i>USB 2.0 Specification</i> . Not significant. B-device connection to not HNP capable A-device. Value after system reset or USB reset is low.
6	R_WK_OK	0 1	The remote wake-up granted bit is automatically set and cleared when the core receives a valid set/clear device feature request from the USB host. It returns a 1 when the USB host grants the function the ability to assert remote wake-up. Not significant. Remote wake-up granted. Value after system reset or USB reset is low.

Table 32-40. Device Status Register (DEVSTAT) Field Descriptions (continued)

Bit	Field	Value	Description
5	USB_RESET	0 1	USB reset signaling is active. The USB_RESET bit returns 1 when the USB host resets the USB bus. A valid USB reset resets the endpoint FIFOs, the other control register bits, except for SYSCON1.CFG_LOCK and the associated configuration registers (0x20 to 0x3F), IRQ_EN.DS_CHG_IE and IRQ_SRC.DS_CHG, and forces the device to the default state for DEVSTAT (this register). This bit is cleared at the end of reset. This bit, as well as other DEVSTAT bits, is double-buffered. If a pending interrupt has not been handled when a USB reset occurs and is handled only when USB reset is finished, the USB device controller does not see the USB_RESET bit going high and then low. Device is not being reset by USB host. Device is being reset by USB host. The value after the USB device controller hardware reset is low, and during USB reset is high (low after USB reset).
4	SUS	0 1	Suspended state bit: the device in suspended state is, at a minimum, attached to the USB, powered, has been reset by the USB host, and has not seen bus activity for 5 ms. It can also have a unique address and be configured for use. Because the device is suspended, however, the host cannot use the device function. This bit returns 1 when the USB device is in a suspended state. Not suspended Suspended Value after system reset or USB reset is low.
3	CFG	0 1	Configured state bit: the device is attached to the USB, powered, has been reset, has a unique address, and is configured. The host can use the function provided by the device. This bit returns 1 when the USB device has been configured after a set SYSCON2.DEV_CFG = 1. This bit remains set to 1 until the device becomes deconfigured. This bit is cleared when the core receives a valid SET_CONFIGURATION request and the USB device controller sets the SYSCON2.CLR_CFG bit. During the time this bit is not set to 1, transactions that are not for control EP0 are ignored. A GET_ENDPOINT_STATUS to a non-control endpoint is stalled. Not configured Configured Value after system reset or USB reset is low.
2	ADD	0 1	Addressed state bit: the device is attached to the USB, powered, has been reset, and a unique device address has been assigned. This bit (ADD) returns 1 after an ET_ADDRESS standard request. This bit remains set to 1 until the device becomes de-addressed. Not addressed Addressed Value after system reset or USB reset is low.
1	DEF	0 1	The default state bit returns 1 when the USB device is attached to the USB and powered, and has been reset. This bit remains set to 1 until the device becomes de-powered. The device moves into default state as soon as the USB reset is effective. Not in default Default The value after CPU is low, and after USB reset is high.
0	ATT	0 1	The attached state bit returns 1 when the device is attached to the USB and is powered. This bit remains set to 1 until the device powers down. Not attached Attached The value after USB device controller hardware reset is low (unattached) or high (attached), and after USB reset is high.

32.3.1.10 Start of Frame Register (SOF)

This read-only register provides a frame timer status for use in ISO communications. A write to this register is denied.

Figure 32-37. Start of Frame Register (SOF) [address = FCF78A12h]

15	13	12	11	10	8
Reserved		FT_LOCK	TS_OK	TS	
R-0		R-0	R-0	R-0	
7					0
TS					
R-0					

LEGEND: R = Read only; -n = value at reset

Table 32-41. Start of Frame Register (SOF) Field Descriptions

Bit	Field	Value	Description
15-13	Reserved	0	Reserved
12	FT_LOCK	0 1	Frame timer locked bit: the USB host sends out a start of frame (SOF) packet every millisecond. When the device does not receive a SOF packet because of a bus error, a local start-of-frame that is used for ISO FIFO switch is generated. Once the core receives two valid SOF, separated by TF (time frame), it sets SOF.FT_LOCK to 1, only if TF is in frame interval IF allowed by the USB 1.1 specification (IF = [11964:12036] USB bit time). If TF is out of this interval, SOF.FT_LOCK value remains 0, and a local SOF is generated by the core. When SOF.FT_LOCK is set and the frame timer is locked to the timing TF, a local SOF is generated, if no valid SOF has been received in an interval of TF since the last valid SOF. The SOF.FT_LOCK bit is cleared if a valid SOF is received out of the interval IF. If the core receives a valid SOF in this interval, the frame timer locks to the new frame time. If the core does not receive a valid SOF, the frame timer remains locked to TF. When SOF.FT_LOCK is cleared, a local SOF is generated after the 12036 USB bit time, if no valid SOF has been received, and SOF.FT_LOCK remains 0. 0 Frame timer is not locked. 1 Frame timer is locked. The value after system reset or USB reset is low. Note: Each time the core receives a valid SOF out of the allowed interval IF, a local SOF is generated and the ISO FIFO switches.
11	TS_OK	0 1	The timestamp OK bit indicates that the timestamp in SOF.TS is valid for the current frame. It returns a 1 if a valid SOF packet was received from the USB host and a 0 otherwise. 0 Timestamp is invalid. 1 Timestamp is valid. Value after system reset or USB reset is low.
10-0	TS	0-7FFh	The timestamp number field returns the timestamp from the last USB host-valid SOF packet. The frame number is valid if SOF.TS_OK is 1. In case of an SOF miss, this value is not updated and TS_OK is cleared. The value after system reset or USB reset is low (all 11 bits).

32.3.1.11 Interrupt Enable Register (IRQ_EN)

This read/write register enables all non-DMA interrupts (control, state changed, ISO, and non-ISO).

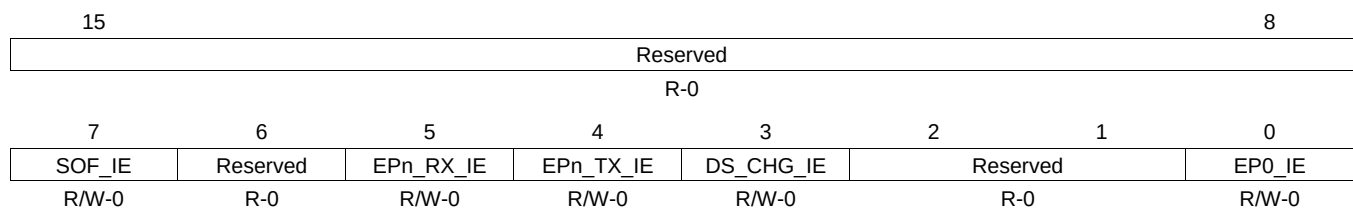
The values depend on whether the reset action comes from the USB device controller (CPU) or the USB host.

When the USB device controller sets a bit position to 1, an interrupt is signaled to the USB device controller if the corresponding IRQ_SRC bit is asserted to 1 by the core for any IRQ_SRC bit controlled by this bit. If reset to 0, the interrupt is masked and not signaled to the USB device controller.

- 0: Interrupt is disabled.
- 1: Interrupt is enabled.

The value after system reset or USB reset is low for all bits except IRQ_EN.DS_CHG_IE bit, which remains unchanged after a reset from the USB host.

Figure 32-38. Interrupt Enable Register (IRQ_EN) [address = FCF78A14h]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-42. Interrupt Enable Register (IRQ_EN) Field Descriptions

Bit	Field	Value	Description
15-8	Reserved	0	Reserved
7	SOF_IE	0-1	Start of frame interrupt enable
6	Reserved	0	Reserved
5	EPn_RX_IE	0-1	Receive endpoint n interrupt enable (non-ISO)
4	EPn_TX_IE	0-1	Transmit endpoint n interrupt enable (non-ISO)
3	DS_CHG_IE	0-1	Device state changed interrupt enable
2-1	Reserved	0	Reserved
0	EP0_IE	0-1	EP0 transactions interrupt enable

32.3.1.12 DMA Interrupt Enable Register (DMA_IRQ_EN)

This read/write register enables all DMA interrupts.

When the USB device controller sets a bit position to 1, an interrupt is signaled to the USB device controller if the corresponding bit in the IRQ_SRC register is asserted to 1 by the core. If reset to 0, the interrupt is masked and not signaled to the USB device controller.

- 0: Interrupt is disabled.
- 1: Interrupt is enabled.

The value after system reset or USB reset is low for all bits.

Figure 32-39. DMA Interrupt Enable Register (DMA_IRQ_EN) [address = FCF78A16h]

15					11		10	9	8
Reserved							TX2_DONE_IE	RX2_CNT_IE	RX2_EOT_IE
R-0							R/W-0	R/W-0	R/W-0
7	6	5	4	3	2	1	0		
Reserved	TX1_DONE_IE	RX1_CNT_IE	RX1_EOT_IE	Reserved	TX0_DONE_IE	RX0_CNT_IE	RX0_EOT_IE		
R-0	R/W-0	R/W-0	R/W-0	R-0	R/W-0	R/W-0	R/W-0		

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-43. DMA Interrupt Enable Register (DMA_IRQ_EN) Field Descriptions

Bit	Field	Value	Description
15-11	Reserved	0	Reserved
10	TX2_DONE_IE	0-1	Transmit DMA channel 2 done interrupt enable
9	RX2_CNT_IE	0-1	Receive DMA channel 2 transactions count interrupt enable
8	RX2_EOT_IE	0-1	Receive DMA channel 2 end of transfer interrupt enable
7	Reserved	0	Reserved
6	TX1_DONE_IE	0-1	Transmit DMA channel 1 done interrupt enable
5	RX1_CNT_IE	0-1	Receive DMA channel 1 transactions count interrupt enable
4	RX1_EOT_IE	0-1	Receive DMA channel 1 end of transfer interrupt enable
3	Reserved	0	Reserved
2	TX0_DONE_IE	0-1	Transmit DMA channel 0 done interrupt enable
1	RX0_CNT_IE	0-1	Receive DMA channel 0 transactions count interrupt enable
0	RX0_EOT_IE	0-1	Receive DMA channel 0 end of transfer interrupt enable

32.3.1.13 Interrupt Source Register (IRQ_SRC)

This read/clear only register identifies and clears the source of the interrupt signaled by a set flag.

The value depends on whether the reset action comes from the USB device controller (CPU) or the USB host.

The USB device controller can clear a set bit location only by writing a 1 into the bit location (except for the setup bit, which is automatically cleared by the core). A write of 0 has no effect.

When the core sets a bit location to 1, an interrupt is signaled to the USB device controller if the interrupt was enabled.

- 0: No interrupt
- 1: Interrupt signaled

The value after the system reset or USB reset is low except for the IRQ_SRC.DS_CHG bit, which is high after a USB reset.

Figure 32-40. Interrupt Source Register (IRQ_SRC) [address = FCF78A18h]

15				11		10	9	8
Reserved						TXn_DONE	RXn_CNT	RXn_EOT
R-0						R-0	R-0	R-0
7	6	5	4	3	2	1	0	
SOF	Reserved	EPn_RX	EPn_TX	DS_CHG	SETUP	EP0_RX	EP0_TX	
R-0	R-0	R-0	R-0	R-0	R-0	R-0	R-0	R-0

LEGEND: R = Read only; -n = value at reset

Table 32-44. Interrupt Source Register (IRQ_SRC) Field Descriptions

Bit	Field	Value	Description
15-11	Reserved	0	Reserved
10	TXn_DONE	0 1	The transmit DMA channel n done interrupt flag (non-ISO) bit is only for non-ISO DMA transfer. This bit is never set for ISO DMA transfer. The core automatically sets this bit when a transmit DMA channel has completed the programmed transfer by servicing the last IN transaction from the USB host. This is when TXDMA.TXn_TSC (transfer size counter) equals 0, and the last IN transaction completes with an ACK. When this bit is asserted, the USB device controller must read the DMAN_STAT register to identify the endpoint number for which the transfer completed. The endpoint interrupt IRQ_SRC.EPn_TX is never set for the assigned endpoint to TX DMA channel n. 0 No action. 1 Non-ISO transmit DMA transfer for a channel has ended. The value after system reset or USB reset is low.
9	RXn_CNT	0 1	The receive DMA channel n transactions count interrupt flag (non-ISO) bit is only for non-ISO DMA transfer. This bit is never set for ISO DMA transfer and never set if the interrupt enable bit associated with it is not set. It means that no poll operation is possible on the DMA. The core automatically sets this bit during an active Receive DMA transfer each time RXDMA.RXn_TC equals 0 after an OUT transaction with ACK status. This bit is set after the RX DMA data have been read (end of DMA request). When this bit is asserted, the USB device controller must read DMAN_STAT register to identify the endpoint number for which the transfer completed. An RXn_CNT IT is asserted also if RXDMA.RXn_STOP is set; in this case, both IRQ_SRC.RXn_EOT and IRQ_SRC.RXn_CNT are asserted. 0 No action. 1 Non-ISO receive DMA transfer for a channel has reached transactions count level. The value after system reset or USB reset is low.

Table 32-44. Interrupt Source Register (IRQ_SRC) Field Descriptions (continued)

Bit	Field	Value	Description
8	RXn_EOT	0 No action. 1 Non-ISO receive DMA transfer for a channel has ended. The value after system reset or USB reset is low.	The receive DMA channel n end of transfer interrupt flag (non-ISO) bit is for non-ISO DMA transfer only. This bit is never set for ISO DMA transfer and never set if the interrupt enable bit associated with it is not set. It means that no poll operation is possible on the DMA. The core automatically sets this bit when a receive DMA channel detects an end-of-transfer (EOT) packet during the last OUT transaction from the USB host. This bit is set after the RX DMA data have been read (end of DMA request). When this happens, the DMA assigned endpoint FIFO is kept disabled (STAT_FLG.FIFO_EN = 0) to avoid receiving a new packet data from the USB host. The USB device controller can grant DMA transfer again to the same endpoint by enabling the FIFO again (STAT_FLG.FIFO_EN = 1). An end of transfer is detected when the core receives a data packet that is less than the configured endpoint FIFO size (or is empty), or when RXDMA_n.RXn_TC equals 0 after an OUT transaction with ACK status and the RXDMA_n.RXn_STOP bit is set. When this bit is asserted, the USB device controller must read DMAN_STAT.DMAN_RX_IT_SRC to identify the endpoint number for which the transfer completed, and DMAN_STAT.DMAN_RX_SB must be informed of an odd number of bytes received during the last transaction (useful for 16-bit read access from the DATA_DMA register). The endpoint interrupt IRQ_SRC.EPn_RX is never set to RX DMA channel for the assigned endpoint.
7	SOF	0 No action 1 Start-of-frame packet received (or internal SOF) The value after system reset or USB reset is low.	Start of frame interrupt flag bit: every millisecond, the USB host outputs a start-of-frame packet to the functions. The SOF bit reflects when a new SOF is received. Writing a 1 in the SOF bit location clears the flag. Writing a 0 has no effect. In accordance with the USB1.1 specification, if SOF is received corrupted or is not received, the core still sets this flag at the same rate (if bit SOF.FT_LOCK = 1) or after 12043 USB bit time (if bit SOF.FT_LOCK = 0).
6	Reserved	0	Reserved
5	EPn_RX	0 No action. 1 OUT transaction detected on an endpoint. The value after system reset or USB reset is low.	The EPn OUT transactions interrupt flag bit only concerns non-ISO endpoints. The core automatically sets the EPn_RX bit when a handshake sequence occurs for an OUT transaction to an interrupt or bulk endpoint (NAK with SYSCON1.NAK_EN set, ACK or STALL). The USB device controller must read the EPn_STAT register to identify the endpoint causing the interrupt.
4	EPn_TX	0 No action 1 IN transaction detected on an endpoint value after system reset or USB reset is low.	The EPn IN transactions interrupt flag bit only concerns non-ISO endpoints. The core automatically sets this bit when a handshake sequence occurs for an IN transaction to an interrupt or bulk endpoint (NAK with SYSCON1.NAK_EN set, ACK or STALL). The USB device controller must read the EPn_STAT register to identify the endpoint causing the interrupt.
3	DS_CHG	0 No action. 1 Device state change detected value after USB device controller hardware reset is low and after USB reset is high.	Device state changed interrupt flag bit: the core automatically resets the DS_CHG bit when the state of the device changes. This is when the core modifies any of the bits present in the DEVSTAT register. When this bit is cleared, the background DEVSTAT register moves into foreground position.
2	SETUP	0 No action. 1 Valid setup transaction occurred on endpoint 0. Value after system reset or USB reset is low.	Setup transaction interrupt flag bit: the core automatically sets the SETUP bit when a valid setup transaction completes on control endpoint for a non-autodecoded control request and automatically clears it when the USB device controller sets EP_NUM.SETUP_SEL bit when reading setup data. A write 1 to it has no effect.

Table 32-44. Interrupt Source Register (IRQ_SRC) Field Descriptions (continued)

Bit	Field	Value	Description
1	EP0_RX	0 1	EP0 OUT transactions interrupt flag bit: the core automatically sets the EP0_RX bit when a handshake sequence occurs for a non-autodecoded OUT transaction to control endpoint (NAK with SYSCON1.NAK_EN set, ACK or STALL). No action. OUT transaction on EP0. Value after system reset or USB reset is low.
0	EP0_TX	0 1	EP0 IN transactions interrupt flag bit: the core automatically sets this bit when a handshake sequence occurs for a non-autodecoded IN transaction to control endpoint (NAK with SYSCON1.NAK_EN set, ACK or STALL). No action. IN transaction on EP0. Value after system reset or USB reset is low.

32.3.1.14 Non-ISO Endpoint Interrupt Status Register (EPN_STAT)

This read-only register identifies the non-ISO endpoint causing an EPn interrupt. A write into it is denied.

NOTE: If a nontransparent transaction occurs before a previous one on another endpoint in the same direction that has been handled, the second interrupt is asserted only after the USB device controller clears the first one and EPN_STAT is updated with the corresponding interrupt assertion.

Figure 32-41. Non-ISO Endpoint Interrupt Status Register (EPN_STAT) [address = FCF78A1Ah]

15	12	11	8
Reserved		EPn_RX_IT_SRC	
R-0		R-0	
7	4	3	0
Reserved		EPn_TX_IT_SRC	
R-0		R-0	

LEGEND: R = Read only; -n = value at reset

Table 32-45. Non-ISO Endpoint Interrupt Status Register (EPN_STAT) Field Descriptions

Bit	Field	Value	Description
15-12	Reserved	0	Reserved
11-8	EPn_RX_IT_SRC	0 1h : Fh	The receive endpoint interrupt source (non-ISO) bit only concerns non-ISO endpoints. When the IRQ_SRC.EPn_RX flag is set, the endpoint causing the interrupt condition is encoded in these four register bits. When the IRQ_SRC.EPn_RX flag is cleared, the four bits read as 0. No receive endpoint interrupt is pending. EP1 : EP15 Values after system reset or USB reset are low (all four bits).
7-4	Reserved	0	Reserved
3-0	EPn_TX_IT_SRC	0 1h : Fh	Transmit endpoint interrupt source (non-ISO) bit only concerns non-ISO endpoints. When the IRQ_SRC.EPn_TX flag is set, the endpoint that causes this flag to be set is encoded in these four register bits. When the IRQ_SRC.EPn_TX flag is cleared, the four bits read as 0. No transmit endpoint interrupt is pending. EP1 : EP15 Values after system reset or USB reset are low (all 4 bits).

32.3.1.15 Non-ISO DMA Interrupt Status Register (DMAN_STAT)

This read-only register identifies the endpoint that causes a DMAn interrupt. A write into it is denied.

NOTE: If a DMA interrupt occurs before a previous one on another endpoint in the same direction that has been handled, the second interrupt is asserted only after the USB device controller clears the first one. DMAn_STAT is updated when the corresponding interrupt is asserted.

Figure 32-42. Non-ISO DMA Interrupt Status Register (DMAN_STAT) [address = FCF78A1Ch]

15	13	12	11	8
Reserved		DMAn_RX_SB	DMAn_RX_IT_SRC	
R-0		R-0	R-0	
7	4	3	0	
Reserved		DMAn_TX_IT_SRC		
R-0		R-0		

LEGEND: R = Read only; -n = value at reset

Table 32-46. Non-ISO DMA Interrupt Status Register (DMAN_STAT) Field Descriptions

Bit	Field	Value	Description
15-13	Reserved	0	Reserved
12	DMAn_RX_SB	0 1	The DMA receive single byte (non-ISO) bit only concerns non-ISO endpoints (ISO endpoints receive a constant number of bytes). This bit is set when an IRQ_SRC.RXn_EOT interrupt is asserted and the core received an odd number of bytes during the last transaction. It is used to know the exact number of bytes received in case of a 16-bit read access from DATA_DMA register. When the IRQ_SRC.RXn_EOT flag is cleared, this bit reads as 0. No EOT DMA interrupt is pending or the core received an even number of bytes during the last transaction. An EOT DMA interrupt is pending and an odd number of bytes was received during the last transaction. Value after system reset or USB reset is low.
11-8	DMAn_RX_IT_SRC	0 1h : Fh	The DMA receive interrupt source (non-ISO) bit only concerns non-ISO endpoints. When the IRQ_SRC.EPn_RX flag is set, the endpoint causing this flag to be set is encoded in these four register bits. When the IRQ_SRC.EPn_RX flag is cleared, the four bits read as 0. No receive DMA interrupt is pending. EP1 : EP15 Values after system reset or USB reset are low (all 4 bits).
7-4	Reserved	0	Reserved
3-0	DMAn_TX_IT_SRC	0 1h : Fh	The DMA transmit interrupt source (non-ISO) bit only concerns non-ISO endpoints. When the IRQ_SRC.EPn_TX flag is set, the endpoint that causes this flag to be set is encoded in these four register bits. When the IRQ_SRC.EPn_TX flag is cleared, the four bits read as 0. No transmit DMA interrupt is pending. EP1 : EP15 Values after system reset or USB reset are low (all 4 bits).

32.3.1.16 DMA Receive Channels Configuration Register (RXDMA_CFG)

This read/write register enables the three possible DMA receive channels and selects the endpoint number that is assigned to each of these DMA channels. An endpoint used by an RX DMA channel must have been configured (through register EPn_RX). The RXDMA_CFG register can be filled when SYSCON1.CFG_LOCK is set.

Only one channel is serviced at a time, so it is better to configure ISO endpoints on the first channel (_EP). If the ISO endpoint is configured on channel 2, it can never be serviced with low software and a fast USB host.

The USB device controller transmit endpoint DMA channels 0, 1, and 2 are associated with device controller inputs.

NOTE: System software must not program RXDMAx_EP to endpoint numbers that are not configured as DMA endpoints.

Figure 32-43. DMA Receive Channels Configuration Register (RXDMA_CFG)
[address = FCF78A20h]

15	13	12	11	8
Reserved		RX_REQ	RXDMA2_EP	
R-0		R/W-0	R/W-0	
7	4	3	0	
RXDMA1_EP		RXDMA0_EP		
R/W-0		R/W-0		

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-47. DMA Receive Channels Configuration Register (RXDMA_CFG)
Field Descriptions

Bit	Field	Value	Description
15-13	Reserved	0	Reserved
12	RX_REQ	0 1	The RX DMA request active level or pulse bit allows the RXDMA request to be configurable level or pulse-sensitive. When pulse-sensitive, the request is active during 2 cycles. RX DMA request active level. RX DMA request active pulse. Value after system reset or USB reset is low.
11-8	RXDMA2_EP	0 1h : Fh	Receive endpoint number for DMA channel 2. The endpoint number binary encoded in these four bits is the current receive endpoint selected for DMA channel 2. A zero value indicates that the DMA channel 2 is deactivated. Any other value automatically enables receive DMA transfer for the selected endpoint. Receive DMA channel 2 is deactivated. EP1 : EP15 Values after system reset or USB reset are low (all 4 bits).
7-4	RXDMA1_EP	0 1h : Fh	Receive endpoint number for DMA channel 1. The endpoint number binary encoded in these four bits is the current receive endpoint selected for DMA channel 1. A zero value indicates that the DMA channel 1 is deactivated. Any other value automatically enables receive DMA transfer for the selected endpoint. Receive DMA channel 1 is deactivated. EP1 : EP15 Values after system reset or USB reset are low (all 4 bits).

**Table 32-47. DMA Receive Channels Configuration Register (RXDMA_CFG)
Field Descriptions (continued)**

Bit	Field	Value	Description
3-0	RXDMA0_EP	0 1h : Fh	Receive endpoint number for DMA channel 0. The endpoint number binary encoded in these four bits is the current receive endpoint selected for DMA channel 0. A zero value indicates that the DMA channel 0 is deactivated. Any other value automatically enables receive DMA transfer for the selected endpoint. Receive DMA channel 0 is deactivated. EP1 : EP15 Values after system reset or USB reset are low (all 4 bits).

32.3.1.17 DMA Transmit Channels Configuration Register (TXDMA_CFG)

This read/write register enables the three possible DMA transmit channels and selects the endpoint number that is assigned to each. An endpoint used by a TX DMA channel must have been configured (through register EPn_TX). TXDMA_CFG register can be filled when SYSCON1.CFG_LOCK is set.

Only one channel is serviced at a time, so it is better to configure ISO endpoints on the first channel (TXDMA0_EP). If, the/your ISO endpoint is configured on the channel 2 and it can never be serviced with low software and a fast USB host.

USB device controller transmit endpoint DMA channels 0, 1, and 2 are associated with device controller inputs.

**Figure 32-44. DMA Transmit Channels Configuration Register (TXDMA_CFG)
[address = FCF78A22h]**

15	13	12	11	8
Reserved		TX_REQ	TXDMA2_EP	
R-0		R/W-0	R/W-0	
7	4	3	0	
TXDMA1_EP		TXDMA0_EP		
R/W-0		R/W-0		

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

**Table 32-48. DMA Transmit Channels Configuration Register (TXDMA_CFG)
Field Descriptions**

Bit	Field	Value	Description
15-13	Reserved	0	Reserved
12	TX_REQ	0 1	The TX DMA request active level or pulse bit allows the TXDMA request to be configurable level or pulse-sensitive. When pulse-sensitive, the request is active for two cycles. TX DMA request active level. TX DMA request active pulse value after system reset or USB reset is low.
11-8	TXDMA2_EP	0 1h : Fh	Transmit endpoint number for DMA channel 2: the endpoint number binary encoded in these four bits is the current transmit endpoint selected for DMA channel 2. A zero value indicates that the DMA channel 2 is deactivated. Any other value automatically enables transmit DMA transfer for the selected endpoint. Transmit DMA channel 2 is deactivated. EP1 : EP15 Values after system reset or USB reset are low (all four bits).

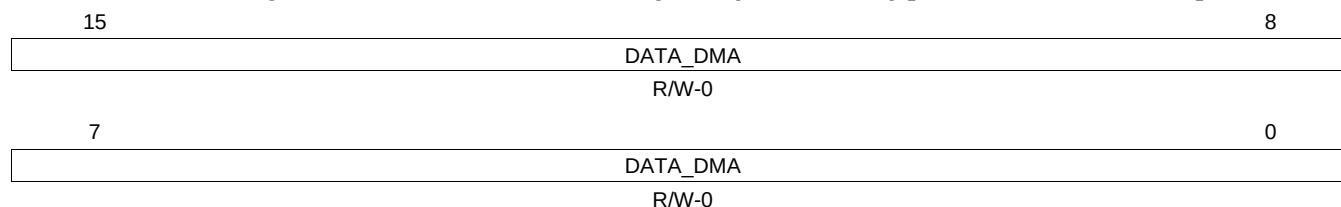
**Table 32-48. DMA Transmit Channels Configuration Register (TXDMA_CFG)
Field Descriptions (continued)**

Bit	Field	Value	Description
7-4	TXDMA1_EP	0 1h : Fh	Transmit endpoint number for DMA channel 1: the endpoint number binary encoded in these four bits is the current transmit endpoint selected for DMA channel 1. A zero value indicates that the DMA channel 1 is deactivated. Any other value automatically enables transmit DMA transfer for the selected endpoint. Transmit DMA channel 1 is deactivated. EP1 : EP15 Values after system reset or USB reset are low (all four bits).
3-0	TXDMA0_EP	0 1h : Fh	Transmit endpoint number for DMA channel 0: the endpoint number binary encoded in these four bits is the current transmit endpoint selected for DMA channel 0. A zero value indicates that the DMA channel 0 is deactivated. Any other value automatically enables transmit DMA transfer for the selected endpoint. Transmit DMA channel 0 is deactivated. EP1 : EP15 Values after system reset or USB reset are low (all four bits).

32.3.1.18 DMA FIFO Data Register (DATA_DMA)

This register is the entry point to write or to read data into/from an endpoint used in a DMA transfer through DMA channel 0, 1 or 2.

Figure 32-45. DMA FIFO Data Register (DATA_DMA) [address = FCF78A24h]



LEGEND: R/W = Read/Write; -n = value at reset

Table 32-49. DMA FIFO Data Register (DATA_DMA) Field Descriptions

Bit	Field	Description
15-0	DATA_DMA	DMA FIFO data. When an RX DMA request is active for a channel (only one active at a given time), this register contains the data that the core received from the USB host OUT transaction using this channel. Data can be accessed by the main DMA controller engine (read access) in response to the DMA request for the channel. When a TX DMA request is active for a channel (only one active at a given time), this register contains the data written by the main DMA controller engine (write access) in response to a DMA request for the transmit channel to be sent to the USB host during the next IN transaction. Note: It is possible for both an RX DMA request and a TX DMA request to be active at the same time. In this case, the main DMA controller engine can access both transmit endpoint and receive endpoint FIFO. A read access to DATA_DMA register affects the endpoint that caused the RX DMA request to be active, and a write access affects the endpoint that caused the TX DMA request to be active. Note: The USB device controller must not access this register directly; however, there is no hardware mechanism to prevent such access. No access into this register must be made out of DMA request handling.

32.3.1.19 Transmit DMA Control Register n (TXDMA_n)

This read/write register controls the operation of the transmit DMA channel n ($n = 0, 1, 2$).

Figure 32-46. Transmit DMA Control Register n (TXDMA_n)
[address = FCF78A28h to FCF78A2Ch]

15	14	13	10	9	8
TXn_EOT	TXn_START	Reserved		TXn_TSC	
R/W-0	R/W-0	R-0		R/W-0	
7					0
TXn_TSC					
R/W-0					

LEGEND: R/W = Read/Write; R = Read only; - n = value at reset

Table 32-50. Transmit DMA Control Register n (TXDMA_n) Field Descriptions

Bit	Field	Value	Description
15	TXn_EOT	0 DMA transfer size is in buffers. 1 DMA transfer size is in bytes. Value after system reset or USB reset is low.	Transmit DMA channel n end of transfer: the TXn_EOT bit can be either 0 or 1 for BULK DMA transfer. When the USB device controller sets it to 1, this bit signals the core that the transfer size set in TXDMA _n .TXn_TSC is in bytes. A TX one interrupt (IRQ_SRC.TXn_DONE) is asserted with the last IN transaction. If the number of bytes set in TXDMA _n .TXn_TSC is a multiple of the endpoint buffer size, the TX done interrupt is asserted only after an IN transaction with an empty data packet. When cleared, the transfer size set in TXn_TSC is in full buffer size for the endpoint selected (BULK only). A TX done interrupt is asserted when the last buffer is sent with the last IN transaction. This mode is to be used for a partial bulk transfer of a large file exceeding 1023 bytes.
14	TXn_START	0 No action. 1 DMA transfer start. Always reads 0.	Transmit DMA channel n start. The USB device controller sets this bit to tell the device that the main DMA system is ready to transmit the number of bytes or buffers. Once set, the DMA transfer cannot be interrupted, unless the USB device controller clears the endpoint in the TXDMA_CFG register. A write 0 into this bit has no effect and a read to this bit always returns 0. The IRQ_SRC.TXn_DONE interrupt bit is asserted when the DMA transfer ends.
13-10	Reserved	0	Reserved
9-0	TXn_TSC	0-3FFh	Transmit DMA channel n transfer size counter. The binary encoded value from 0 to 1023, which the USB device controller writes into this register, corresponds to the number of bytes or number of buffer transfers (function of TXDMA _n .TXn_EOT) to be transmitted by the transmit DMA channel n . When read, the register reflects the number of bytes/buffers the USB device has still to transmit. Read mode is only provided for software debug purposes. Note: For ISO transfer, the user must verify that the set value does not exceed the ISO FIFO size for the endpoint. There is no hardware mechanism to prevent this situation. If this situation occurs, results are unpredictable. Note: For bulk transfer, when TXDMA _n .TXn_EOT = 0, a set value of TXDMA _n .TXn_TSC = 0 means 1024 buffers and not 0. The counter then operates in the following way: 000, 3FF, 3FE, 0001, 000, stop. When TXDMA _n .TXn_EOT = 1, a set value of TXDMA _n .TXn_TSC = 0 a NULL packet is sent in response to the next IN token. Values after system reset or USB reset are low (all 10 bits).

32.3.1.20 Receive DMA Control Register n (RXDMAn)

This read/write register monitors incoming OUT transactions during DMA transfer on channel n (n=0,1,2).

Figure 32-47. Receive DMA Control Register n (RXDMAn)
[address = FCF78A30h to FCF78A34h]

15	14	8
RXn_STOP	Reserved	
R/W-0	R-0	
7	0	
	RXn_TC	
	R/W-0	

LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-51. Receive DMA Control Register n (RXDMAn) Field Descriptions

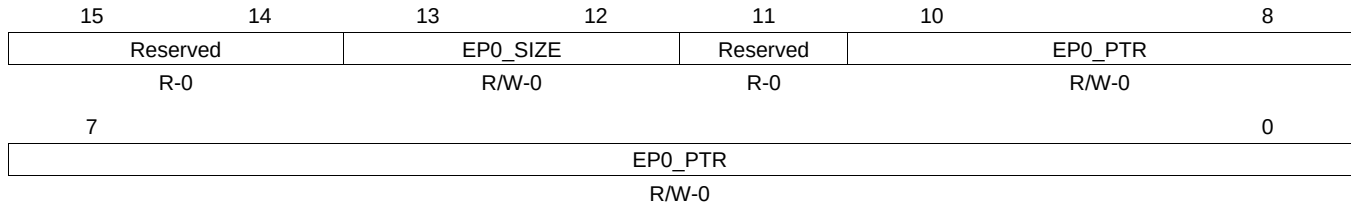
Bit	Field	Value	Description
15	RXn_STOP	0-1	Receive DMA channel n transfer stop. When this bit is set, an RXn_EOT interrupt is asserted to the USB device controller after n OUT transactions where n is the encoded binary value + 1 programmed into RXDMAn.RXn_TC field. This register is used when no smaller than buffer size packet is received at an end-of-transfer (EOT), and the USB device controller expects a given amount of data for the transfer. Note: At end of transfer, the DMA channel is disabled and all OUT transactions received to the assigned endpoint are sent NAK by the core. The USB device controller must set CTRL.SET_FIFO_EN for the endpoint to reenable the channel. Values after system reset or USB reset are low.
14-8	Reserved	0	Reserved
7-0	RXn_TC	0-FFh	Receive DMA channel n transactions count. The USB device controller can ask for an interrupt each n OUT transactions where n is the encoded binary value + 1 programmed into RXDMAn.RXn_TC field. This register must be programmed to the desired transaction watermark limit prior to enabling the DMA transfer for the receive DMA channel n. Note: A reached watermark does not disable an active DMA transfer if RXDMAn.RXn_STOP was not set. If RXDMAn.RXn_STOP was set for the transfer both RXn_CNT and RXn_EOT interrupts are asserted. A read to that register returns the number of transactions remaining before the IRQ_SRC.RXn_CNT interrupt flag is asserted. This read mode is only provided for software debug purposes. Values after system reset or USB reset are low (all 8 bits).

32.3.1.21 Endpoint 0 Configuration Register (EP0)

This read/write register gives the device configuration for control endpoint 0.

Values depend on whether the reset action comes from USB device controller (CPU) or the USB host.

Figure 32-48. Endpoint 0 Configuration Register (EP0) [address = FCF78A40h]



LEGEND: R/W = Read/Write; R = Read only; -n = value at reset

Table 32-52. Endpoint 0 Configuration Register (EP0) Field Descriptions

Bit	Field	Value	Description
15-14	Reserved	0	Reserved
13-12	EP0_SIZE	0 1 2h 3h	Endpoint 0 FIFO size. This field contains the endpoint 0 FIFO size value and must match the value sent by the USB device controller to the USB host during the GET_DEVICE_DESCRIPTOR request preceding configuration phase. Status flags (STAT_FLG.NON_ISO_FIFO_EMPTY and STAT_FLG.NON_ISO_FIFO_FULL) and overrun and underrun conditions are based on this value for all IN and OUT transactions to endpoint 0. The USB device controller must fill this field before setting SYSCON1.CFG_LOCK. 8 bytes 16 bytes 32 bytes 64 bytes Values after USB device controller hardware reset or USB reset are unchanged (which means that value is unknown until first write access).
11	Reserved	0	Reserved
10-0	EP0_PTR	0 1h 2h 3h : FFh	Endpoint 0 pointer. This field contains the address of the endpoint 0 pointer. Value 0 is not permitted (reserved for setup FIFO). Note: The pointer value must be set to a value < FFh, because the memory size is 2K bytes and a pointer coded value = FFh corresponds to 2040 bytes. Addressing upper bytes results in memory overlap. Address = BASE (not permitted) Address = BASE + 8 bytes Address = BASE + 16 bytes Address = BASE + 24 bytes : Address = BASE + 2040 bytes Values after USB device controller hardware reset or USB reset are unchanged (which means that values are unknown until first write access).

32.3.1.22 Receive Endpoint n Configuration Register (EPn_RX)

This read/write register gives the device configuration for non-control receive endpoint n (1:15). The endpoints size fields must match values sent by the USB device controller to the USB host in response to the GET_CONFIGURATION_DESCRIPTOR during configuration phase.

The USB device controller must fill this field before setting SYSCON1.CFG_LOCK and must not change the values once SYSCON1.CFG_LOCK is set.

Values depend on whether the reset action comes from USB device controller (CPU) or USB host.

Figure 32-49. Receive Endpoint n Configuration Register (EPn_RX)
[address = FCF78A42h to FCF78A5Eh]

15	14	13	12	11	10	8
EPn_RX_VALID	EPn_RX_SIZE_DB	EPn_RX_SIZE	EPn_RX_ISO	EPn_RX_PTR		
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	
7						0
EPn_RX_PTR						
R/W-0						

LEGEND: R/W = Read/Write; -n = value at reset

Table 32-53. Receive Endpoint n Configuration Register (EPn_RX) Field Descriptions

Bit	Field	Value	Description
15	EPn_RX_VALID	0 1	The receive endpoint n valid bit must be set by the USB device controller to allow receive endpoint n to be used for USB transfers as part of the device configuration. If not set, all transactions to this endpoint are ignored by the core. Receive endpoint n does not exist for this configuration. Receive endpoint n is part of the device configuration. Values after USB device controller hardware reset are low, after USB reset is unchanged.
14	EPn_RX_SIZE_DB	0 1	Receive non-ISO (or ISO) endpoint n double-buffer (DB). This bit is only for non-ISO endpoints. For ISO endpoints, which are always double-buffered, this bit is endpoint size MSB. This bit must be set by the USB device controller to allow double buffering for receive non-ISO endpoint n. This is used to reduce number of transactions resulting in NAK handshake. No double buffer for non-ISO receive endpoint n. Double buffer used for non-ISO receive endpoint n. Values after USB device controller hardware reset or USB reset are unchanged (which means that values are unknown until first write access).

Table 32-53. Receive Endpoint n Configuration Register (EPn_RX) Field Descriptions (continued)

Bit	Field	Value	Description
13-12	EPn_RX_SIZE	0 8 bytes 1h 16 bytes 2h 32 bytes 3h 64 bytes Non-ISO (bits 13-12): 0 8 bytes 1h 16 bytes 2h 32 bytes 3h 64 bytes 4h 128 bytes 5h 256 bytes 6h 512 bytes 7h 1023 bytes Note: For ISO endpoints, a size of 1023 bytes takes up the whole memory and prevents programming with a 2K-byte USB device controller. Values after USB device controller hardware reset or USB reset are unchanged (which means that values are unknown until first write access).	The receive endpoint n size field contains the endpoint n FIFO size value. Status flags (STAT_FLG.NON_ISO_FIFO_EMPTY, STAT_FLG.NON_ISO_FIFO_FULL, STAT_FLG.ISO_FIFO_EMPTY, and STAT_FLG.ISO_FIFO_FULL) and overrun and underrun conditions are based on this value for all OUT transactions to endpoint n.
11	EPn_RX_ISO	0 Receive endpoint n type is bulk or interrupt. 1 Receive endpoint n type is isochronous. Values after USB device controller hardware reset or USB reset are unchanged.	The receive ISO endpoint n field must be set if the receive endpoint n type is isochronous in the desired device configuration. If not set, the endpoint type is bulk or interrupt (the hardware does not distinguish bulk type from interrupt).
10-0	EPn_RX_PTR	0 Address = BASE (not permitted) 1h Address = BASE + 8 bytes 2h Address = BASE + 16 bytes 3h Address = BASE + 24 bytes : FFh Address = BASE + 2040 bytes Values after USB device controller hardware reset or USB reset are unchanged (which means that values are unknown until first write access). Note: Pointer value must be set to a value < FFh, because the memory size is 2K bytes and a pointer coded value = FFh corresponds to 2040 bytes. Addressing upper bytes results in memory overlap.	The receive endpoint n pointer field contains the address of the receive endpoint n pointer. Value 0x000 is not permitted (reserved for setup FIFO). Note: For ISO endpoints or for non-ISO endpoints that allow double-buffering, 2 × RX buffer size must be reserved for ping-pong.

32.3.1.23 Transmit Endpoint n Configuration Register (EPn_TX)

This read/write register gives the device configuration for non-control transmit endpoint n (1:15). The endpoints size fields must match the values sent by the USB device controller to the USB host in response to the GET_CONFIGURATION_DESCRIPTOR during configuration phase.

NOTE: The USB device controller must fill this field before setting SYSCON1.CFG_LOCK and must not change the values once SYSCON1.CFG_LOCK is set.

Figure 32-50. Transmit Endpoint n Configuration Register (EPn_TX)
[address = FCF78A62h to FCF78A7Eh]

15	14	13	12	11	10	8
EPn_TX_VALID	EPn_TX_SIZE_DB	EPn_TX_SIZE	EPn_TX_ISO	EPn_TX_PTR		
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	
7						0
EPn_TX_PTR						
R/W-0						

LEGEND: R/W = Read/Write; -n = value at reset

Table 32-54. Transmit Endpoint n Configuration Register (EPn_TX) Field Descriptions

Bit	Field	Value	Description
15	EPn_TX_VALID	0 1	The transmit endpoint n valid bit must be set by the USB device controller to allow transmit endpoint n to be used for USB transfers as part of the device configuration. If not set, all transactions to this endpoint are ignored by the core. Transmit endpoint n does not exist for this configuration. Transmit endpoint n is part of the device configuration. Values after USB device controller hardware reset are low, after USB reset is unchanged.
14	EPn_TX_SIZE_DB	0 1	Transmit non-ISO (or ISO) endpoint n double buffer. This bit is only for non-ISO endpoints used in DMA mode. For ISO endpoints, which are always double buffered, this bit is endpoint size MSB. For non-ISO endpoints that are not used in a DMA transfer, double buffering is not provided. This bit must be set by the USB device controller to allow double buffering for transmit non-ISO endpoint n, when used in a DMA transfer. This is used to reduce the number of transactions resulting in NAK handshake. No double buffer for non-ISO transmit endpoint n. Double buffer used for non-ISO transmit endpoint n. Values after system reset or USB reset is unchanged. Or transmit ISO endpoint n size[2]

Table 32-54. Transmit Endpoint n Configuration Register (EPn_TX) Field Descriptions (continued)

Bit	Field	Value	Description
13-12	EPn_TX_SIZE	0 1h 2h 3h 0 1h 2h 3h 4h 5h 6h 7h	The transmit endpoint n size field contains the endpoint n FIFO size value. Status flags (FIFO_EMPTY, FIFO_FULL) and underrun condition are based on this value for all IN transactions to endpoint n. Non-ISO (bits 13-12): 8 bytes 16 bytes 32 bytes 64 bytes ISO (bits 14-12): 8 bytes 16 bytes 32 bytes 64 bytes 128 bytes 256 bytes 512 bytes 1023 bytes Note: For ISO endpoints, a size of 1023 bytes takes up the whole memory and prevents programming with a 2K-byte USB device controller. Values after USB device controller hardware reset or USB reset are unchanged (which means that values are unknown until first write access).
11	EPn_TX_ISO	0 1	The transmit ISO endpoint n field must be set if the transmit endpoint n type is isochronous in the desired device configuration. If not set, the endpoint type is bulk or interrupt (the hardware does not distinguish bulk type from interrupt). Transmit endpoint n type is bulk or interrupt. Transmit endpoint n type is isochronous. Value after USB reset is unchanged.
10-0	EPn_TX_PTR	0 1h 2h 3h : FFh	The transmit endpoint n pointer field contains the address of the transmit endpoint n pointer. For ISO endpoints or for non-ISO endpoints that allow double-buffering, (2 × TX buffer size) must be reserved for ping-pong. Address = BASE Address = BASE + 8 bytes Address = BASE + 16 bytes Address = BASE + 24 bytes : Address = BASE + 2040 bytes Values after USB device controller hardware reset or USB reset are unchanged (which means that values are unknown until first write access). Pointer value must be set to a value < FFh, because the memory size is 2K bytes and a pointer coded value = FFh corresponds to 2040 bytes. Addressing upper bytes results in memory overlap.

32.3.2 USB Device Transactions

There is an interrupt to the CPU at the end of an USB transaction if the CPU has actions to perform. Isochronous transactions are an exception because isochronous interrupt information is available at start of frame interrupts. The CPU ISR code determines which endpoint and direction caused the interrupt and acts appropriately. The following subsections describe in detail the activities surrounding USB transactions that are not part of a DMA transfer. Cases where a transaction occurs before the previous one has been handled by the CPU are not taken into account in this part. The information is organized so that each section deals with one type and direction of transaction, such as:

- Non-isochronous, non-setup OUT transactions
- Non-isochronous IN transactions
- Isochronous OUT transactions
- Isochronous IN transactions

This allows each section to focus only on a specific style of transaction without adding in the confusion of special cases related to other styles.

32.3.3 Non-Isochronous, Non-Setup OUT (USB HOST→CPU) Transactions

Non-isochronous, non-setup OUT transactions refer to USB transactions where data is moved from the USB host to the CPU, the USB handshaking protocols are in effect, and data transmission is ensured. These types of transactions apply to all OUT transactions on bulk and interrupt endpoint types, and to non-setup transactions on control endpoints.

Figure 32-51 shows the various USB protocol conditions that can occur during non-isochronous, non-setup OUT transactions. The diagram shows the three phases that can occur in an OUT transaction, the direction of information flow for each phase, when endpoint interrupts are generated, and the resulting STAT_FLG bits for the endpoint. The top three cases show the normal USB handshaking modes: acknowledge (good data received), NAK (device not ready to receive data), and STALL (device in a condition where the endpoint cannot handle OUT transactions). The last case is an abnormal instance where the token packet or the data packet was received with errors. The RX FIFO only contains valid receive data under the first (acknowledge) case.

Figure 32-51. Non-Isochronous, Non-Control OUT Transaction Phases and Interrupts

Successful data transfer to USB Host. (Endpoint STAT_FLG.FIFO_EN bit was set when token was received.)

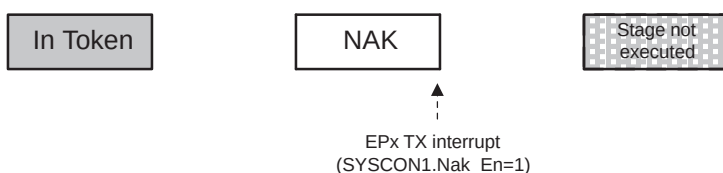


After interrupt, EP's TX FIFO is empty.

STAT_FLG bits after interrupt

EP_Halted	STALL	NAK	ACK
0	0	0	1

No data transmitted by the LH. (Endpoint STAT_FLG.FIFO_EN bit was clear when token was received.)



EP TX FIFO is unchanged by this USB transaction.

STAT_FLG bits after interrupt (SYSCON1.Nak_En=1)

EP_Halted	STALL	NAK	ACK
0	0	1	0

EP stalled. No data transmitted by the LH. (Endpoint STAT_FLG.EP_HALTED bit was set when token was received or an EPO control request error has occurred.)

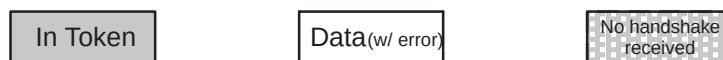


EP TX FIFO is cleared by this USB transaction.

STAT_FLG bits after interrupt

EP_Halted	STALL	NAK	ACK
1	1	0	0
or	0	1	0

EP TX Data Error during transmission.



EP TX FIFO is unchanged by this USB transaction. No interrupt occurs. STAT_FLG is unchanged.

 Indicates a packet received by the device

 Indicates a packet sent by the device

32.3.3.1 Non-Isochronous, Non-Control OUT Endpoint Handshaking Conditions

An endpoint CTRL.SET_FIFO_EN bit provides the main control for the endpoint ability to allow successful OUT transaction data reception for the endpoint. If at the beginning of an OUT transaction to an endpoint, the endpoint STAT_FLG.FIFO_EN bit is 1, the USB module is allowed to accept the OUT transaction data to the RX FIFO, and, when the transaction completes, the USB module can return ACK to the PC to indicate that the data was received correctly (this is the top case shown in [Figure 32-51](#)). If, however, the endpoint STAT_FLG.FIFO_EN bit was 0 at the beginning of an OUT transaction to the endpoint, the USB module returns NAK during the handshake phase to indicate that the endpoint did not accept the data (the second case shown in [Figure 32-51](#)).

The USB host need not send a whole RX FIFO worth of data to the endpoint during an OUT transaction. In this case, the RX FIFO is not full when the endpoint RX interrupt is generated. The CPU code must be careful not to read too much data. CPU code must read RXFSTAT.RXF_COUNT value before reading data from the RX FIFO.

At the end of an USB OUT transaction to an endpoint where the data is accepted (ACKed), the hardware clears the endpoint STAT_FLG.FIFO_EN bit. Once the CPU software has dealt with the OUT transaction data in the endpoint RX FIFO, it must re-enable the endpoint OUT transaction reception by setting the endpoint CTRL.SET_FIFO_EN bit. CPU software can use the endpoint CTRL.SET_FIFO_EN bit as a receive flow control mechanism.

32.3.3.1.1 Acknowledged Transactions (ACK)

At completion of an OUT transaction to an endpoint, the USB module issues an endpoint-specific interrupt to the CPU and STAT_FLG is updated. In response to the endpoint interrupt, the CPU must read EPN_STAT register to identify the endpoint causing the interrupt, then write a 1 to the interrupt bit to clear it. The CPU must then set EP_NUM.EP_NUM to the endpoint number and EP_NUM.EP_SEL to 1, and then read the endpoint status from STAT_FLG. STAT_FLG.ACK is set to indicate that the endpoint received a transaction to which the USB module signaled ACK handshaking.

If STAT_FLG.FIFO_EMPTY is cleared, the transaction sent 1 or more bytes of data (but less than or equal to the physical size of the endpoint RX FIFO), and the data is in the endpoint RX FIFO. The CPU knows the number of bytes to read from RX FIFO by reading RXFSTAT.RXF_COUNT value. The CPU can then read RX data from DATA register. Once the CPU has read the data from the FIFO, it sets the CTRL.SET_FIFO_EN bit to allow the next USB OUT transaction to the endpoint to be placed into the RX FIFO, and then clears the EP_NUM.EP_SEL bit. This clears the STAT_FLG.ACK bit for this endpoint to allow next transaction status to be written into the STAT_FLG register.

32.3.3.1.2 Non-Acknowledged Transactions (NAK)

The device can be configured via the SYSCON1.NAK_EN either to inform the CPU of a NAKed transaction or not. If SYSCON1.NAK_EN is cleared, no interrupt is asserted to the CPU if an OUT transaction completes with a NAK handshake and STAT_FLG.NAK bit is not set. If SYSCON1.NAK_EN is set, the USB module issues an endpoint-specific interrupt to the CPU at completion of an OUT transaction to an endpoint, and STAT_FLG.NAK bit is set. In response to the endpoint interrupt, the CPU must read EPN_STAT register to identify the endpoint causing the interrupt, and then write a 1 to the interrupt bit to clear it. The CPU must then set EP_NUM.EP_NUM to the endpoint number and EP_NUM.EP_SEL to 1, and then read the endpoint status from STAT_FLG. STAT_FLG.NAK is set to indicate that the endpoint received a transaction to which the USB module signaled NAK handshaking.

The CPU must set the CTRL.SET_FIFO_EN bit to allow the next USB OUT transaction to the endpoint to be placed into the RX FIFO, and then clear the EP_NUM.EP_SEL bit. This clears the STAT_FLG.NAK bit for this endpoint to allow the next transaction status to be written into the STAT_FLG register.

32.3.3.2 Non-Isochronous, Non-Control OUT Transaction Error Conditions

32.3.3.2.1 STALLed Transactions

The USB module responds to an endpoint OUT transaction with a STALL handshake to indicate an error condition on the endpoint either if the endpoint STAT_FLG.EP_HALTED bit is set or if a request error occurs (control transactions only). When an endpoint OUT transaction is given a STALL handshake, the endpoint STAT_FLG.STALL bit is set and an endpoint-specific interrupt is generated for the endpoint. STAT_FLG.FIFO_EN is of lower priority than STAT_FLG.EP_HALTED; when the STAT_FLG.EP_HALTED bit is set and transactions to the RX endpoint are stalled, regardless of the STAT_FLG.FIFO_EN value. If STAT_FLG.FIFO_EN is set, the STAT_FLG.FIFO_EN bit is automatically cleared at the end of the STALLed transaction and RX FIFO is cleared.

In response to the endpoint interrupt, the CPU must read the EPN_STAT register to identify the endpoint causing the interrupt, and then write a 1 to the interrupt bit to clear it. The CPU must then set EP_NUM.EP_NUM to the endpoint number and EP_NUM.EP_SEL to 1, and then read the endpoint status from STAT_FLG. STAT_FLG.STALL is set to indicate that the endpoint received a transaction to which the USB module signaled STALL handshaking.

If STAT_FLG.EP_HALTED has been set by the CPU and can be removed, the CPU must set CTRL.CLR_HALT to clear the condition and set CTRL.SET_FIFO_EN to allow the next USB OUT transaction to the endpoint to be placed into the RX FIFO. If STAT_FLG.EP_HALTED has been set in response to a SET_FEATURE request sent by the USB host, or if the bit is cleared (control transaction only), the CPU has no action to perform and must clear the EP_NUM.EP_SEL bit. This clears the STAT_FLG.STALL bit for this endpoint and allows the next transaction status to be written into the STAT_FLG register.

32.3.3.2.2 Packet Errors

In case of a receive data error during an endpoint OUT transaction (token or data packet), the USB module does not provide a handshake during the handshake phase of the transaction and no interrupt is asserted to the CPU (the fourth case shown in [Figure 32-51](#)). Additionally, the endpoint RX FIFO is not filled, and STAT_FLG.FIFO_EN bit is not cleared. If the CPU clears the RX FIFO during the data packet of an OUT transaction, no handshake is returned to the USB host to signal an error.

32.3.3.2.3 Sequence Bit Errors

If the core does not receive expected DATA PID during an OUT transaction, the module automatically returns ACK handshake to the USB host, regardless of the STAT_FLG.FIFO_EN bit (per the USB specification). Data is ignored, and no interrupt is asserted to the CPU.

This error occurs if ACK handshake from previous OUT transaction is received corrupted by the USB host.

32.3.3.3 Non-Isochronous, Non-Control OUT Endpoint FIFO Error Conditions

If the USB host attempts to fill more data into an endpoint RX FIFO than the FIFO can hold, a FIFO overrun occurs. The USB module does not provide a handshake during the handshake phase of the transaction and no interrupt is asserted to the CPU. Additionally, the endpoint RX FIFO is not filled, and STAT_FLG.FIFO_EN bit is not cleared.

The CPU must not read more data from RX FIFO than the value indicated by RXFSTAT.RXF_COUNT.

32.3.4 Non-Isochronous IN (CPU→USB HOST) Transactions

Non-isochronous IN transactions refer to USB transactions in which data is moved from the CPU to the USB host, where the USB handshaking protocols are in effect, and data transmission is ensured. These transactions are the IN transactions that occur on control, bulk, and interrupt endpoints. These transactions do not ensure USB bandwidth.

To provide data for an endpoint IN transaction, the CPU code writes the transmit data into the endpoint transmit FIFO. CPU code must first wait until the USB is done with any previous TX data for the endpoint (if data had previously been written to the TX FIFO). This must be done by proper response to endpoint-specific transmit interrupts. When an IN transaction to the endpoint occurs, if the endpoint `STAT_FLG.FIFO_EN` bit is set, the USB module sends any data that is in the endpoint TX FIFO during the data phase. If the TX FIFO is empty and `STAT_FLG.FIFO_EN` is set when an IN transaction to the endpoint occurs, a 0-byte data packet is sent.

Once the endpoint previous transmit activity is taken care of, the CPU code gains access to endpoint FIFO and status by setting the `EP_NUM.EP_SEL` bit. Then the CPU can write the new transmit data to the endpoint TX FIFO via the `DATA` register (being careful not to overflow the FIFO). Once all of the transmit data has been written to the endpoint FIFO, CPU code sets the `CTRL.SET_FIFO_EN` bit to allow the USB to use the endpoint TX FIFO, and then clears the `EP_NUM.EP_SEL` bit. The data in the endpoint TX FIFO is sent to the USB host the next time an IN transaction to the endpoint occurs.

[Figure 32-52](#) shows the various USB protocol conditions that can occur during non-isochronous IN transactions. It diagrams the three phases of the IN transaction, the direction of information flow for each phase, when endpoint-specific interrupts are generated, and the resulting `STAT_FLG` bits for the endpoint. The top three cases show the normal USB handshaking: acknowledge (data sent by USB module and received properly by the USB host), NAK (device not ready to send data to USB host), and STALL (device in a condition where the endpoint cannot handle IN transactions). The last case shows an abnormal case in which there is an error either in the token packet received by the core or in the data packet received by the USB host.

32.3.4.1 Non-Isochronous IN Endpoint Handshaking

Per USB specifications for IN transactions, the USB host can provide only one of two handshakes to the USB function during the handshake phase: ACK or no handshake at all. The first indicates successful transfer (first case shown in [Figure 32-52](#)), and the second indicates that the host received a garbled data packet (last case shown in [Figure 32-52](#)).

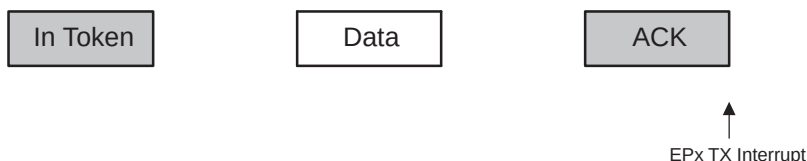
32.3.4.1.1 Acknowledged Transactions (ACK)

When the endpoint IN transaction completes on the USB bus with an ACK handshake, the endpoint generates an endpoint-specific interrupt to the CPU (see first case in [Figure 32-52](#)). In response to the endpoint interrupt, the CPU must read the `EPN_STAT` register to identify the endpoint causing the interrupt, and then write a 1 to the interrupt bit to clear it. The CPU must then set `EP_NUM.EP_NUM` to the endpoint number, `EP_NUM.EP_DIR` to 1 (to signal an IN endpoint), and `EP_NUM.EP_SEL` to 1, and then read the endpoint status from `STAT_FLG`. `STAT_FLG.ACK` is set to indicate that the endpoint received an ACK handshake from the USB host, and the TX FIFO is empty (because any data that was in the TX FIFO was transmitted during the IN transaction).

If the CPU has more data to transmit to the USB host, it must fill the TX FIFO following the process indicated above. It must then clear `EP_NUM.EP_SEL` bit. This clears the `STAT_FLG.ACK` bit for this endpoint to allow next transaction status to be written into the `STAT_FLG` register.

Figure 32-52. Non-Isochronous IN Transaction Phases and Interrupts

Successful data transfer to USB Host. (Endpoint STAT_FLG.FIFO_EN bit was set when token was received.)

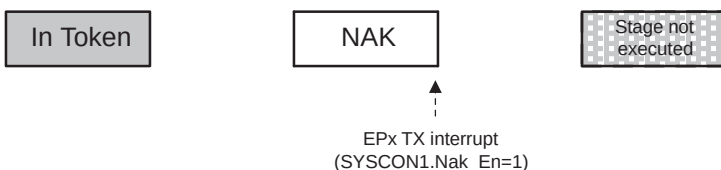


After interrupt, EP's TX FIFO is empty.

STAT_FLG bits after interrupt

EP_Halted	STALL	NAK	ACK
0	0	0	1

No data transmitted by the LH. (Endpoint STAT_FLG.FIFO_EN bit was clear when token was received.)



EP TX FIFO is unchanged by this USB transaction.

STAT_FLG bits after interrupt (SYSCON1.Nak_En=1)

EP_Halted	STALL	NAK	ACK
0	0	1	0

EP stalled. No data transmitted by the LH. (Endpoint STAT_FLG.EP_HALTED bit was set when token was received or an EPO control request error has occurred.)



EP TX FIFO is cleared by this USB transaction.

STAT_FLG bits after interrupt

EP_Halted	STALL	NAK	ACK
1	1	0	0
or	0	1	0

EP TX Data Error during transmission.



EP TX FIFO is unchanged by this USB transaction. No interrupt occurs. STAT_FLG is unchanged.

 Indicates a packet received by the device

 Indicates a packet sent by the device

32.3.4.1.2 Non-Acknowledged Transactions (NAK)

For the case in which the CPU is not ready to provide transmit data for transactions to an IN endpoint, the core provides a NAK handshake to the host for any USB IN transaction to that endpoint. Readiness to transmit data is signaled via the endpoint `STAT_FLG.FIFO_EN` bit; when 1 it indicates that data in the TX FIFO can be sent to the USB host. When the endpoint `STAT_FLG.FIFO_EN` bit is 0 and an IN transaction to the endpoint occurs, NAK handshake is sent, indicating that the CPU is not ready to handle the request.

If `SYSCON1.NAK_EN` bit is cleared, when the NAK handshake is sent in the data packet portion of the transaction to the IN endpoint, `STAT_FLG` is not updated and no endpoint-specific interrupt to the CPU is generated. If the `SYSCON1.NAK_EN` bit is set, when the NAK handshake is sent in the data packet portion of the transaction to the IN endpoint, the `STAT_FLG.NAK` bit is set and an endpoint-specific interrupt to the CPU is generated.

In response to the endpoint interrupt, the CPU must read the `EPN_STAT` register to identify the endpoint causing the interrupt, and then write a 1 to the interrupt bit to clear it. The CPU must then set `EP_NUM.EP_NUM` to the endpoint number, `EP_NUM.EP_DIR` to 1 (to signal an IN endpoint), and `EP_NUM.EP_SEL` to 1, and then read the endpoint status from `STAT_FLG`. `STAT_FLG.NAK` is set to indicate that the endpoint sent a NAK handshake to the USB host. If the CPU has data to transmit to the USB host, it must fill the TX FIFO following the process indicated above. The CPU must then clear the `EP_NUM.EP_SEL` bit. That clears the `STAT_FLG.NAK` bit for this endpoint to allow the next transaction status to be written into the `STAT_FLG` register. Signaling NAK does not cause the endpoint TX FIFO to be cleared (because the CPU still retains control of the FIFO).

Signaling NAK handshake for several endpoint transactions in a row can cause the PC host to discard the transaction, so NAK is not necessarily a good mechanism in cases where the CPU is not able to service a request for long periods of time.

32.3.4.2 Non-Isochronous IN Transaction Error Conditions

32.3.4.2.1 STALLed Transactions

The USB module sends a STALL handshake to the USB host during the data phase of the transaction to the IN endpoint either if the endpoint `STAT_FLG.EP_HALTED` flag is set, or if a request error occurs (control transaction only). A USB STALL handshake indicates that the device endpoint is in a condition in which it is not able to transfer data and instructs the USB host not to retry the transaction. The device typically requires intervention via some other mechanism to clear the condition, usually a control transfer via endpoint 0. The CPU can set the endpoint `EP_HALTED` bit by selecting the endpoint by writing the appropriate value in `EP_NUM` register, and then setting the endpoint `CTRL.Set_HALT` bit, and clear it by selecting the endpoint, and then setting the endpoint `CTRL.Clr_HALT` bit. When the endpoint `EP_HALTED` bit is set, the endpoint signals STALL for its IN transactions until the HALT condition is cleared. When the STALL handshake is sent in response to a transaction to the endpoint, the `STAT_FLG.STALL` bit is set, and an endpoint-specific interrupt to the CPU is generated.

In response to the endpoint interrupt, the CPU must read the `EPN_STAT` register to identify the endpoint causing the interrupt, and then write a 1 to the interrupt bit to clear it. The CPU must then set `EP_NUM.EP_NUM` to the endpoint number, `EP_NUM.EP_DIR` to 1 (to signal an IN endpoint), and `EP_NUM.EP_SEL` to 1, and then read the endpoint status from `STAT_FLG`. `STAT_FLG.STALL` is set to indicate that the endpoint sent a STALL handshake to the USB host. The CPU must then clear `EP_NUM.EP_SEL` bit. This clears the `STAT_FLG.STALL` bit for this endpoint and allows the next transaction status to be written into the `STAT_FLG` register.

Except for control endpoint 0, separate endpoint halt bits are defined for each direction; so for a given endpoint number, the TX can be halted when the RX is not.

32.3.4.2.2 Packet Errors

If an error (CRC, bit stuffing, or PID check) occurs during the token packet of a USB IN transaction to a non-isochronous endpoint, the USB block ignores the transaction. No endpoint-specific interrupt to the CPU occurs for transactions with corrupted packets. If the CPU clears the TX FIFO during the data packet of an IN transaction, a bit stuffing error is forced.

If the USB host returns no handshake after an IN transaction (in case of an error during transmission), the USB device controller module detects after a time-out that an error has occurred. The data to transmit is still in the TX FIFO to be re-sent during next IN transaction, STAT_FLG.FIFO_EN is not cleared, and no interrupt is asserted to the CPU.

32.3.4.3 Non-Isochronous IN Endpoint FIFO Error Conditions

The CPU cannot write more data into the TX FIFO than the configured FIFO size.

32.3.5 Isochronous OUT (USB HOST→CPU) Transactions

Isochronous OUT transactions are USB transactions in which a given amount of data is transferred from the USB host to the USB device controller module device every 1-ms USB frame. No USB handshaking is provided, and no endpoint-specific interrupt to the CPU is generated at completion of an isochronous OUT transaction. The CPU is responsible for handling isochronous OUT data at each start of frame (SOF) interrupt.

At every SOF interrupt, for each isochronous OUT endpoint, CPU code must select the endpoint by writing the appropriate value in the EP_NUM register and check STAT_FLG.ISO_FIFO_EMPTY. If the RX FIFO contains data, code must read the RXFSTAT.RXF_COUNT value (if the number of bytes to read from RX FIFO is not known), read all the bytes from RX FIFO via the data register, and then clear the EP_NUM.EP_SEL bit.

Because the USB transaction for the isochronous endpoint can occur at any time during the USB 1-ms frame, the USB interface implements a double-buffering of the endpoint receive data FIFO. The endpoint includes two FIFOs, each of which is the length of the configured isochronous endpoint. At all times, one of the two FIFOs is foreground and the other is background. The USB interface side of the USB module is allowed to write to the background RX FIFO, and the CPU is allowed to read to the foreground RX FIFO. The designations foreground and background are swapped at each start of frame (SOF). Isochronous endpoint FIFOs in the background are always enabled to the USB, whereas the foreground FIFOs are enabled to the CPU.

[Figure 32-53](#) shows the two phases (ISO OUT token and data) of an isochronous OUT data transfer in the top portion of the figure. No endpoint-specific interrupt to the CPU is generated for the isochronous OUT transaction. The data for isochronous endpoints are instead handled by the CPU at each start of frame (SOF) interrupt, which is shown as the second case in [Figure 32-53](#).

Figure 32-53. Isochronous OUT Transaction Phases and Interrupts

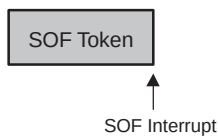
Successful data transfer from USB Host



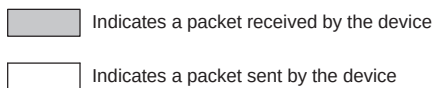
No handshake occurs. EP RX FIFO contains received data after data packet completes. No interrupt occurs.

Reception of SOF causes SOF interrupt.

Note: An SOF interrupt is generated even if the SOF packet is corrupted.



LH code for SOF interrupt service routine must fill all isochronous in EP TX FIFOs with new transmit data and pull new receive data from all Isochronous Out EP RX FIFOs.



32.3.5.1 Isochronous OUT Endpoint Handshaking

Because isochronous endpoint transactions have no handshake packets, the STAT_FLG.STALL, STAT_FLG.NAK, and STAT_FLG.ACK bits for isochronous endpoints always return 0. Because there is no handshake, the endpoint-specific interrupt for isochronous endpoints is not used.

32.3.5.2 Isochronous OUT Transaction Error Conditions

If the CPU fails to read all of the data in the ISO OUT endpoint foreground FIFO by the time the foreground and background FIFOs are switched (at the next SOF), the endpoint FIFO that is being switched to the background is flushed, and the STAT_FLG.DATA_FLUSH bit is asserted for the duration of the next frame.

There is no special indication for the case in which the USB host does not provide a transaction to an ISO OUT endpoint during a frame, but once the FIFO that was background in that frame is foreground, the FIFO is empty (a 0-length data ISO OUT transaction also results in an empty FIFO and cannot be distinguished from a missed ISO OUT transaction).

If an ISO OUT transaction occurs with data error (CRC, PID check, or bit stuffing), the RX FIFO is empty at the next SOF interrupt, and the STAT_FLG.ISO_ERR bit is asserted for the duration of the next frame.

32.3.5.3 Isochronous OUT Endpoint FIFO Error Conditions

The CPU must never read more data than the value given by RXFSTAT.RXF_COUNT.

If the USB host sends more data than the FIFO can contain, the FIFO is cleared and the STAT_FLG.ISO_ERR is set at the next SOF interrupt. A properly configured USB system does not do this.

NOTE: Both foreground and background isochronous FIFOs are cleared when the CTRL.CLR_EP bit is set.

32.3.6 Isochronous IN (CPU→USB HOST) Transactions

Isochronous IN transactions are USB transactions in which a given amount of data is transferred from the USB device controller module device to the USB host every 1-ms USB frame. No handshaking is provided.

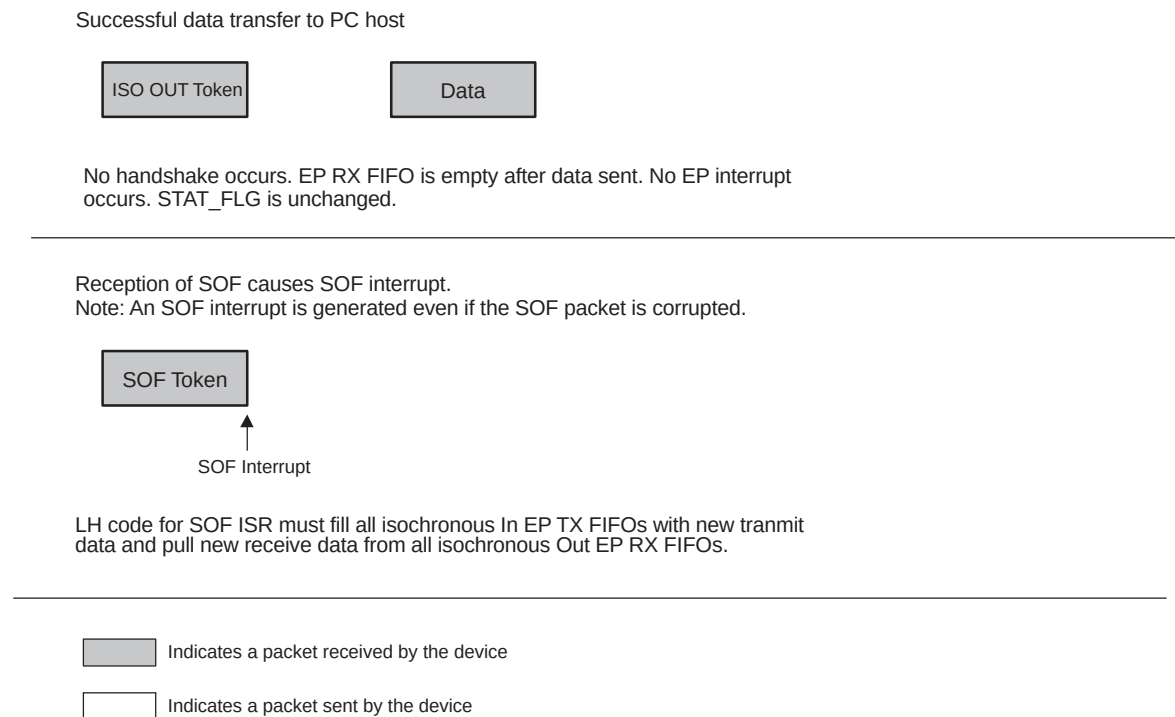
The USB module provides double-buffering of data for ISO IN endpoints; the background FIFO is used as the source of data for IN transactions to the ISO endpoint, and the foreground FIFO can be written to by the CPU. When an IN transaction to an ISO endpoint occurs, the USB module sends all data found in the endpoint background TX FIFO. The CPU is responsible for providing new data to the isochronous IN endpoint foreground TX FIFO at each start of frame interrupt.

In response to the SOF interrupt, for each isochronous IN endpoint, CPU code selects the endpoint (via the EP_NUM register), and then fills the endpoint TX FIFO (via the DATA register). Once all the transmit data have been written to the FIFO, the CPU code must clear the EP_NUM.EP_SEL bit.

Because the USB transaction for the isochronous endpoint can occur at any time during the USB 1-ms frame, the USB interface implements a double-buffering of the endpoint transmit data FIFO. The endpoint includes two FIFOs, each of which is the length of the configured isochronous endpoint. At all times, one of the two FIFOs is foreground and the other is background. The USB interface side of the USB module is allowed to read from the background TX FIFO, and the CPU is allowed to write to the foreground TX FIFO. The designations foreground and background are swapped, and the new background TX FIFO is cleared at each start of frame (SOF). Because ISO endpoints implement double-buffering, ISO endpoints do not control access to the FIFOs via a CTRL.SET_FIFO_EN bit; the CTRL.SET_FIFO_EN and the STAT_FLG.FIFO_EN bits are not implemented for ISO IN endpoints.

Figure 32-54 shows the transaction phases associated with isochronous IN transactions and the SOF transaction. No endpoint-specific interrupt to the CPU is generated as a result of an isochronous IN transaction, and there is no handshake phase. The SOF transaction causes an SOF interrupt to the CPU; it is assumed that the CPU refills the isochronous IN endpoint transmit FIFO at each SOF interrupt.

Figure 32-54. Isochronous IN Transaction Phases and Interrupts



32.3.6.1 Isochronous IN Endpoint Handshaking

Because isochronous endpoint transactions have no handshake packets, the `STAT_FLG.STALL`, `STAT_FLG.NAK`, and `STAT_FLG.ACK` bits for isochronous endpoints always return 0. Because there is no handshake, there is no endpoint-specific interrupt to the CPU to report handshake results for isochronous endpoints.

32.3.6.2 Isochronous IN Transaction Error Conditions

If the USB host did not successfully complete an ISO IN transaction in the previous frame, and if data were present in TX FIFO to be sent at the IN transaction, the `STAT_FLG.MISS_IN` bit is asserted for the duration of the following frame. If the ISO IN endpoint is cleared in the middle of a USB transaction to the background FIFO, the macro forces a bit stuffing error for the ISO transaction.

32.3.6.3 Isochronous IN Endpoint FIFO Error Conditions

If the CPU attempts to overfill the configured endpoint FIFO, data written to DATA register after the TX FIFO is full is lost, but any data that was successfully put into the FIFO is transmitted when that FIFO is the background FIFO and an IN transaction for that endpoint occurs. Because an ISO TX FIFO is cleared automatically on the toggle from background to foreground, there is no reason to clear the FIFO. However, if the CPU does not wish to send the data it wrote, clearing the endpoint is the only mechanism to do this.

32.3.7 Control Transfers on Endpoint 0

Control transfers on endpoint 0 include control write and control read transfers. Control write and control read transfers are each composed of two or more transactions to endpoint 0. Additionally, the USB device controller module is capable of autodecoding some control write and control read transfers. These operations are summarized in [Figure 32-55](#) and [Figure 32-56](#). An IN or an OUT transaction is received out of a control request. This transaction is automatically stalled by the core.

Non-autodecoded control read and control write transfers are sets of transactions that occur on endpoint 0 and have specific USB protocol meaning but are not handled automatically by the core. The USB device controller block automatically provides an ACK handshake for the setup stage transaction, but the data and status stage transaction handshaking is accomplished using the usual RX and TX control bits that affect transaction handshaking. A general USB interrupt to the CPU occurs at the end of each transaction of each stage of a control transfer. The CPU must perform the following actions to act on non-autodecoded control transfers:

- Process the setup phase setup USB interrupt. The CPU reads the control transfer command from the setup FIFO and decodes the command. For control reads, the CPU fetches the requested read data and places it (or as much of the read data as fits) into the endpoint 0 FIFO, and then enables the endpoint 0 FIFO. For control writes, the CPU code only enables the endpoint 0 FIFO. CPU code also sets any flags needed for processing endpoint 0 USB interrupts during the control transfer.
- Process the data phase endpoint 0 general USB interrupt(s). For control reads, the data phase general USB endpoint 0 TX interrupt indicates that the previously provided transmit data has been sent. Any additional data must be written to the endpoint 0 FIFO. For control writes, the write data must be pulled from the endpoint 0 FIFO, and when all control write data is available, interpret the write data and act on the write request. After handling the last data phase interrupt, the CPU must set the endpoint 0 control bits to signal the desired status to the host.
- Process the status stage endpoint 0 general USB interrupt. The CPU provides its completion status back to the USB host during this stage, either via status in the data phase of the transaction (for control write transfers) or via the handshake phase of the transaction (for control read transfers).

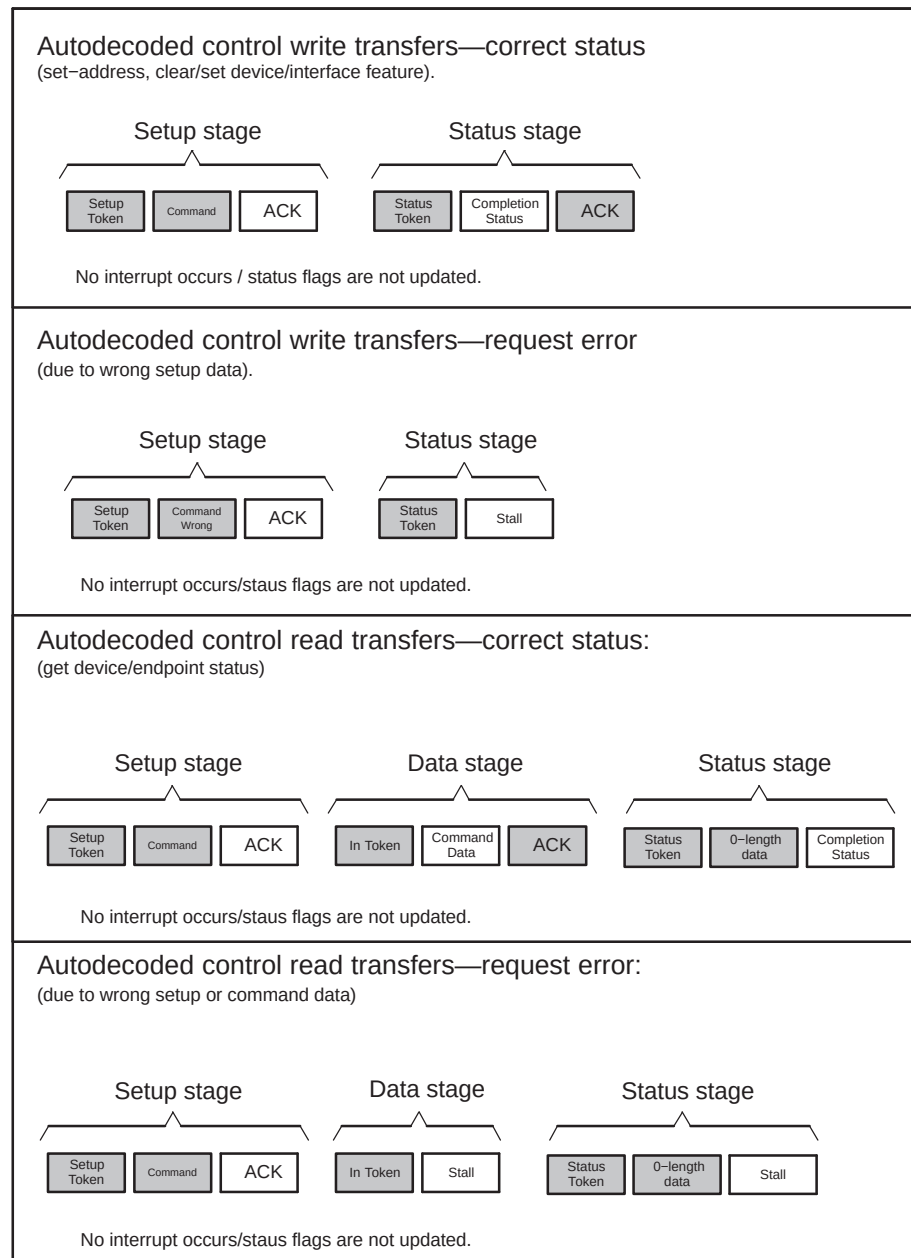
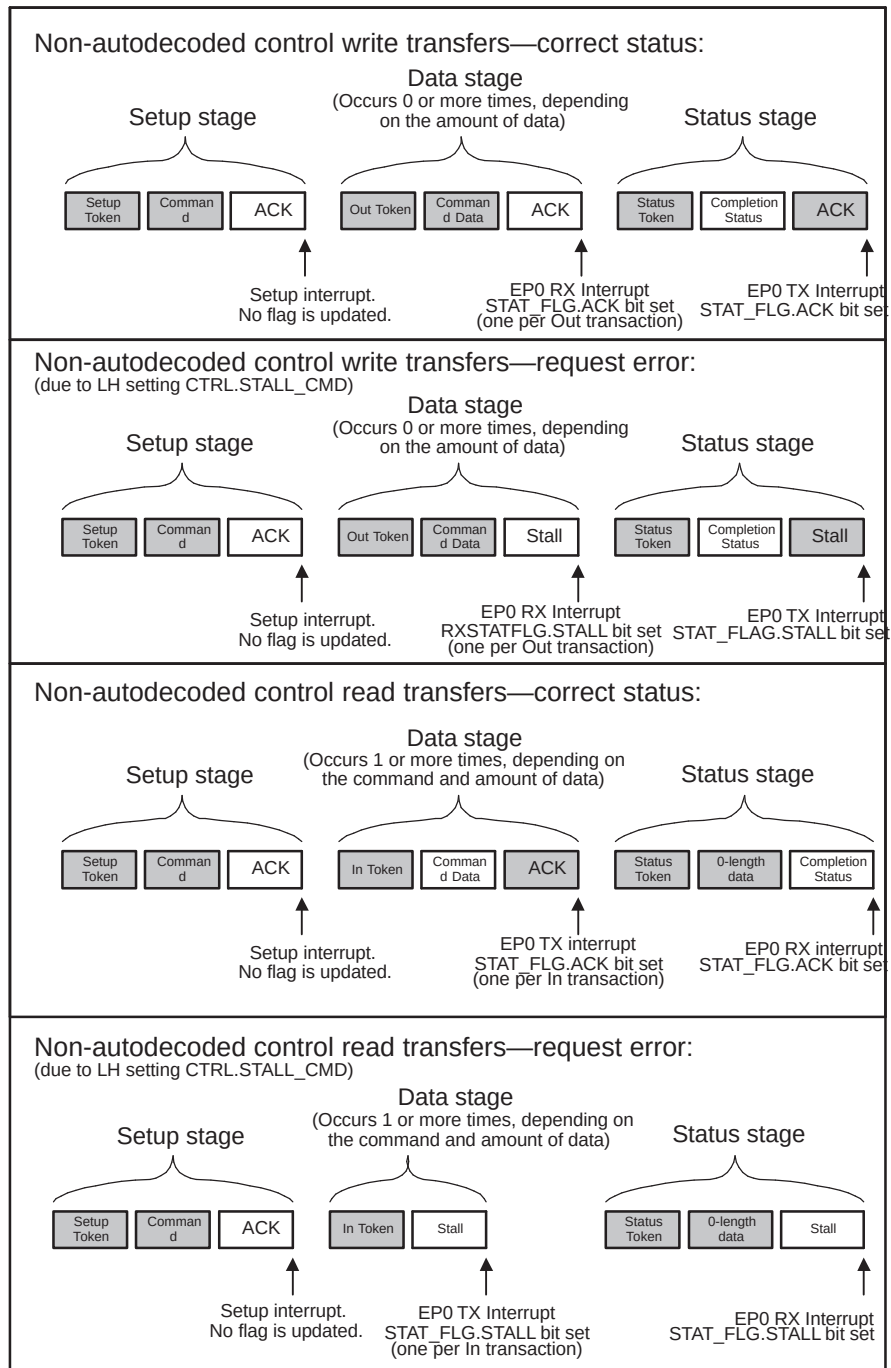
Figure 32-55. Stages and Transaction Phases of Autodecoded Control Transfers


Figure 32-56. Stages and Transaction Phases of Non-Autodecoded Control Transfers



Autodecoded control read and control write transfers are sets of transactions that occur on endpoint 0 that have specific USB protocol meaning and are handled automatically by the USB device controller block without any intervention by the CPU. The USB device controller block handles all handshaking automatically and without regard to the endpoint 0 control bits that affect normal (non-control transfer) transaction handshaking. No interrupt is asserted to the CPU during autodecoded control transfers.

If no USB1.1 specification defined request is associated with the data of the setup phase, request is stalled by the core and the CPU is not informed of its occurrence (autodecoded).

When a setup token is identified, the USB decode module must monitor the setup stage data packet, decode it, and determine whether it is an autodecoded or a non-autodecoded transfer and a control read or a control write. If it is a valid non-autodecoded request, the setup FIFO is immediately cleared and control of the FIFO is immediately taken away from the CPU (if the CPU had control of the FIFO). New setup data are placed into the setup FIFO, and the setup interrupt flag is set (IRQ_SRC.setup).

In response to the setup interrupt, the CPU must select the setup FIFO by setting the EP_NUM.SETUP_SEL bit. This clears the IRQ_SRC.SETUP flag. The CPU must then read 8 bytes from the setup FIFO, clear the EP_NUM.SETUP_SEL bit, and confirm that IRQ_SRC.SETUP bit has not been reset by a new setup transaction. If the IRQ_SRC.SETUP flag is asserted, the CPU must discard the previously read data and handle the new setup packet as explained above. Thus the CPU never misses a new occurring setup transaction (per USB 1.1 specification).

32.3.7.1 Autodecoded Control Write Transfers

For set address control write transfers, the USB address provided in the setup token is captured to the USB module device address register. If new address is different from 0, the device moves into addressed state (DEVSTAT.ADD set) if it was not already addressed.

For set and clear feature control writes, the appropriate feature information bit is set or cleared. When a set or clear feature transfer occurs to set or clear the device remote wake-up feature, the DEVSTAT.R_WK_OK bit is set or cleared, as appropriate. If a set or clear interface feature occurs, the request is automatically stalled by the core because no feature is defined for interface (see the USB 1.1 specification).

Per the USB 1.1 specifications, a SET_ADDRESS request is effective after the status stage of the request, even if the status stage does not end with an ACK handshake. SET/CLEAR_FEATURE requests are effective after setup stage, even if no status stage occurs.

32.3.7.1.1 Autodecoded Control Write Transfer Handshaking

The USB device controller module automatically provides ACK handshaking for all transactions of all stages of autodecoded control write transfers, except if a corrupted packet is received, which the USB module ignores. The CTRL.SET_FIFO_EN and SYSCON2.STALL_CMD bits have no effect on handshaking.

32.3.7.1.2 Autodecoded Control Write Transfer Error Conditions

If the token packet or the data packet of a setup stage transaction has an error (bad CRC, PID check, or bit stuffing error), the USB block ignores the transaction. The USB block does not provide ACK handshaking in this case.

32.3.7.2 Autodecoded Control Read Transfers

Autodecoded control reads include the standard device requests GET_ENDPOINT and DEVICE_STATUS. These control read transfers access information that is kept in registers inside the USB module, so CPU code is not involved in filling the read data into the TX FIFO.

The USB module returns the currently selected appropriate status information (depending on the wIndex value in the setup stage data packet) during the data phase of the single IN transaction of the data stage, and provides ACK as the handshake for the status stage handshake phase. The CPU receives no interrupt.

32.3.7.2.1 Autodecoded Control Read Transfer Handshaking

The USB device controller module automatically provides ACK handshaking for all transactions of all stages of autodecoded control read transfers, except if a corrupted token packet is received, which the USB module ignores. The CTRL.FIFO_EN and SYSCON2.STALL_CMD bits have no effect on the handshaking. If the status packet has a DATA0 PID instead of a DATA1 PID, status is STALLED and no interrupt is asserted to the CPU. If the setup packet has a DATA1 PID instead of a DATA0 PID, setup transaction is ignored (error).

32.3.7.2.2 Autodecoded Control Read Transfer Error Conditions

If the token phase or the data phase of a setup stage transaction has an error (bad CRC, PID check, or bit stuffing error), the USB block ignores the transaction. The USB block does not provide ACK handshaking in this case.

Data errors during the data stage of autodecoded control write transfers are handled in the standard way; any data stage transaction from the host where a data error occurs is ignored.

It is possible that the USB host sends a GET_ENDPOINT/DEVICE_STATUS request with a bad parameter. If the autodecode mechanism senses a bad parameter in the setup stage data phase, the autodecode mechanism causes a STALL handshake to be signaled during the data phase of the data stage and during the status stage.

32.3.7.3 Non-Autodecoded Control Write Transfers

Non-autodecoded control write transfers include the SET_/CLEAR_ENDPOINT feature, SET_CONFIGURATION, SET_INTERFACE, SET_DESCRIPTOR and class- or vendor-specific control write transfers. Non-autodecoded control write transfers consist of two or three stages [setup, data (optional), and status].

The setup stage of a valid non-autodecoded control write transfer consists of one SETUP transaction from USB host to USB device. At the end of the setup stage handshake, the USB module generates an CPU general USB interrupt with the IRQ_SRC.SETUP flag set. The CPU must respond to this general USB interrupt by setting EP_NUM.SETUP_SEL bit, which clears the setup interrupt flag. The CPU must then read 8 bytes from the setup FIFO via the DATA register, clear EP_NUM.EP_SEL bit, and check the IRQ_SRC.SETUP flag. If the IRQ_SRC.SETUP flag is set, the CPU must discard the setup data it has just read and handle the new setup data packet following the same scheme. If the IRQ_SRC.SETUP flag is cleared, the CPU code interprets this request information and performs any application-specific activity needed because of the setup stage request. If there is one or more data stage for the transfer, the CPU must set the CTRL.SET_FIFO_EN bit for endpoint 0 to allow the core to accept RX data from the coming OUT transaction.

The data stage for non-autodecoded control writes consists of zero or more OUT transactions. Transaction handshaking and interrupt generation as for non-isochronous, non-control OUT endpoints applies. The CPU can cause NAK, STALL, or ACK signaling for the data stage transactions. If ACK was signaled on a given general USB interrupt, the CPU must respond by reading the data from the endpoint 0 RX FIFO and saving it for processing.

After completion of the data stage, a status stage IN transaction occurs. The USB module provides handshaking to the USB host based on the endpoint 0 handshaking control bit `STAT_FLG.FIFO_EN`. The CPU can delay signaling completion of the control write transfer by forcing NAK handshaking to the host during the status stage (by holding `STAT_FLG.FIFO_EN` 0), or causing ACK handshaking by setting the `CTRL.SET_FIFO_EN` bit with an empty endpoint 0 FIFO. An endpoint 0 TX general USB interrupt is sent to the CPU at completion of the status stage.

After a `SET_CONFIGURATION` request, the device moves in addressed or configured state as soon as the CPU sets the `SYSCON2.DEV_CFG` or `SYSCON2.CLR_CFG` bits.

32.3.7.3.1 Specific CPU Required Actions

If the device receives a valid set endpoint halt feature request, it must set the appropriate `CTRL.SET_HALT` control bit.

If the device receives a valid `CLEAR_ENDPOINT` halt feature request, it must set the appropriate `CTRL.RESET_EP` bit to clear the halt condition, FIFO flags, and reset data PID to `DATA0` for the endpoint. If the specified endpoint number is 0, the CPU has only to set `CTRL.CLR_HALT` bit to clear the halt condition.

If the device receives a valid `SET_CONFIGURATION` request, it must reset all endpoints by setting the `CTRL.RESET_EP` control bits, set the `SYSCON1.SELF_PWR` bit to the appropriate value, and then set halt conditions for endpoints not used by the default interface set for the configuration. If the device was addressed when the `SET_CONFIGURATION` was received, the CPU must write 1 to the `SYSCON2.DEV_CFG` bit to allow the device to move into the configured state (`DEVSTAT.CFG` bit set). If the device was configured when the `SET_CONFIGURATION` was received, and the new configuration value is 0, the CPU must write 1 to the `SYSCON2.CLR_CFG` bit to allow the device to move back into the addressed state (`DEVSTAT.CFG` bit cleared).

If the device receives a valid set interface request, it must reset all endpoints used by the interface set, by setting `CTRL.RESET_EP` control bits, and then set halt conditions for endpoints not used by this interface.

Other CPU required actions are specific to the request and not detailed in this document.

32.3.7.3.2 Non-Autodecoded Control Write Transfer Handshaking

Setup stage transactions that are valid are signaled ACK. Transactions with invalid setup stage token or data packets are ignored and receive no handshake packet from the USB module, and there is no interrupt generated.

Data stage handshaking for non-autodecoded control write transfers is dependent on the endpoint 0 `STAT_FLG.FIFO_EN`, `STAT_FLG.EP_HALTED`, and `SYSCON2.STALL_CMD` bits. The CPU can delay completion of any transaction of the data stage by signaling NAK (via `CTRL.SET_FIFO_EN` bit not set). The USB specification requires that once `STALL` is signaled in a control transfer, it must be signaled on that endpoint until the next setup token is received. Either the `SYSCON2.STALL_CMD` or the `CTRL.SET_HALT` (reflected in the `STAT_FLG.EP_HALTED` register bit) register bits provide this functionality. `STAT_FLG.EP_HALTED` does not reflect the forced `STALL` caused by `SYSCON2.STALL_CMD`; it retains its previous value.

Status stage handshaking is controlled by the endpoint 0 `STAT_FLG.FIFO_EN` and `SYSCON2.STALL_CMD` bits. Successful completion of a non-autodecoded control write transfer is indicated by the USB device controller module returning a zero length data payload for the data phase of the status stage and an ACK handshake from the host for the handshake phase of the status stage. Although NAK handshaking can be used to indicate delays in completion of the requested control write, the USB host can choose to abort the control write after some number of NAKs.

32.3.7.3.3 Non-Autodecoded Control Write Transfer Error Conditions

If an error occurs while dealing with the control write, which the CPU cannot deal with itself, it must signal `STALL` to the USB host for all subsequent transactions until a new setup token to endpoint 0 occurs. This is true for both data stage and status stage transactions. This is most conveniently done by setting endpoint 0 `SYSCON2.STALL_CMD` bit, which causes stalling of all the remaining transactions of all remaining stages of a non-autodecoded control transfer, up to the reception of the next valid `SETUP` command.

Error conditions are handled as for BULK/INTERRUPT transactions. If a packet is received corrupted, the core ignores the transaction and no interrupt is asserted.

32.3.7.4 Non-Autodecoded Control Read Transfers

Non-autodecoded control read transfers include the GET_INTERFACE_STATUS, GET_CONFIGURATION, GET_INTERFACE, GET_DESCRIPTOR, SYNCH_FRAME and class- or vendor-specific control read transfers. Non-autodecoded control read transfers consist of three stages (setup, data, and status).

The setup stage of a valid non-autodecoded control read transfer consists of one SETUP transaction from USB host to USB device. At the end of the setup stage handshake, the USB module generates a CPU general USB interrupt with the IRQ_SRC.SETUP flag set. The CPU must respond to this general USB interrupt by setting the EP_NUM.SETUP_SEL bit, which clears the setup interrupt flag. The CPU must then read 8 bytes from the setup FIFO via the DATA register, clear the EP_NUM.EP_SEL bit, and check the IRQ_SRC.SETUP flag. If the IRQ_SRC.SETUP flag is set, the CPU must discard the setup data it has just read and handle the new setup data packet following the same scheme. If the IRQ_SRC.SETUP flag is cleared, the CPU code interprets this request information and then prepares data for the IN transaction that follow. This includes placing the data being requested (or the first few bytes, if more than one FIFO worth of data is being returned) into the endpoint 0 FIFO, and setting the CTRL.SET_FIFO_EN bit.

The data stage of a control read transfer consists of one or more IN transactions. Transaction handshaking and interrupt generation as for non-isochronous, non-control IN endpoints applies; the CPU can cause NAK, STALL, or ACK signaling for the data stage transactions. At endpoint 0 TX general USB interrupts, CPU code must move more data to the endpoint 0 FIFO, until the last bytes of the requested data have been provided. Although SETUP packets have a defined payload length, the USB host can cancel the transaction at any time, without the status stage, and resend another SETUP command. The CPU code must be able to operate correctly in this situation.

After completion of the data stage, a status stage OUT transaction occurs. The USB host sends a zero-length data packet, and the CPU code must return its completion status for the control read standard request via standard handshaking mechanisms.

NOTE: In the case of returning exactly what the host requested and that request was a multiple of the maximum packet size, no zero-length packet is required. A zero-length packet is required only when the amount of data the device has to return is less than the amount requested by the host and the amount returned is a multiple of the maximum packet size (source USB forum).

32.3.7.4.1 Non-Autodecoded Control Read Transfer Handshaking

Handshaking for the setup stage of non-autodecoded control read transfers is forced by the USB module to always be ACK, unless there is a data error in the packet, in which case the USB module ignores the transaction. If the setup packet has a DATA1 PID instead of a DATA0 PID, the setup transaction is ignored (error).

Data stage handshaking for non-autodecoded control read transfers is dependent on the endpoint 0 STAT_FLG.FIFO_EN, STAT_FLG.EP_HALTED, and SYSCON2.STALL_CMD bits. The handshaking information is used during the data phase of the data stage transaction. The USB specification requires that once STALL is signaled in a control transfer, it must be signaled until the next setup token is received. The SYSCON2.STALL_CMD and CTRL.SET_HALT (reflected through the STAT_FLG.EP_HALTED register bit) register bits provide this functionality. STAT_FLG.EP_HALTED does not reflect the forced STALL caused by SYSCON2.STALL_CMD; it retains its previous value.

The status stage is controlled by the CTRL.FIFO_EN and SYSCON2.STALL_CMD bits.

Successful completion of non-autodecoded control read transfers is indicated by the host sending an OUT token followed by an empty packet and the USB device controller responding with ACK. If the data packet sent by the USB host during the status stage of a control read request is not empty, the OUT transaction is accepted by the core, but OUT data is not put into the endpoint 0 RX FIFO. If the status packet has a DATA0 PID instead of a DATA1 PID, a STALLED is returned by the core and an interrupt is asserted.

32.3.7.4.2 Non-Autodecoded Control Read Transfer Error Conditions

If an error occurs while dealing with the control read, which the CPU cannot deal with itself, it must signal STALL to the USB host for all subsequent transactions until a new setup token to endpoint 0 occurs. This is true for both data stage and status stage transactions. This is most conveniently done by setting endpoint 0 SYSCON2.STALL_CMD bit, which causes stalling of all the remaining transactions of all remaining stages of a non-autodecoded control transfer, up to the reception of the next valid SETUP command.

Error conditions are handled as for BULK/INTERRUPT transactions. The USB device controller module responds to control read status stage transactions that have a bad token or bad data by not sending a handshake packet. In both cases, the transaction is ignored and no general USB interrupt is generated to the CPU.

32.3.7.5 Autodecoded Versus Non-Autodecoded Control Requests

Table 32-55. Autodecoded Versus Non-Autodecoded Control Requests ⁽¹⁾ ⁽²⁾ ⁽³⁾ ⁽⁴⁾

Request	Recipient	Status	CPU Required Action	Device Behavior if Device Is Not Configured
GET_STATUS	Device	Autodecoded (function of AUTODEC_DIS register bit)	None	Device status is returned (SYSCON1.SELF_PWR and DEVSTAT.R_WK_OK bits).
	Interface	Non-autodecoded	The CPU must stall the command (via SYSCON2.STALL_CMD bit) if interface number is not correct. No feature is defined for interface.	Command is passed to the CPU.
	Endpoint	Autodecoded(function of AUTODEC_DIS register bit)	None	The core automatically stalls the command if endpoint number is different from 0.
CLEAR/SET FEATURE	Device	Autodecoded (function of AUTODEC_DIS register bit)	None (DS_CHG IT is asserted to the CPU after any DEVSTAT.R_WK_OK bit modification).	The core handles the request.
	Interface	Autodecoded (function of AUTODEC_DIS register bit)	None. (No feature is defined in USB 1.1 spec for interface. These requests are stalled).	Command is stalled anyway.
	Endpoint	Non-autodecoded	The CPU must stall the command (via SYSCON2.STALL_CMD bit) if endpoint number/type/direction is not correct. The CPU must reset the EP after having handled the pending transactions (if CLEAR) or set halt condition (if SET). For EP 0, CPU must only clear or set halt condition; FIFO and data PID is always correct for next setup.	Command is passed to the CPU.
SET_ADDRESS	Device	Autodecoded	None (see ⁽⁵⁾) (Whether the device is addressed or not is available in DEVSTAT register. A valid SET_ADDRESS request with address number from 0 generates a DS_CHG interrupt to the CPU).	Default: device moves in the addressed state if address number is different from 0. Addressed: device takes the new address value or moves in default state if address number is 0. Configured: request is STALLED.

⁽¹⁾ Transactions on endpoints other than zero are ignored if the device is not configured (addressed state).

⁽²⁾ If some endpoints are not used by the interface currently set, transactions on these endpoints are not ignored; the CPU must set halt feature for the endpoint. This does not happen if USB host works correctly.)

⁽³⁾ If endpoint 0 is halted, per *USB 1.1 Specification* (see 9.4.5: Get_Status), all requests are stalled except GET_STATUS, CLEAR_FEATURE, and SET_FEATURE requests.)

⁽⁴⁾ Requests are handled with respect to USB 1.1 when specified as such, but many device reactions are not specified by USB 1.1.)

⁽⁵⁾ During a SET_ADDRESS autodecoded command, only the seven LSBs are significant for the new address (decimal value from 0 to 127); all others are discarded.

Table 32-55. Autodecoded Versus Non-Autodecoded Control Requests ^{(1) (2) (3) (4)} (continued)

Request	Recipient	Status	CPU Required Action	Device Behavior if Device Is Not Configured
GET_DESCRIPTOR	All	Non-autodecoded	The CPU must write descriptor data into endpoint 0 FIFO.	Command is passed to the CPU.
SET_DESCRIPTOR	All	Non-autodecoded	The CPU must stall the command (via SYSCON2.STALL_CMD bit) if it does not support set descriptor requests.	Command is passed to the CPU.
GET/SET CONFIGURATION	Device	Non-autodecoded	The CPU must stall the command (via SYSCON2.STALL_CMD bit) if configuration number is not correct. If the request is SET_CONFIG, the CPU must reset all endpoints, halt endpoints not used by the default interface setting, set SYSCON1.SELF_PWR value if device is self-powered for the configuration set, and then set SYSCON2.DEV_CFG bit (if config nb is not 0), or set SYSCON2.CLR_CFG bit (if config nb is 0) before allowing status stage to complete. The device moves to configured state (if DEV_CFG set), or moves to addressed state (if CLR_CFG set) and a DS_CHG interrupt is asserted to the CPU.	Command is passed to the CPU.
GET/SET INTERFACE	Interface	Non-autodecoded	The CPU must stall the command (via SYSCON2.STALL_CMD bit) if interface/setting number is not correct. If the request is SET_INTERFACE, the CPU must reset endpoints used by the interface and halt endpoints not used by the interface setting before allowing status stage to complete.	Command is passed to the CPU.
SYNCH_FRAME	Endpoint	Non-autodecoded	The CPU must stall the command if it does not support SYNCH_FRAME request, else write requested data in the endpoint 0 FIFO.	Command is passed to the CPU.

32.3.7.6 Note on Control Transfers Data Stage Length

The control transfer data stage length is indicated in the setup data packet.

During control reads, if the USB host requests more data than indicated in the setup packet, unexpected IN transaction is STALLED, causing STALL handshake for all remaining transactions of the transfer until next SETUP. If the USB host requires less data than indicated in the setup packet, the transfer is not STALLED. However, if the host moves to status stage earlier than expected for a non-autodecoded request, the OUT status stage is NAKed because the CPU has not enabled the RX FIFO.

During control writes, if the USB host sends more bytes than indicated in the setup packet, the transfer is STALLED. If the USB host sends fewer bytes than were expected, the request is accepted. But if the USB host moves to status stage earlier than expected for a non-autodecoded request, the IN status stage is NAKed because the CPU has not enabled the TX FIFO.

32.3.8 USB Device Initialization

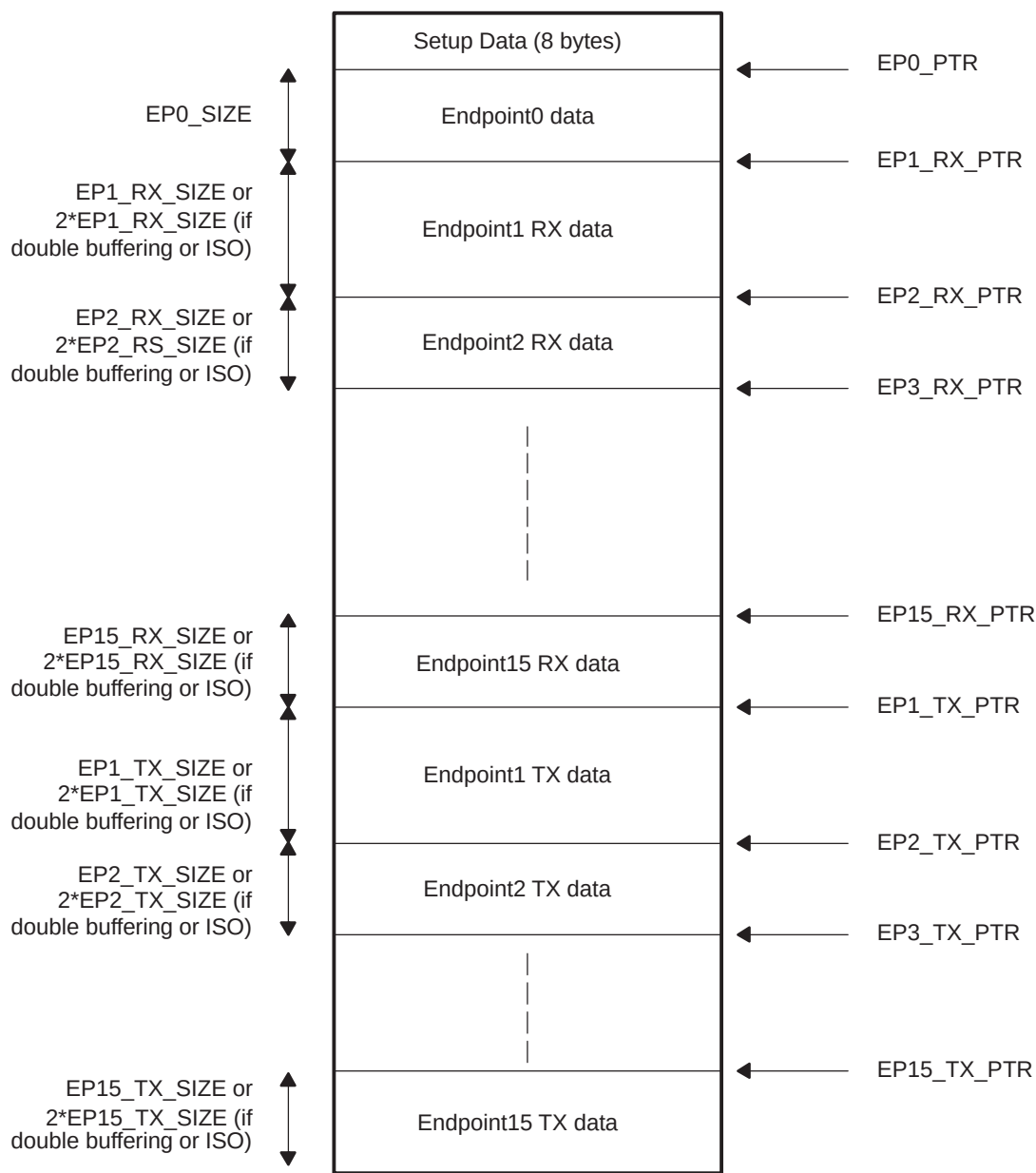
To allow communication between the device and a USB host, the CPU must configure the device by filling the configuration registers.

For each endpoint, the CPU must write on the dedicated register:

- Endpoint size
- Whether double-buffering is allowed for endpoint or not
- Endpoint type (ISO or non-ISO)
- Address of the pointer

System software must choose how to allocate the 2040 available bytes of USB device controller RAM to the USB endpoints. Receive endpoint size and type are configured using the EP1_RX through EP15_RX registers. Transmit endpoint size and type are configured using the EP1_TX through EP15_TX registers. Figure 32-57 shows an example of the RAM organization, obtained by following the flowchart shown in Figure 32-58.

Figure 32-57. Example of RAM Organization



Once the endpoints are configured, the CPU must set the SYSCON1.CFG_LOCK bit. If this bit is not set, all transactions are ignored by the core. Then, when the CPU is ready to communicate with the USB host, it must set the SYSCON1.PULLUP_EN bit. The CPU can wait until the DS_CHG attach interrupt has been detected and handled before setting the SYSCON1.PULLUP_EN bit. The USB host cannot detect the device until this bit is set.

Figure 32-58 and Figure 32-59 show flowcharts for the configuration phase.

Figure 32-58. Device Configuration Routine

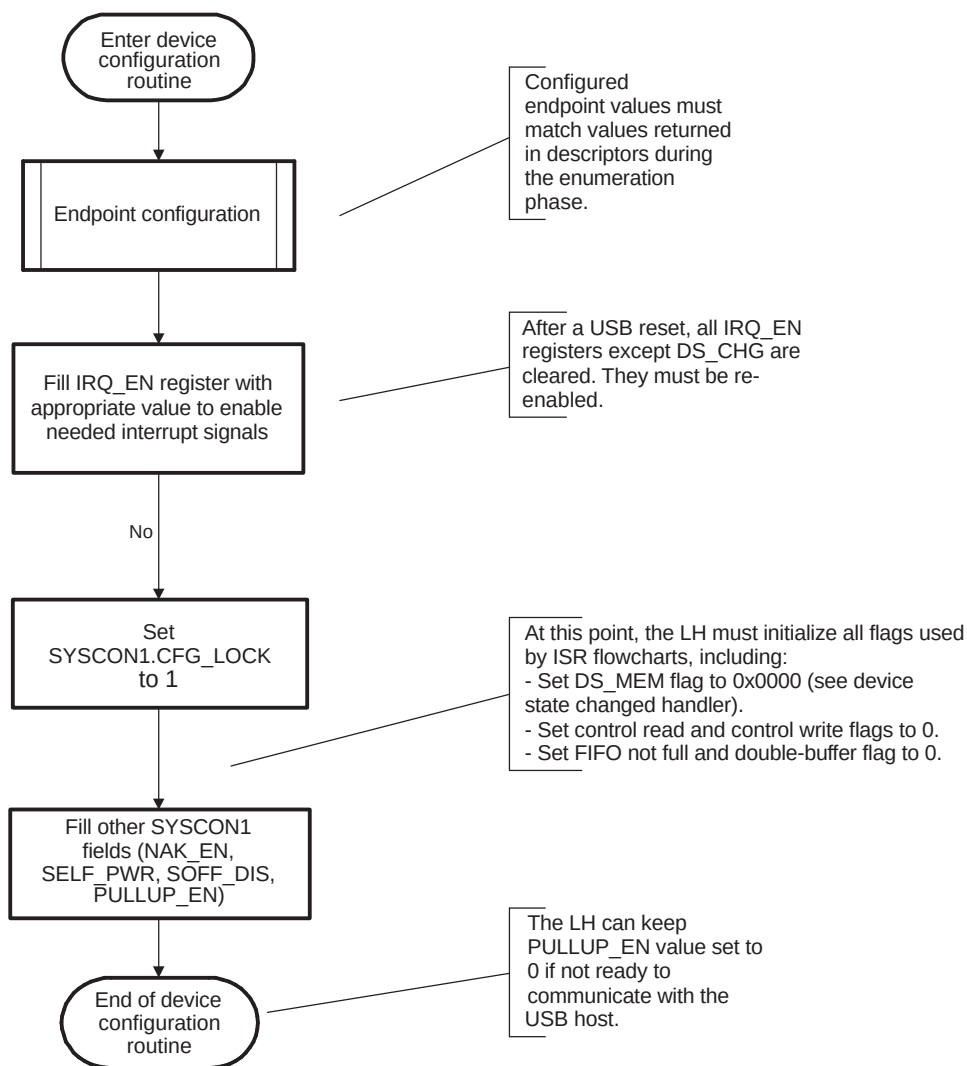
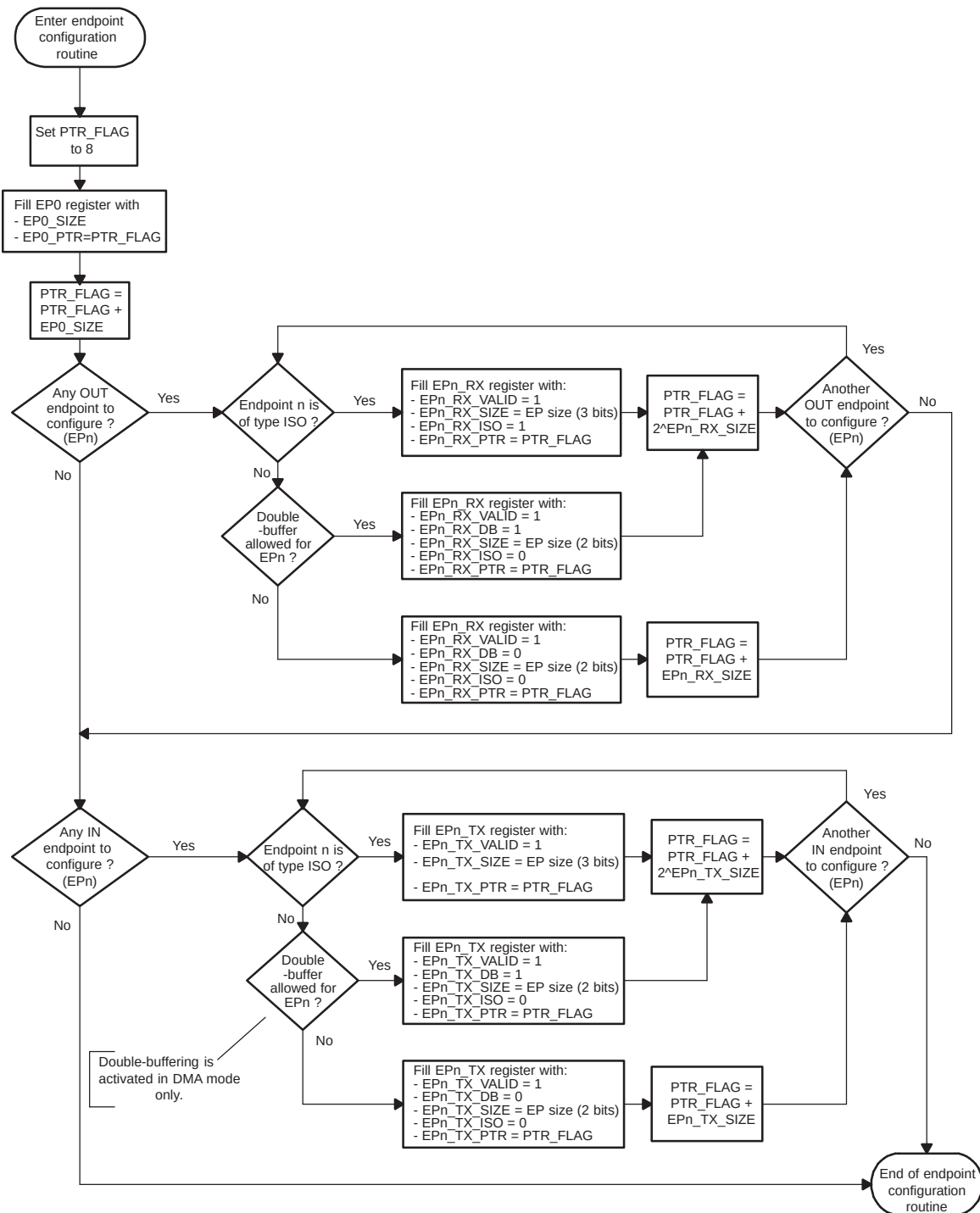


Figure 32-59. Endpoint Configuration Routine


32.3.9 Preparing for Transfers

To avoid NAK handshakes for the first transaction on an endpoint, the CPU must prepare the endpoint FIFO for receiving or transferring data. After the first transaction, the FIFO is enabled during the interrupt handling (ISR flowchart).

For receive endpoints, this phase consists of enabling the FIFO to receive data from the USB host. If double buffering is allowed for the endpoint, setting the CTRL.SET_FIFO_EN bit enables both FIFOs. Therefore, it is not possible to allow a single transaction when double buffering is used.

The CPU enters the prepare for USB RX transfers routine, presented once after the enumeration phase, and then properly reacts to EP interrupts. Whether double-buffering is allowed or not is transparent to the CPU, unless both FIFOs are cleared through a CTRL.CLR_EP or CTRL.RESET_EP. In that case, and in the case where the CPU finishes to handle an interrupt without having set the CTRL.SET_FIFO_EN bit, the CPU must reenter the prepare for USB RX transfers routine.

For transmit endpoints, the CPU enters the prepare for endpoint n TX transfer routine, presented each time a new file must be transmitted from endpoint n to USB host. The CPU must not enter this routine until data written into TX FIFO from the previous transfer have all been received successfully by the USB host (ACK interrupt received), unless TX FIFO is cleared through the CTRL.CLR_EP or CTRL.RESET_EP bits (see Figure 32-60 and Figure 32-61).

NOTE: This does not apply to endpoint 0, which is not used before a setup interrupt occurs. At setup interrupt, the CPU reacts appropriately, and enables EP0 FIFO only if necessary.

To ensure proper usage of the module, you cannot prepare data on different endpoints at the same time. You can enter a routine once the routine is not being used by another endpoint (no parallelism); the EP_NUM register can be accessed via different routines.

Figure 32-60. Prepare for USB RX Transfers Routine

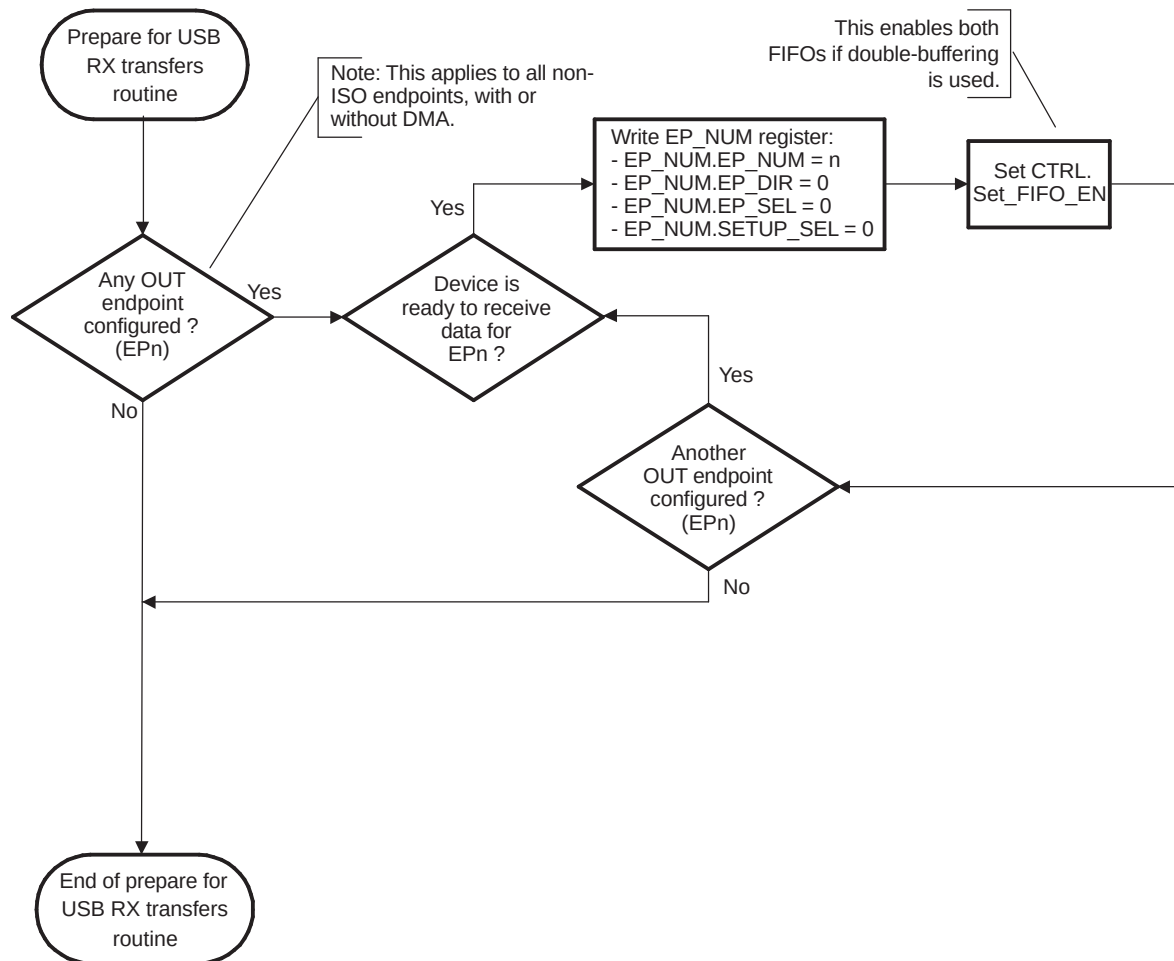
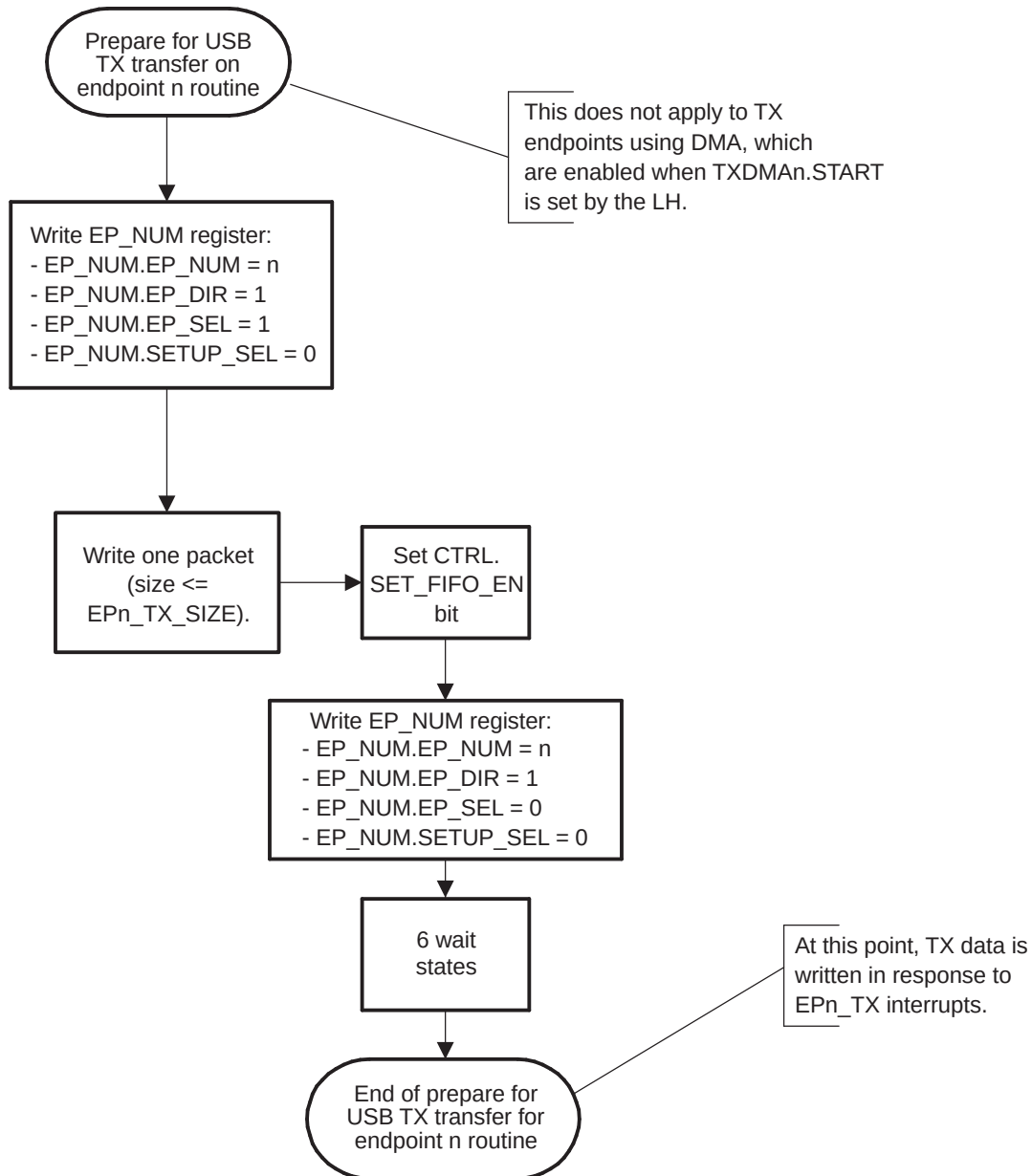


Figure 32-61. Prepare for TX Transfer on Endpoint n Routine


32.3.10 USB Device Interrupt Service Routine (ISR) Flowcharts

The flowcharts in this section give general operational guidelines for USB device ISR processing. System-architecture-specific details are left to the engineers who write the CPU and USB host code. One USB-specific interrupt register is provided (IRQ_SRC), including:

- General USB interrupts (including endpoint 0, DMA and device states interrupts) on Vectored Interrupt Manager (VIM) channel # 69 (by default)
- Non-ISO endpoint-specific interrupt on VIM channel # 70 (by default)
- Start of frame (SOF) interrupt for ISO transactions on VIM channel # 68 (by default)

The general USB interrupt ISR must handle non-autodecoded control transfers on endpoint 0 and some specialty interrupts generated because of USB device state modifications or DMA transfers. The ISR for the endpoint-specific interrupt must handle interrupts from the USB module that are generated because of USB activity for non-isochronous endpoints. The SOF ISR is responsible for handling isochronous endpoints and, if needed by the application, tracking the USB frame number. Many flowcharts are presented in this chapter to provide guidelines for how to handle the interrupts related to the USB device controller module. The flowcharts in this part suppose that the SYSCON1.NAK_EN bit is cleared.

NOTE: A key assumption behind the flowcharts presented here is that the application provides separate buffers for each direction of endpoint, except for endpoint 0. The flowcharts read from these application buffers for IN transactions on TX endpoints and write to these application buffers for OUT transactions on RX endpoints.

The USB device controller does not support reentrant interrupts. Each USB device controller must be handled completely before handling another USB device controller interrupt. This restriction occurs because there is only one EP_NUM register, so endpoint control operations must be completed before working with another endpoint or endpoint direction.

32.3.11 Important Note on USB Device Interrupts

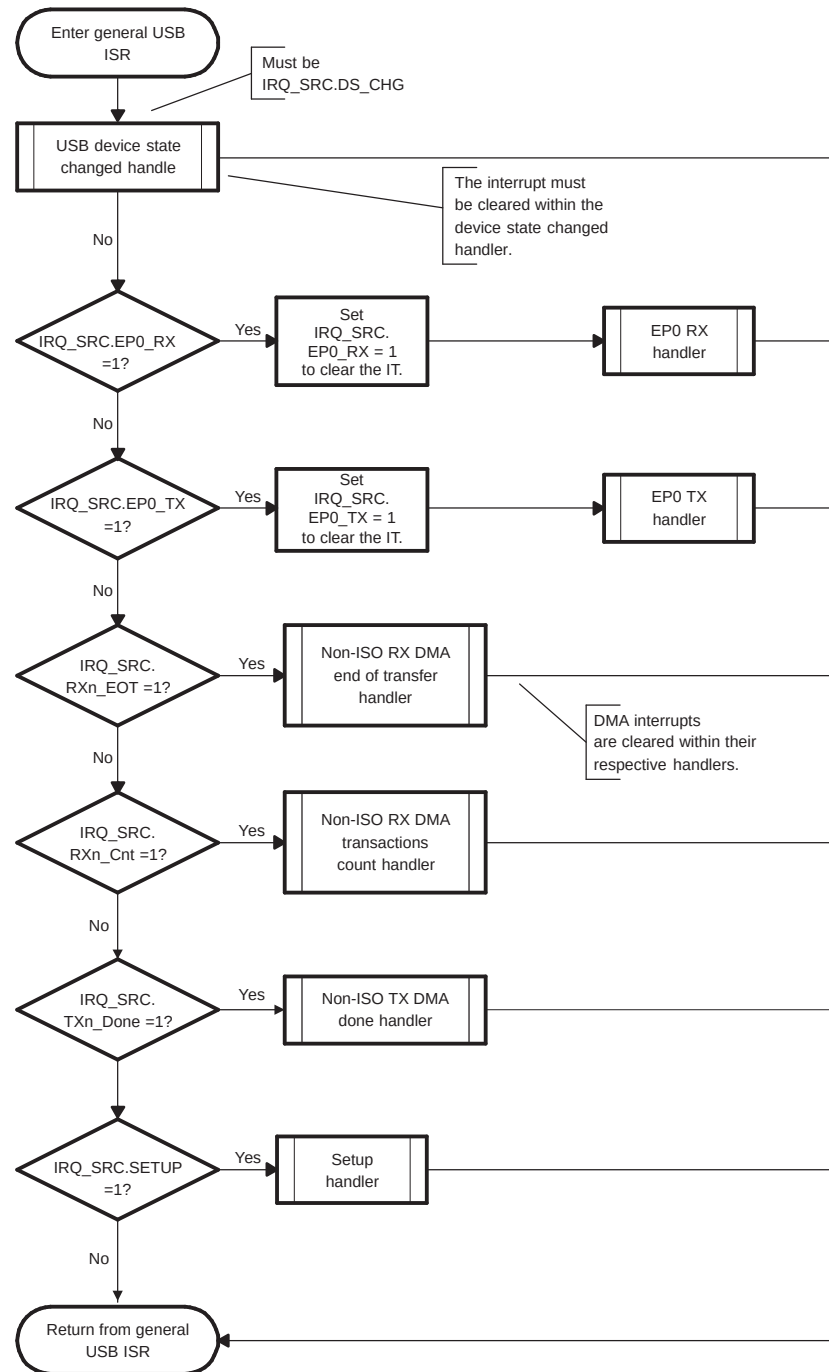
When an endpoint interrupt is asserted, the CPU writes the EP_NUM register with the EP_NUM.EP_SEL bit set to 1. The CPU must finish the interrupt handling before clearing the EP_NUM.EP_SEL bit, because clearing this bit clears the corresponding status bit in the STAT_FLG register (ACK, NAK, STALL). When an interrupt is pending on an endpoint, the CPU must not select and then unselect the endpoint without handling the interrupt, because this clears the pending transaction status flags. The CPU does not need to set EP_NUM.EP_SEL to 1 when setting CTRL.SET_FIFO_EN, CTRL.SET_HALT, and CTRL.CLR_HALT bits.

The endpoint status (STAT_FLG register) is updated at the end of each USB transaction if the previous transaction has been handled. If a pending interrupt has not been handled when a new non-transparent transaction occurs, status flags are not updated (and NAK is returned, even if FIFO was enabled, or STALLED if the EP halt feature was set), so that the CPU never misses an ACKed transaction. If double-buffering is used for an endpoint, STAT_FLG is updated if there is zero or one interrupt pending for the endpoint, and is not updated if there are already two interrupts pending on the endpoint.

The CPU does not need to set the SYSCON1.NAK_EN bit during normal operation. However, for debugging process, this bit can be set when the CPU finishes handling an EP interrupt without having set the corresponding CTRL.SET_FIFO_EN bit. During TX transaction, if the SYSCON1.NAK_EN bit is set, the CPU must wait for a NAK interrupt to write the TX data, to avoid a possible conflict caused by the NAK interrupt received while the CPU was writing the TX data.

32.3.12 Parsing General USB Device Interrupt

The general USB interrupt (VIM channel # 69 by default) ISR must parse the interrupt identifier register IRQ_SRC to determine the types of general USB interrupts that are active. These include interrupts relating to USB device state modifications (USB reset, suspend/resume, during enumeration phase) and control transfers on endpoint 0 or non-ISO DMA transfers in either receive or transmit mode. Multiple interrupts can be active at any time, and all must be dealt with by the ISR before returning from the ISR. [Figure 32-62](#) shows an appropriate flowchart for parsing the general USB interrupts.

Figure 32-62. General USB Interrupt ISR Source Parsing Flowchart


32.3.13 Setup Interrupt Handler

A separate interrupt flag exists for setup transactions so that the CPU cannot miss a setup transaction, even if it occurs during the data or status phase of another transfer (case of an aborted transfer). The setup parsing function captures the control transfer request information for use in determining which USB bus activity is needed and in controlling how the CPU must generate or respond to the control transfer. This information includes the following:

- bmRequestType
- bmRequest

- wValue
- wIndex
- wLength

The setup interrupt handler shown in [Figure 32-63](#) is responsible for processing setup transactions occurring on endpoint 0. It calls the routine that parses the control transfer request information, shown in [Figure 32-64](#) to set flags that the rest of the ISR code can use to control proper response to control transfers. Two flags are set by the setup interrupt handler, to be used during endpoint 0 interrupt handlers:

- Control read flag
- Control write flag

Figure 32-63. Setup Interrupt Handler

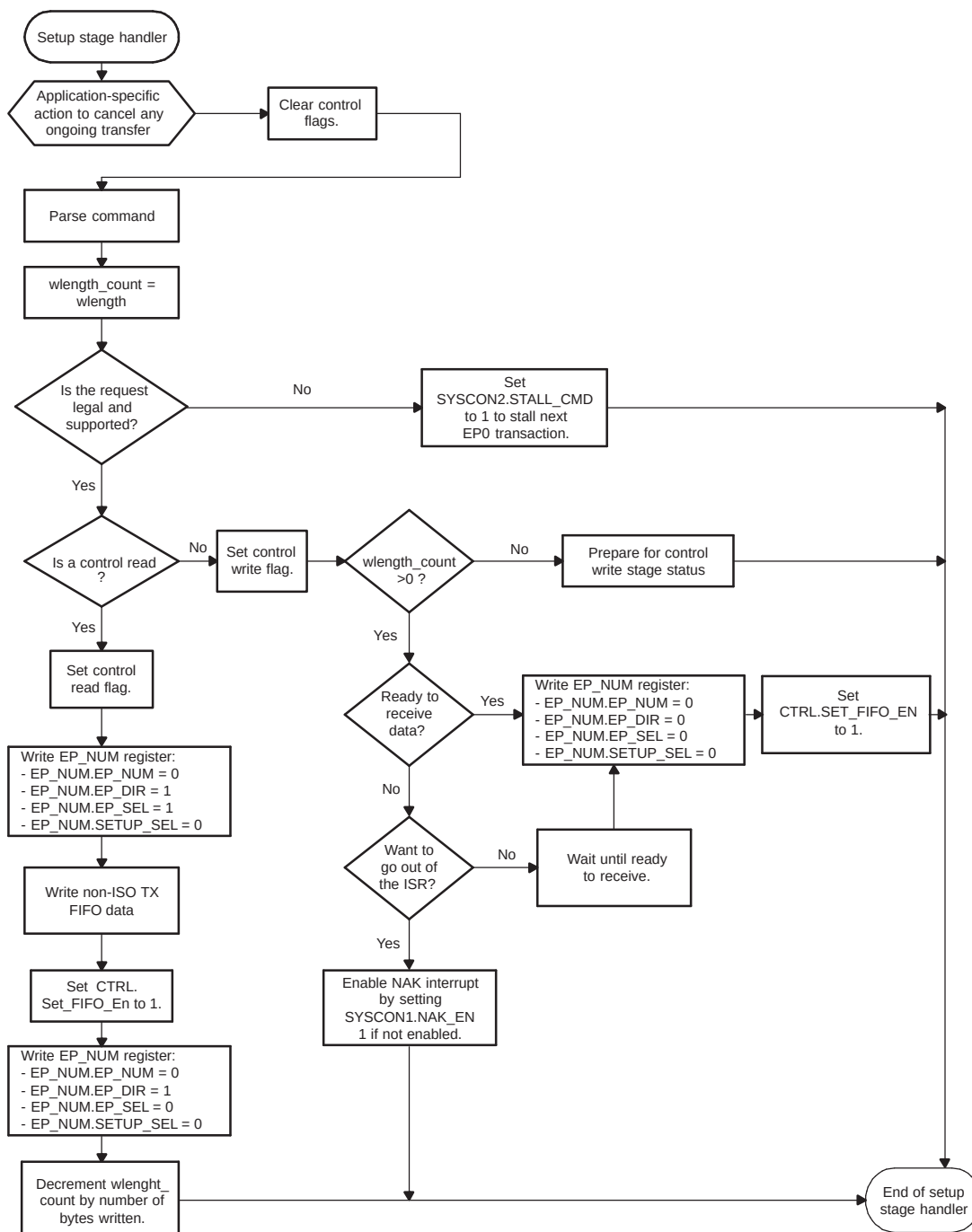
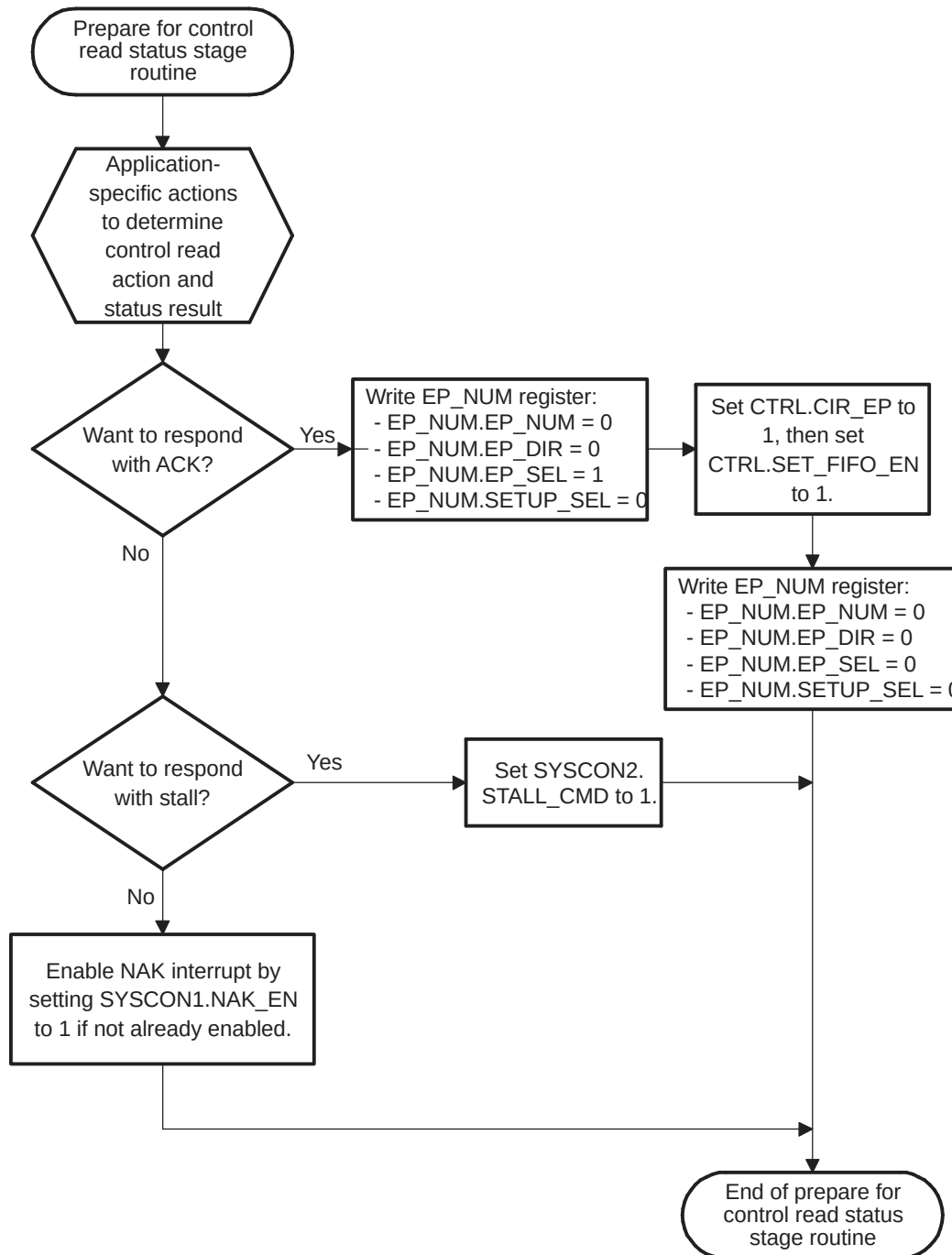


Figure 32-64. Parse Command Routine (Setup Stage Control Transfer Request)


32.3.14 Endpoint 0 RX Interrupt Handler

Figure 32-65 shows the endpoint 0 RX portion of the general USB interrupt handler, which must handle general USB interrupts related to control OUT transactions on endpoint 0. No EP0 interrupt is generated for autodecoded control transfers.

Figure 32-65. Endpoint 0 RX Interrupt Handler

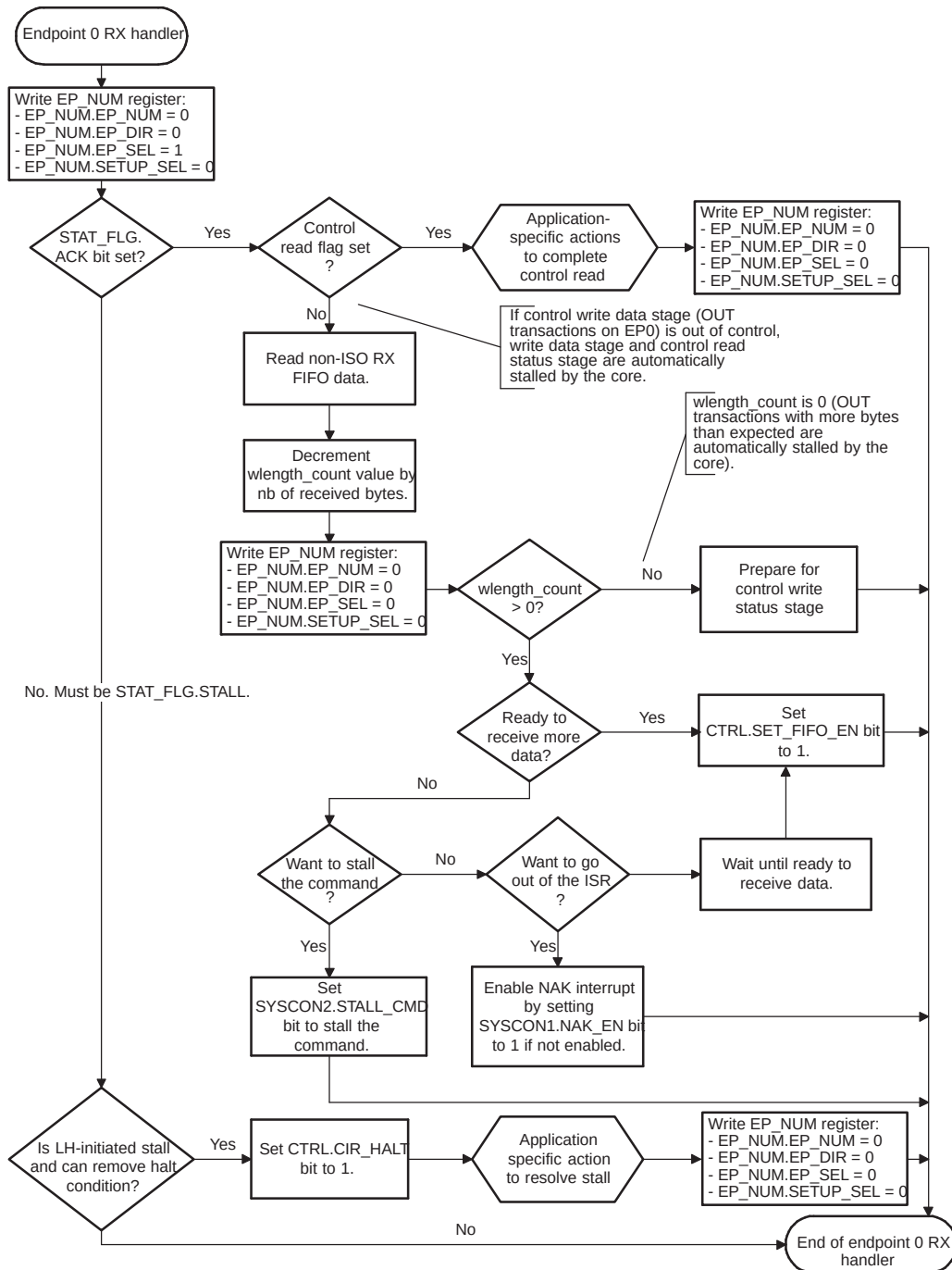
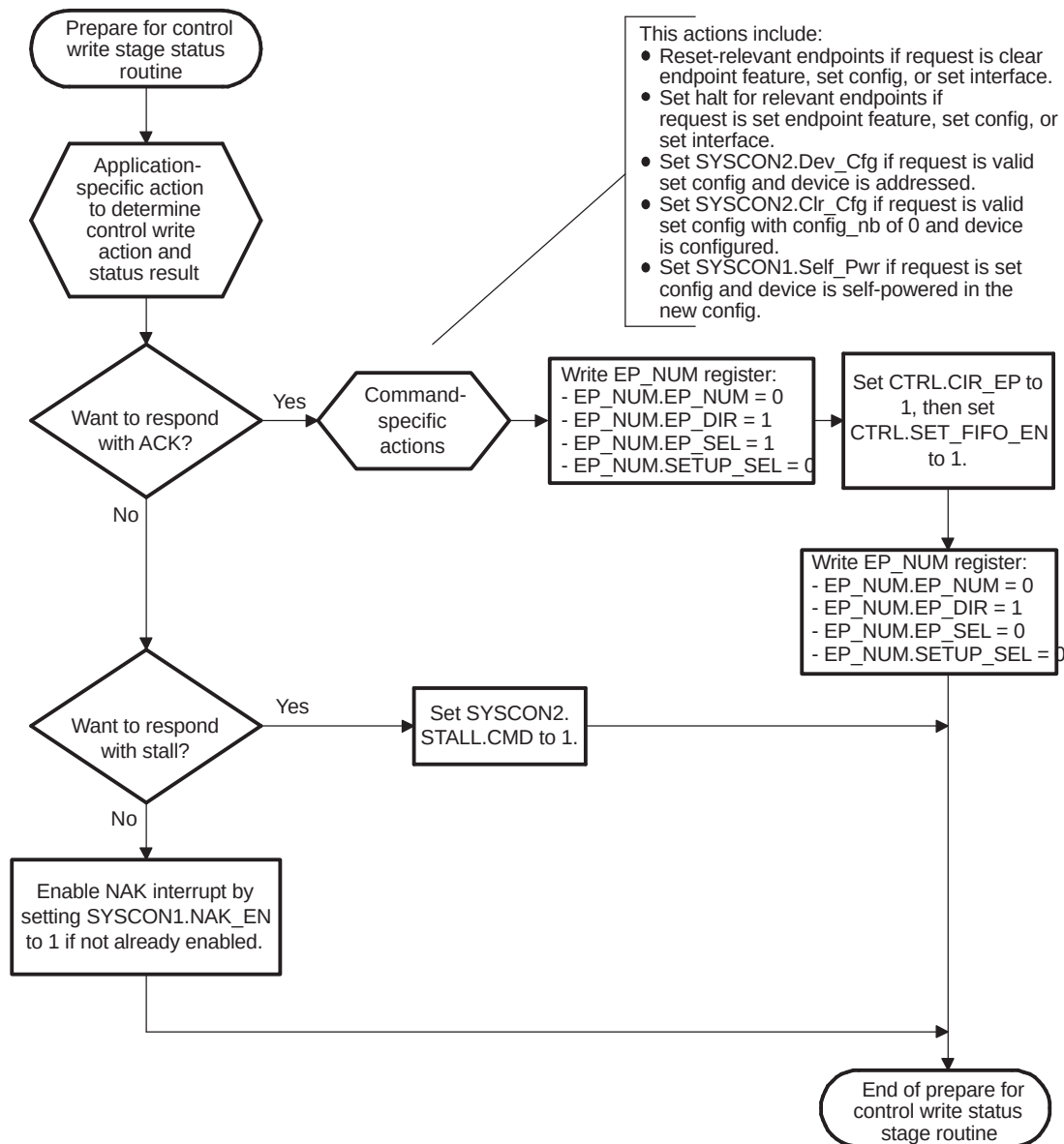


Figure 32-66. Prepare for Control Write Status Stage Routine


To support the SET_DESCRIPTOR command, when in commspecific actions, you must first reset all endpoints by selecting the endpoint and then clear it, unlock the configuration, change the new one, lock the configuration, go in prepare for transfers, and enable all interrupts (see [Figure 32-66](#)).

32.3.15 Endpoint 0 TX Interrupt Handler

[Figure 32-67](#) shows the TX portion of the general USB interrupt handler, which must handle general USB interrupts related to control IN transactions on endpoint 0.

The endpoint 0 TX interrupt handler must be able to move data into the endpoint 0 TX FIFO when the application buffer for endpoint 0 TX data is not empty and an endpoint 0 TX interrupt occurs signaling an ACKed non-autodecoded endpoint 0 IN transaction. This data can be control read data stage information or control write status stage handshaking information (see [Figure 32-68](#)).

Figure 32-67. Endpoint 0 TX Interrupt Handler

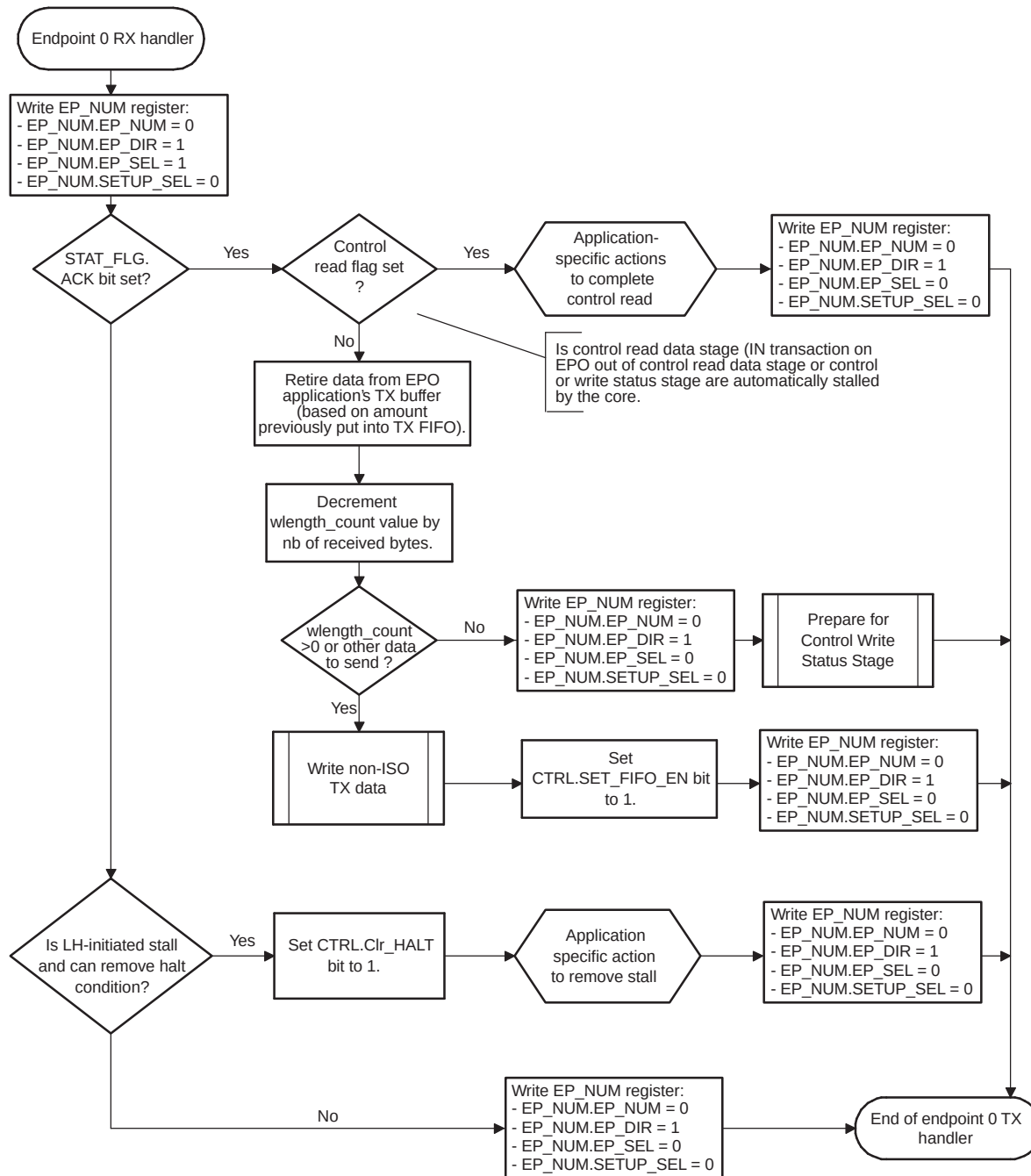
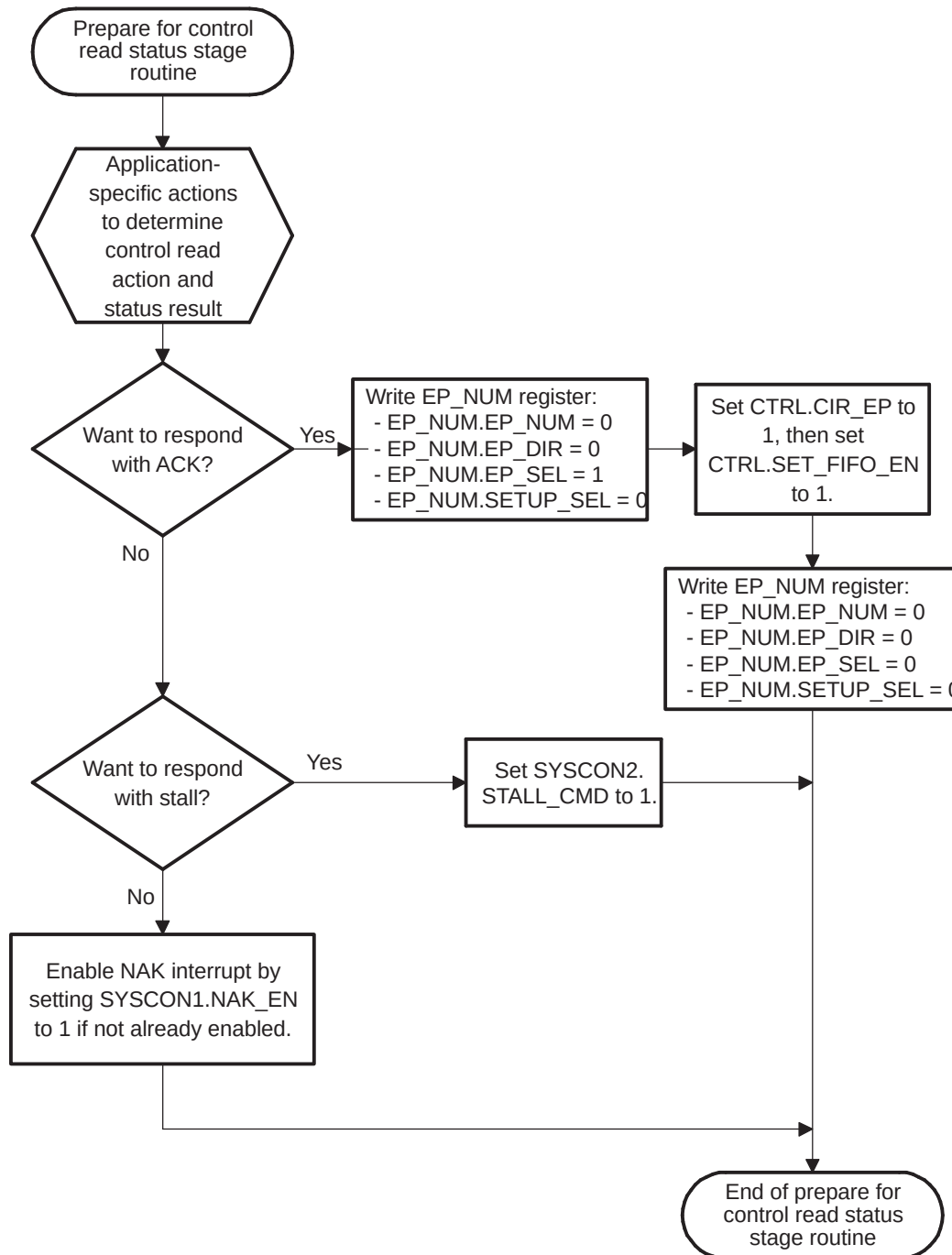


Figure 32-68. Prepare for Control Read Status Stage Routine


32.3.16 Device States Changed Handler

This section describes how USB device states and transitions states are decoded by the USB device controller and how they can be handled.

The state machine (see [Figure 32-69](#)) shows how the USB device controller device moves from one state to another state with respect to the USB1.1 specification. Attach/unattach transition is not shown in the transition flow.

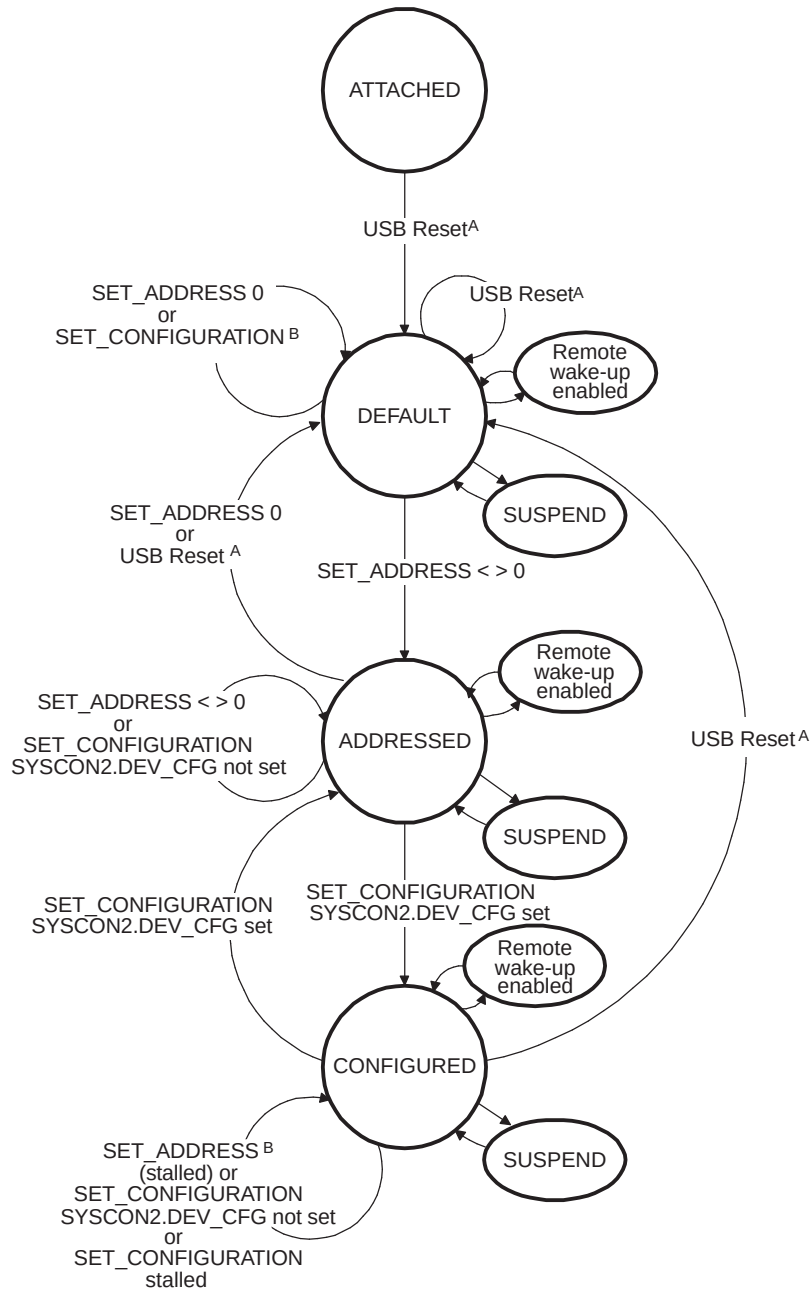
Because SET_CONFIGURATION is not decoded by the core, the CPU has the responsibility to distinguish a SET_CONFIGURATION with a nonvalid configuration value from other SET_CONFIGURATION requests and to set SYSCON2.DEV_CFG only if the configuration value is valid (value 0 is nonvalid), when the device is in addressed state. When the device is in configured state, the CPU has the responsibility to set SYSCON2.CLR_CFG if the configuration number is 0 so that the device moves to addressed state.

Device states are visible in the DEVSTAT register and are decoded as follows.

- Attached: The device is attached to the USB and powered.
- Default: The device is attached to the USB, is powered, and has been reset.
- Addressed: The device is attached to the USB, is powered, has been reset, and an address has been assigned. The device moves into the addressed state after a SET_ADDRESS request with an address number other than 0.
- Configured: The device is attached to the USB, is powered, has been reset, has an address other than 0, and is configured. The device moves into the configured state after a valid SET_CONFIGURATION request, only if the CPU has set the SYSCON2.DEV_CFG bit (meaning the configuration is valid).
- Suspended: The device is at minimum default and has not seen bus activity for 5 ms.
- Reset: When set, the device is receiving a valid USB host reset.
- R_WK_OK: This bit is set (cleared) automatically after a valid SET_DEVICE_FEATURE (CLEAR_DEVICE_FEATURE) request from the USB host.

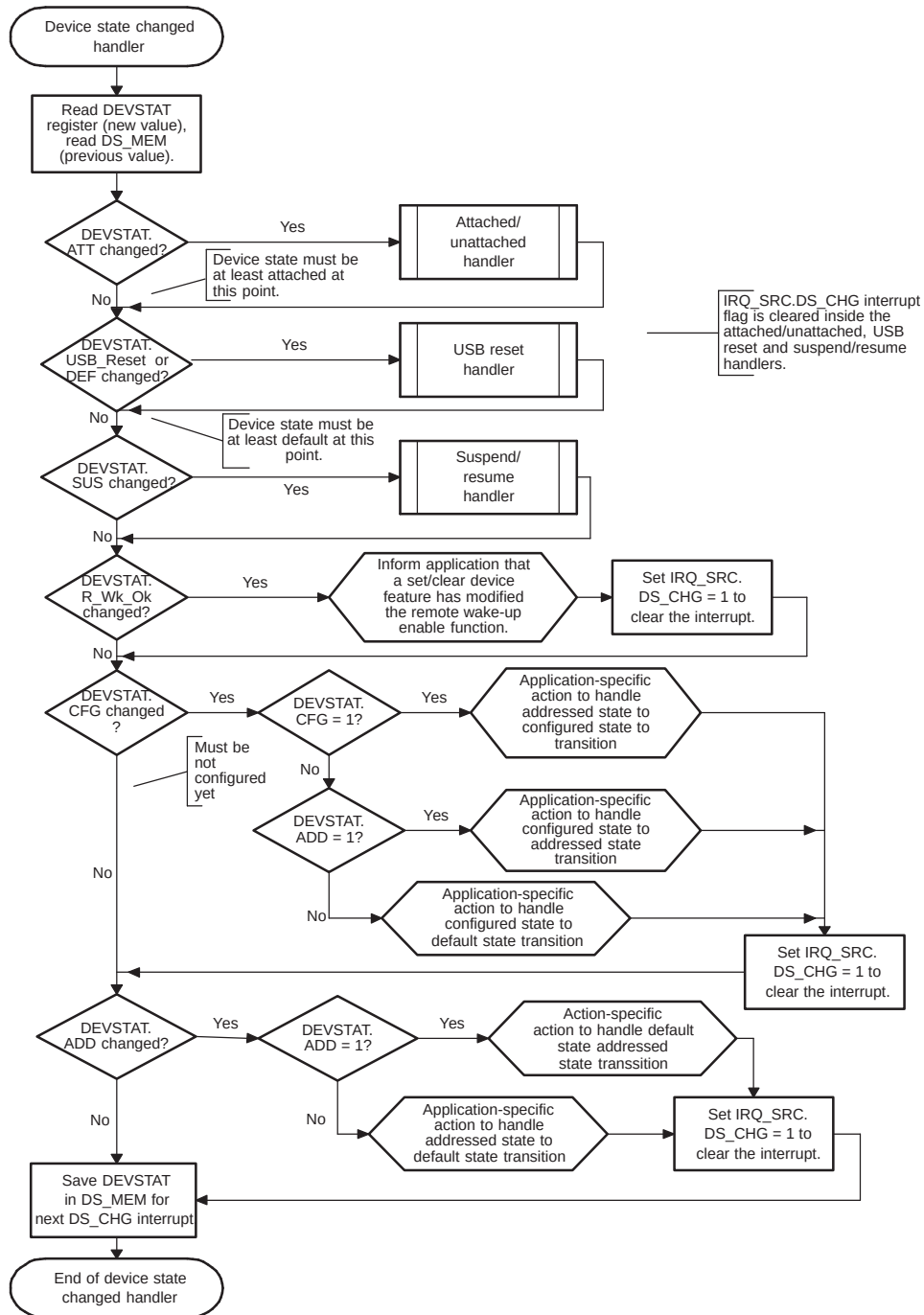
Any change in the DEVSTAT register bits triggers a device change interrupt (IRQ_SRC.DS_CHG), if enabled.

The device moves to addressed state after the status stage of a valid SET_ADDRESS, even if the status stage ACK handshake is received corrupted or not sent by the USB host. A SET_DEVICE_FEATURE or a CLEAR_DEVICE_FEATURE is effective after setup transaction, even if no status stage occurs. A SET_CONFIGURATION request is effective before status stage, when the CPU sets the SYSCON2.CLR_CFG or SYSCON2.DEV_CFG bit (see [Figure 32-70](#)).

Figure 32-69. USB Device Controller Device State Transitions


- A USB reset generates two interrupts (when USB reset is asserted and then when USB reset completes.)
- B No interrupt is asserted by the core for transitions shown with dashed lines.

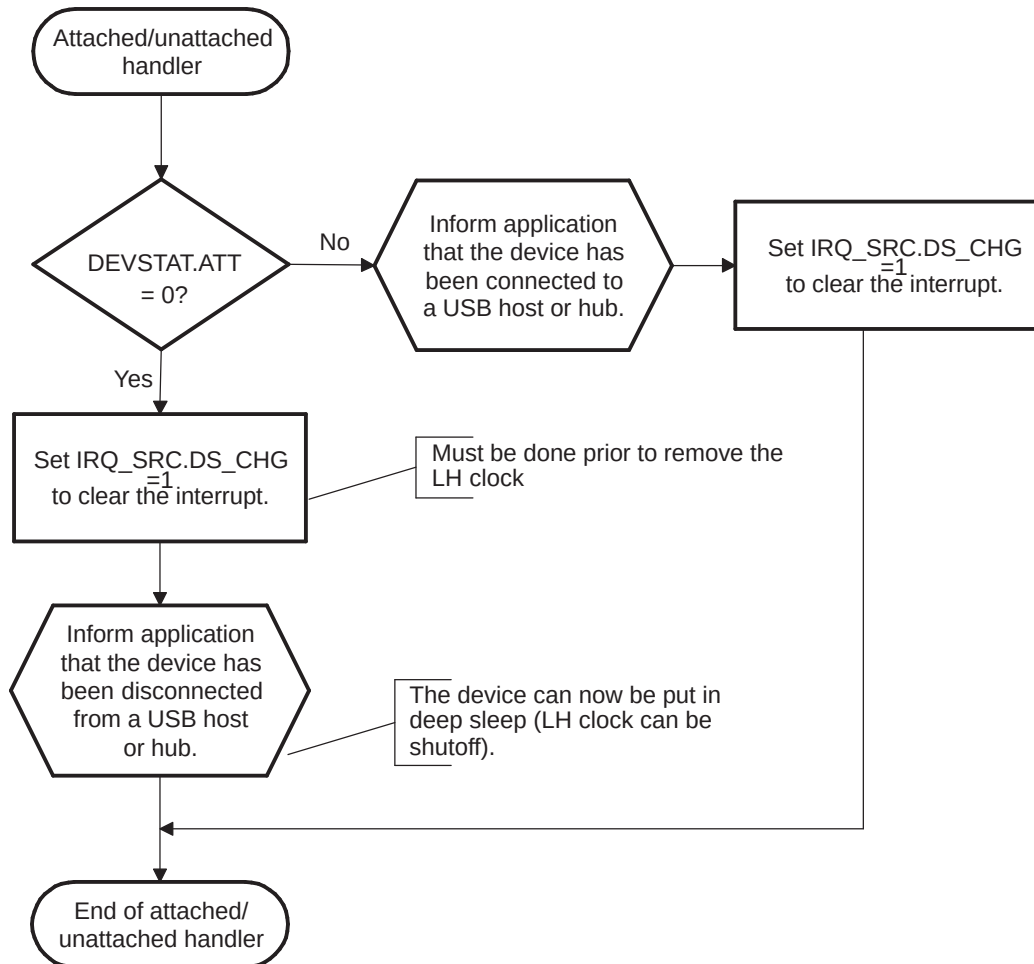
Figure 32-70. Typical Operation for USB Device State Changed Interrupt Handler



32.3.17 Device States Attached/Unattached Handler

Device attached/unattached interrupts occur when the device detects that its VBUS has changed. System software can disable the USB device controller clock after IRQ_SRC.DS_CHG is cleared after servicing an unattached interrupt. Disabling the USB device controller clock before IRQ_SRC.DS_CHG is cleared can result in improper functionality for future USB device controller interrupts (see [Figure 32-71](#)).

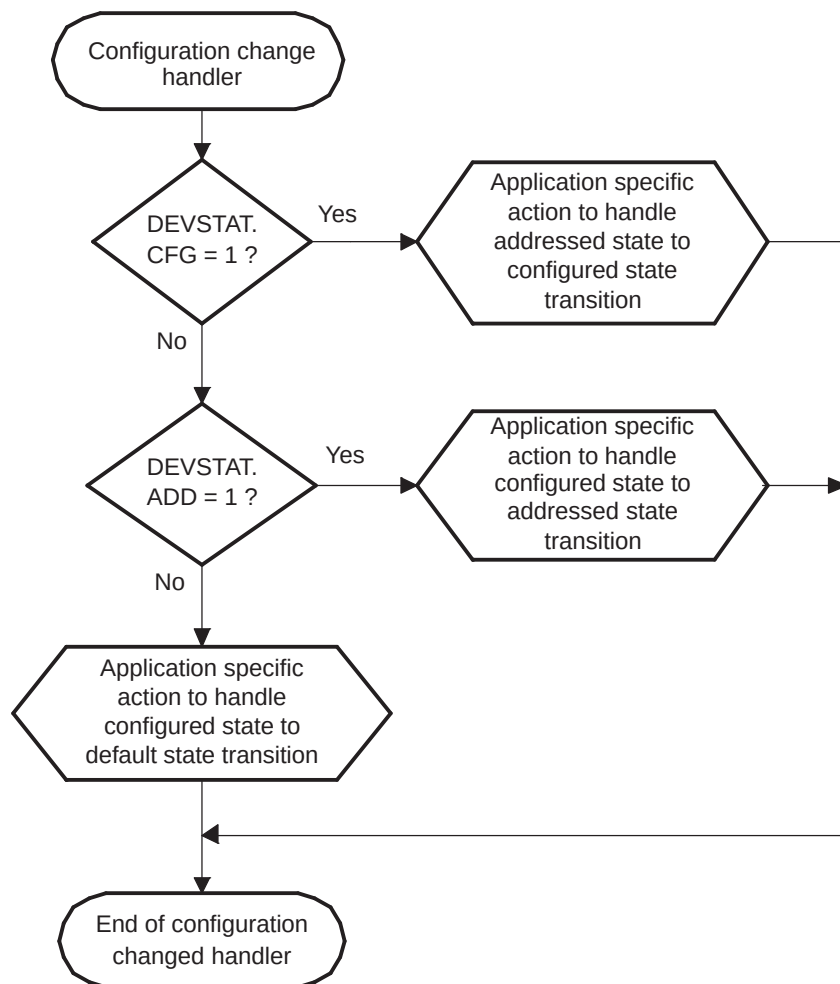
Figure 32-71. Attached/Unattached Handler



32.3.18 Device State Configuration Changed Handler

When a configuration changed interrupt occurs, the USB device has received a set configuration operation. When this occurs, the configuration-changed handler performs the operations shown in Figure 32-72.

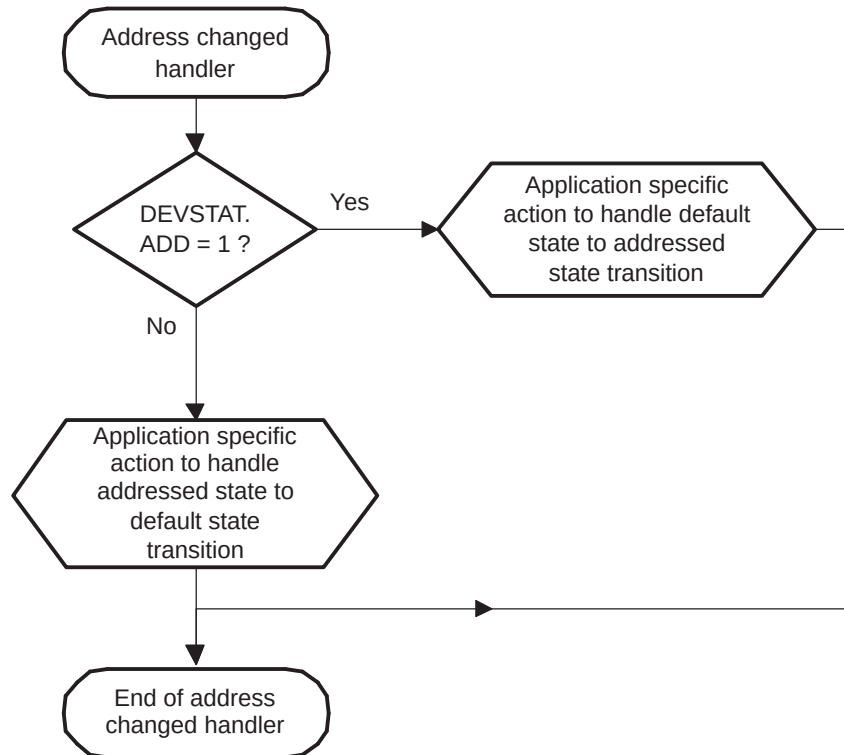
Figure 32-72. Configuration Changed Handler



32.3.19 Device State Address Changed Handler

When a address changed interrupt occurs, the USB device has received a set address operation. When this occurs, the address-changed handler performs the operations shown in [Figure 32-73](#).

Figure 32-73. Address Changed Handler

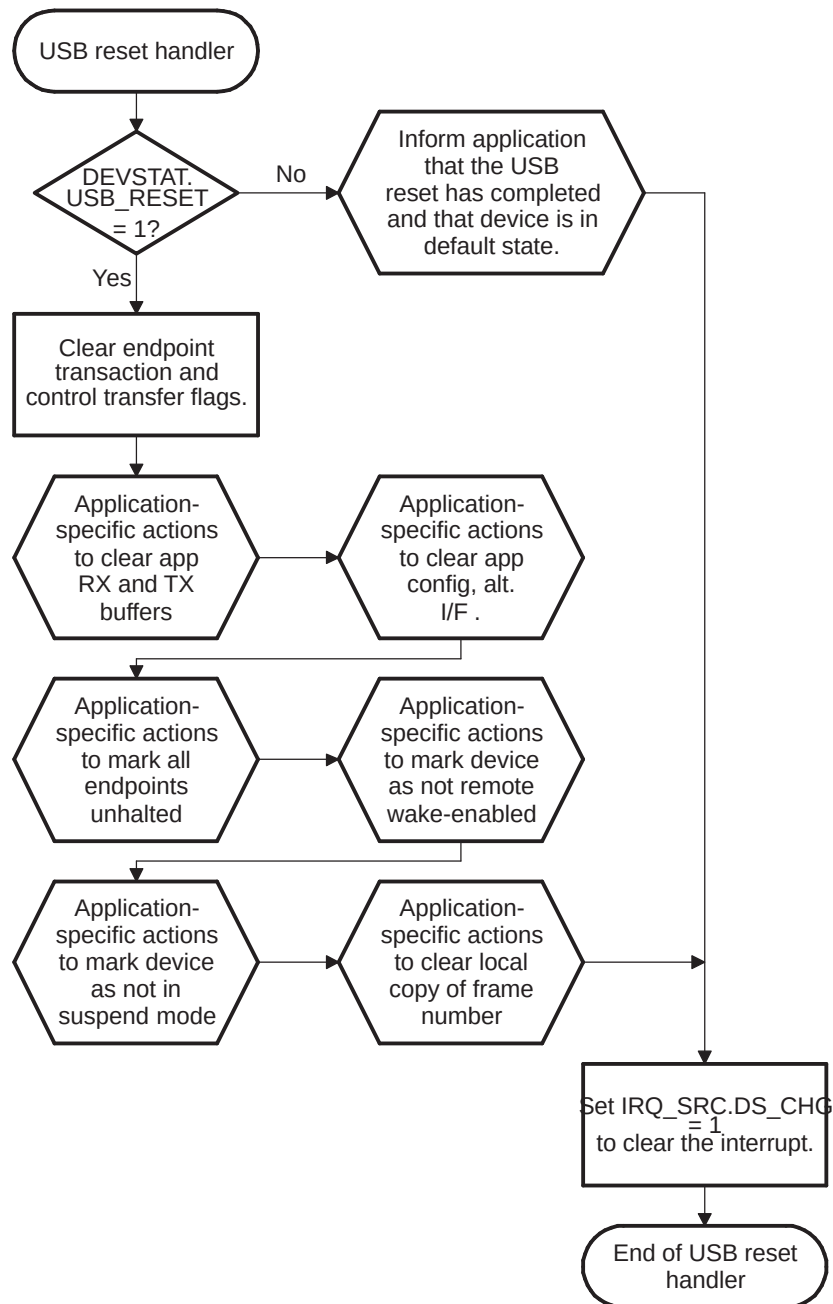


32.3.20 USB Device Reset Interrupt Handler

When a USB reset occurs, the USB module generates a general USB interrupt to the CPU (see [Figure 32-74](#)). The CPU responds to this interrupt by performing the following operations:

- Cancels any ongoing USB transaction and/or control transfer handling
- Clears any copies that the application has of configuration number or alternate interface numbers
- Clears any application-specific information relating to halted endpoints
- Clears any application-specific information relating to the remote wake enable flag
- Clears any application-specific information relating to the suspend mode flag
- Clears any application-specific copy of the frame number

Figure 32-74. USB Device Reset Handler Flowchart



32.3.21 Suspend/Resume Interrupt Handler

When a USB device suspend/resume general USB interrupt occurs, the USB module has either entered or left suspend mode. The CPU code must determine which and react appropriately (see [Figure 32-75](#)).

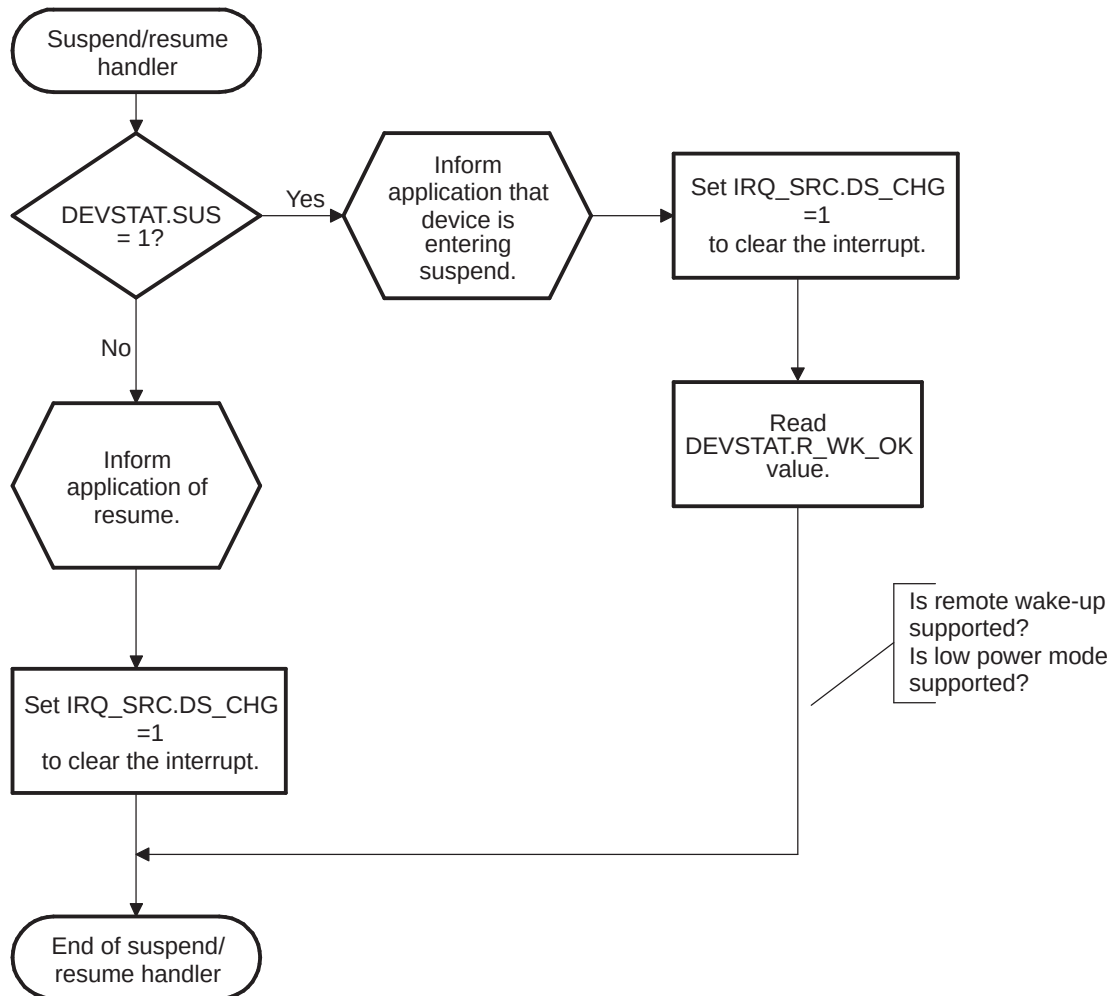
The suspend sense hardware is implemented to trigger only after 5 ms of bus idle. This forces compliance with the *USB Specification Version 1.1* T_{WTRSM} timing parameter (3 ms of IDLE to identify suspend, 2 ms before the remote device can signal resume).

If the CPU wants to wake the device from suspend mode and remote wake-up enable is set ($DEVSTAT.R_WK_OK = 1$), it must first turn its clock on (if stopped), and then set $SYSCON2.RMT_WKP$. The device then drives resume.

If shutoff is enabled (SYSCON1.SOFF_DIS=0), the 48-MHz clock is automatically shut off at suspend and turned on at resume (USB host or CPU driven). Setting or not setting the SYSCON1.SOFF_DIS bit is part of the device configuration. However, the CPU can modify its value at suspend interrupt time if necessary.

A USB reset is also a valid way to exit suspend mode. But the suspend/resume handler and the USB reset handler do not have to take this into account, because three interrupts are generated in that case (one for resume, one for reset, and one for end of reset).

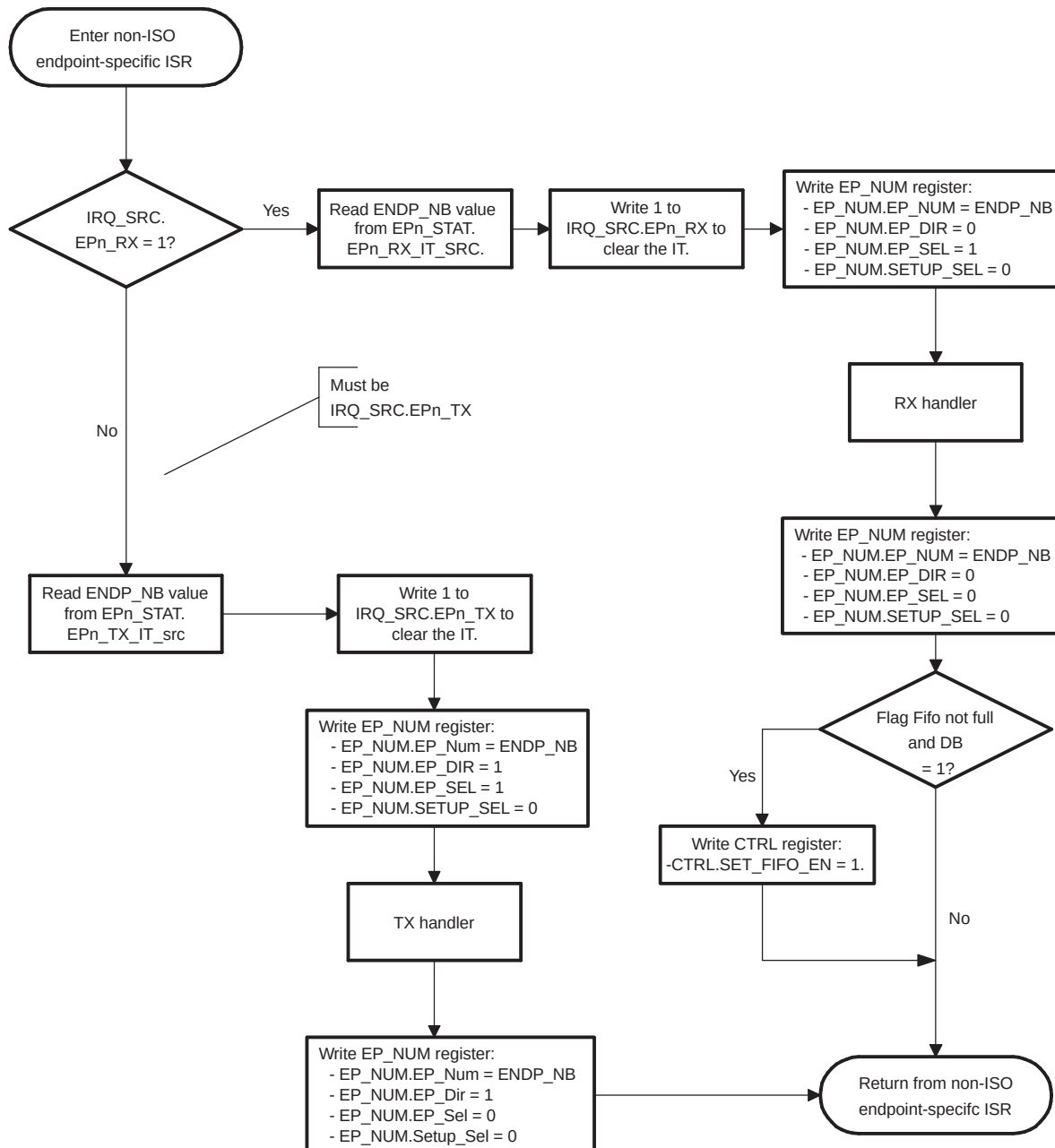
Figure 32-75. Typical Operation for USB Suspend/Resume General USB Interrupt Handler



32.3.22 Parsing Non-ISO Endpoint-Specific Interrupt

The endpoint-specific interrupt ISR (also known as the non-isochronous interrupt, on VIM channel # 70 by default) must parse the interrupt identifier register IRQ_SRC to determine the interrupts that are active (IRQ_SRC.EPN_RX, IRQ_SRC.EPN_TX, or both). Two interrupts can be active at any time. The ISR must then read the EPN_STAT register to determine the endpoint causing the interrupt. For each direction, only one endpoint interrupt can be active at a time (see [Figure 32-76](#)).

Figure 32-76. Non-ISO Endpoint-Specific (Except EP 0) ISR Flowchart



32.3.23 Non-ISO, Non-Control OUT Endpoint Receive Interrupt Handler

Figure 32-77 shows the operations necessary to handle non-ISO, non-control OUT endpoint-specific receive interrupts. This flowchart shows two different RX transaction handshaking interrupts. There is a third interrupt handshaking possibility when NAK interrupts are enabled, which is not depicted here. Depending on the application-specific actions needed for various endpoints in the real system, it is possible to use one routine that is common to all of the non-ISO, non-control receive endpoints, where the only differences are in the EP_NUM register value set and the selection of the proper application RX data buffer in the read non-ISO RX FIFO data routine (see Figure 32-78).

This flowchart does not attempt to document control endpoint 0 receive interrupts, which are discussed separately because of the more complex three-stage transfer mechanism used for control writes.

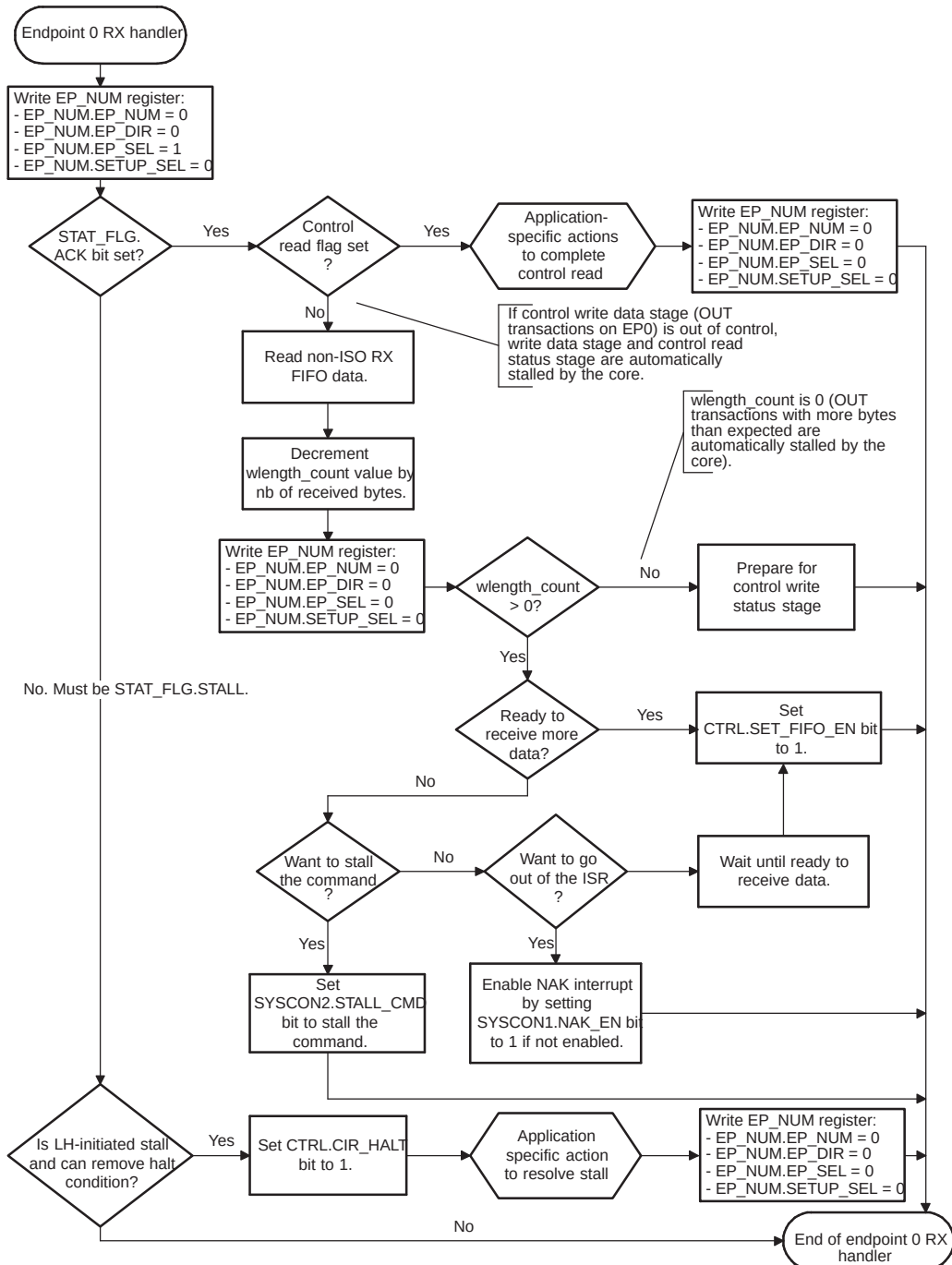
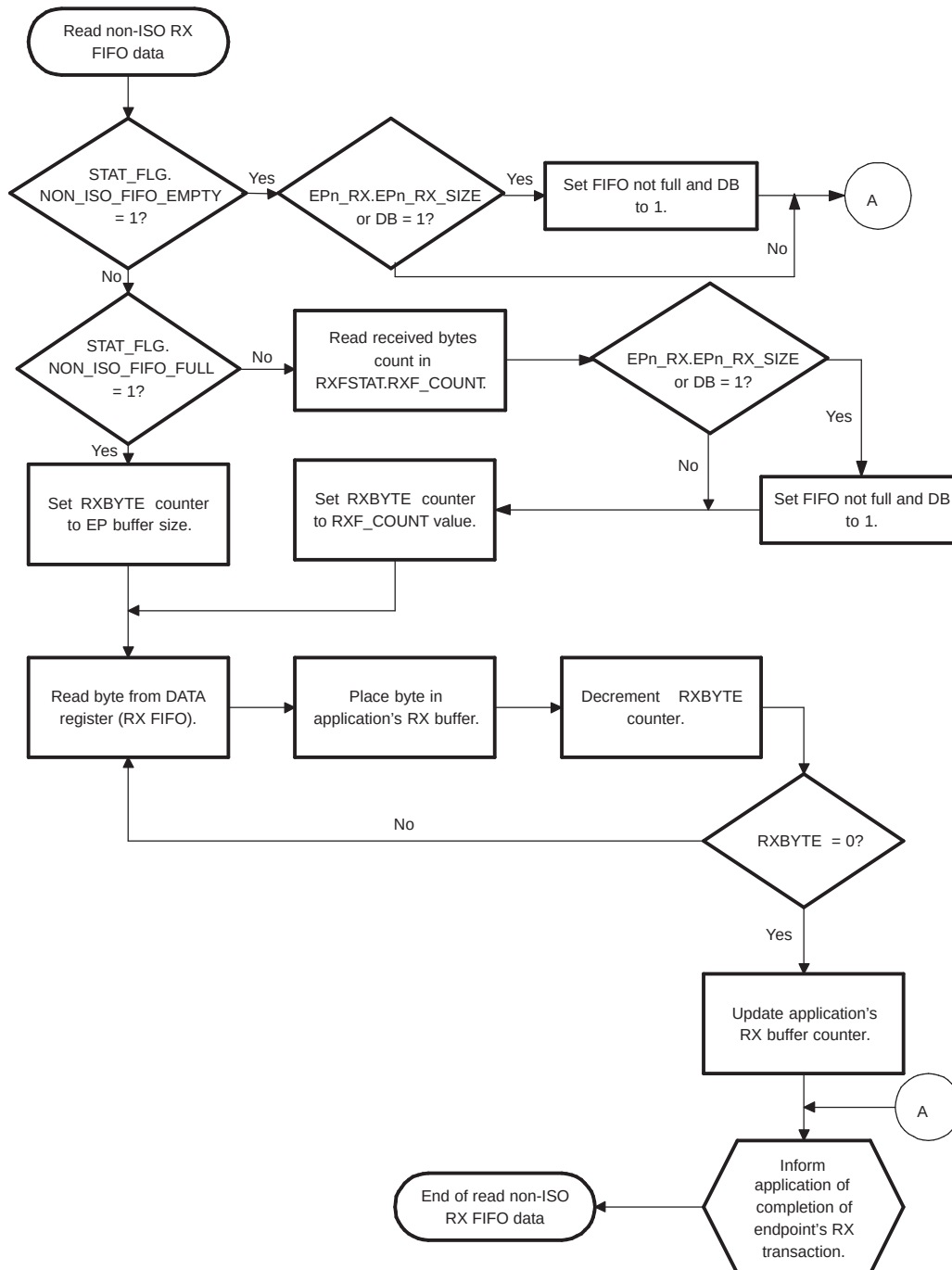
Figure 32-77. Non-Isochronous Non-Control Endpoint Receive Interrupt Handler


Figure 32-78. Read Non-Isochronous RX FIFO Data Flowchart


32.3.24 Non-ISO, Non-Control IN Endpoint Transmit Interrupt Handler

Figure 32-79 shows the operations necessary to handle non-ISO, non-control IN endpoint-specific transmit interrupts. This flowchart shows two TX transaction handshaking interrupts. There is a third interrupt handshaking possibility when NAK interrupts are enabled, which is not depicted here. Depending on the application-specific actions needed for various endpoints in the real system, it is possible to use one routine that is common to all of the non-ISO, non-control transmit endpoints, where the only differences are in the EP_NUM register value set and in the routine selection of the application TX buffer (see Figure 32-80).

This flowchart does not attempt to document control endpoint 0 transmit interrupts, which are discussed separately because of the more complex three-stage transfer mechanism used for control reads.

Figure 32-79. Non-Isochronous Non-Control Endpoint Transmit Interrupt Handler

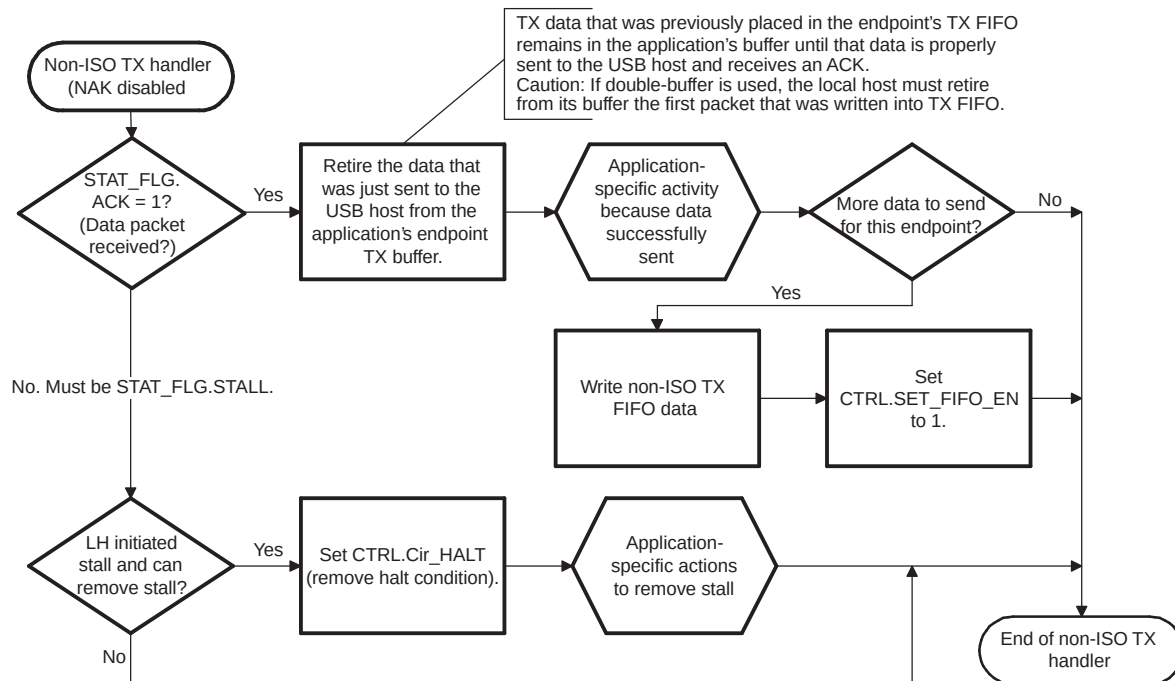
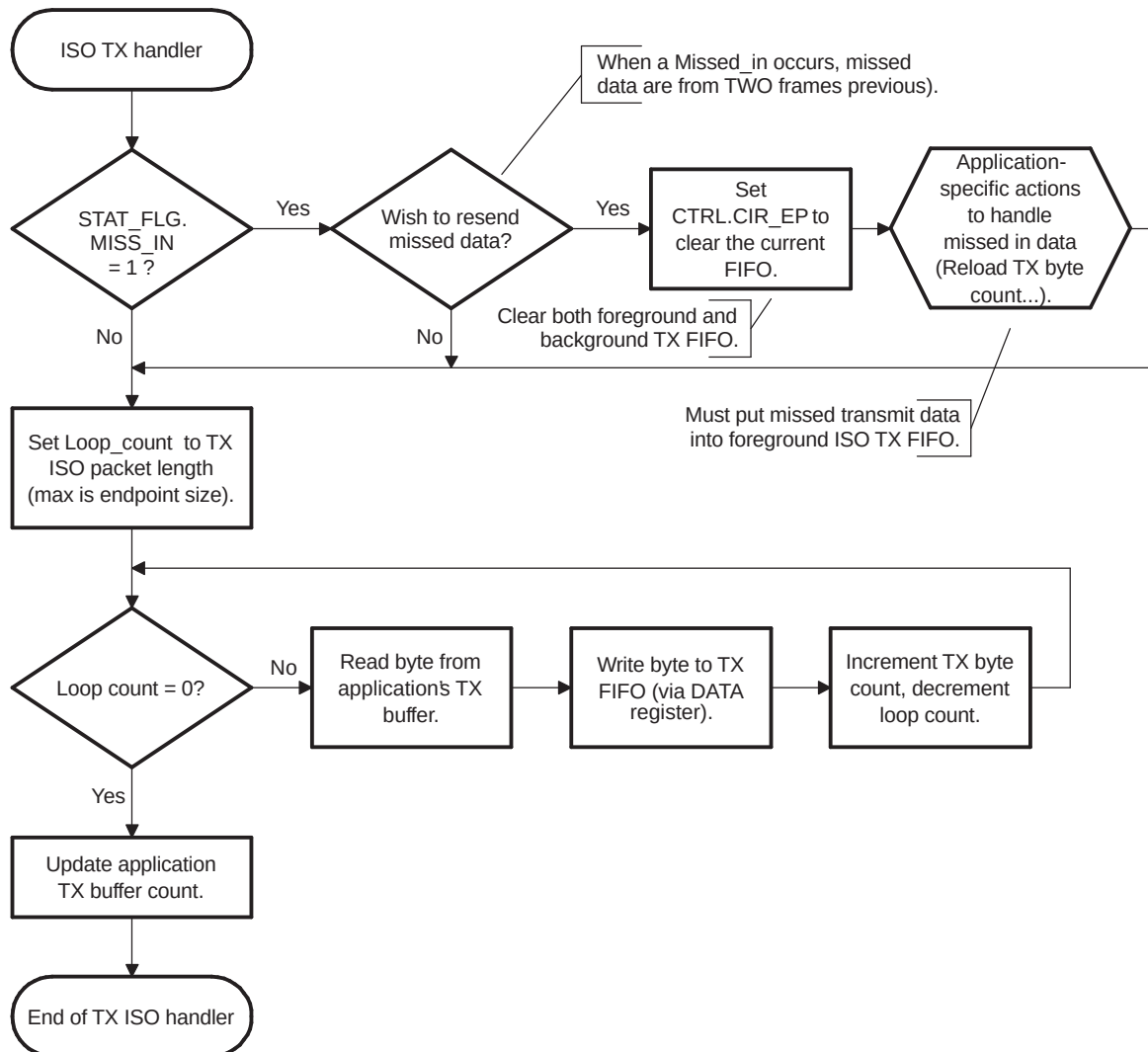


Figure 32-80. Write Non-Isochronous TX FIFO Data Flowchart



32.3.25 SOF Interrupt Handler

SOF interrupts to the CPU (also known as isochronous interrupts on VIM channel # 68 by default) occur once per USB frame. The CPU must handle data traffic for the isochronous endpoints at each SOF interrupt. In addition, the SOF ISR can handle any application-specific activity related to the implicit timing of the SOF interrupt. [Figure 32-81](#) shows the SOF ISR flowchart. The read ISO RX FIFO data and write ISO TX FIFO data procedures are shown in [Figure 32-82](#) and [Figure 32-83](#), respectively.

Figure 32-81. SOF Interrupt Handler Flowchart

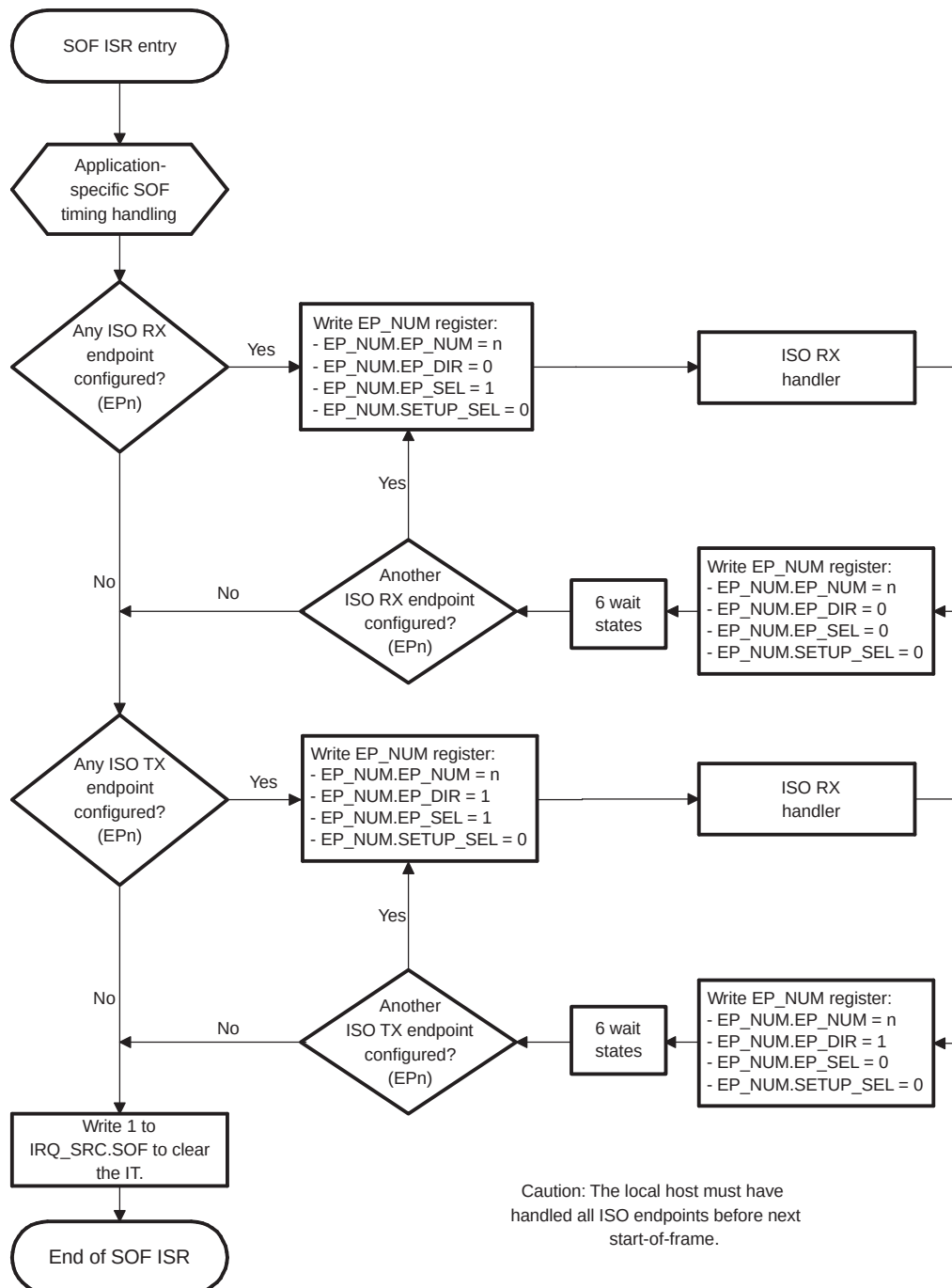


Figure 32-82. Read Isochronous RX FIFO Data Flowchart

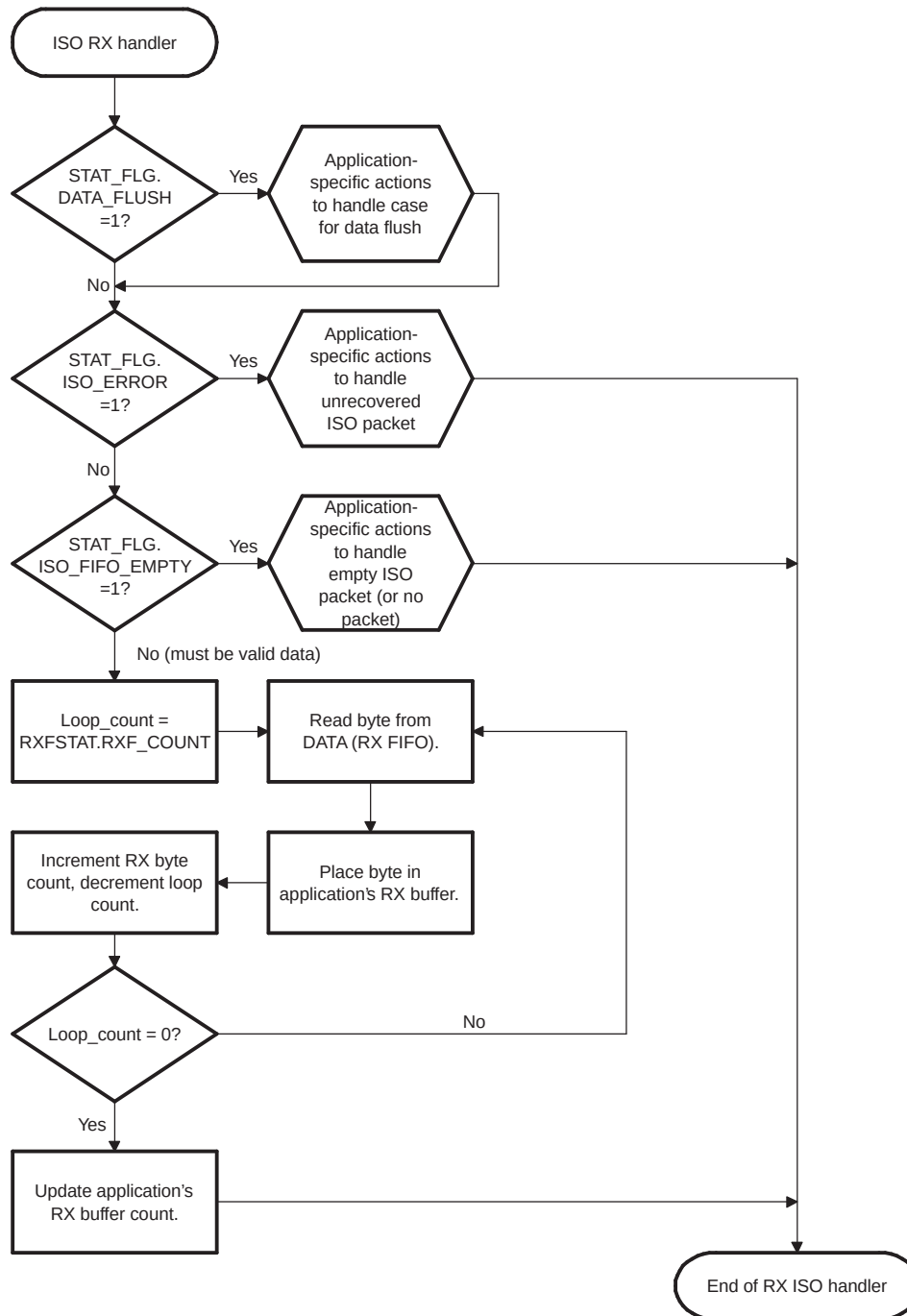
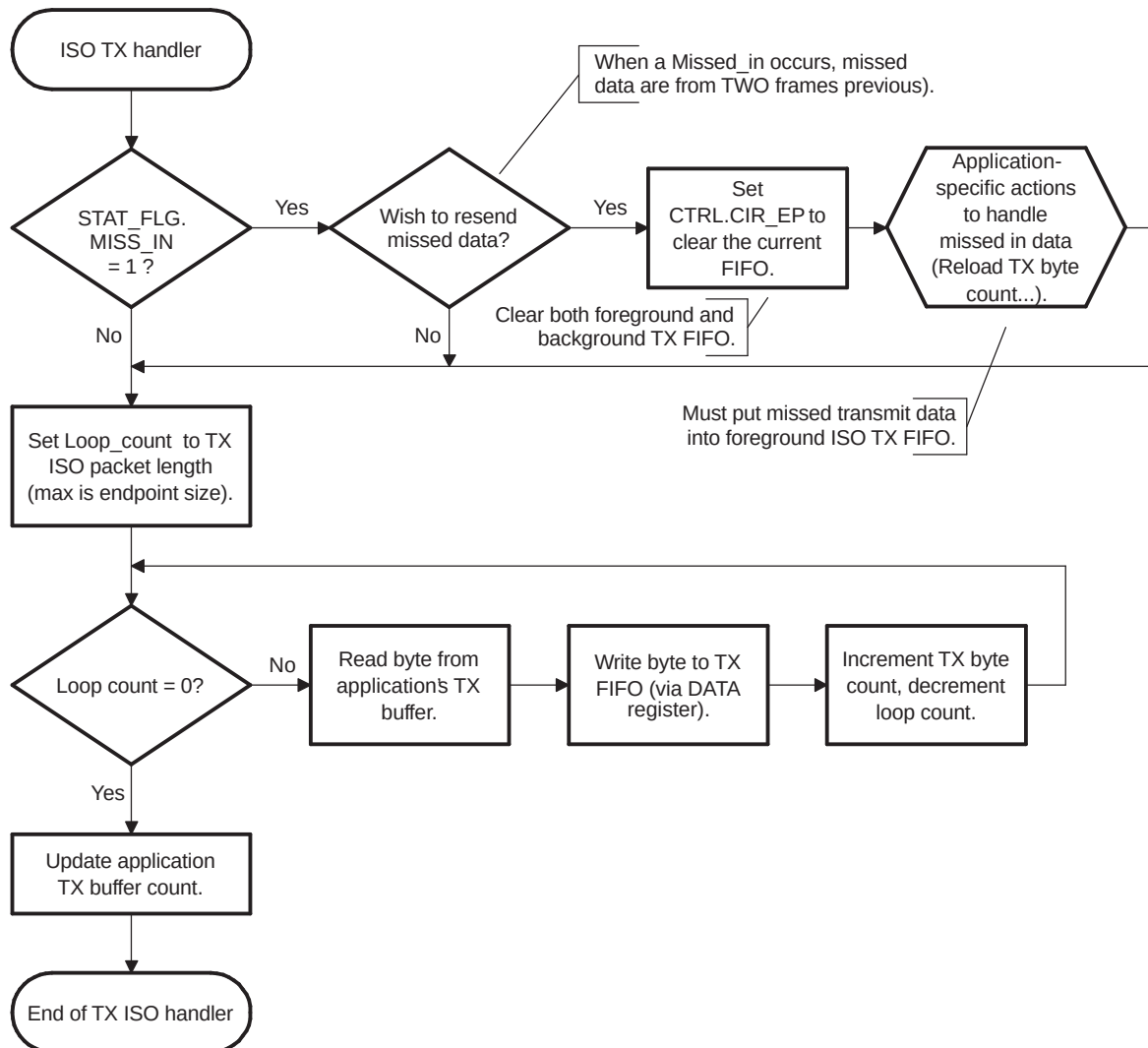


Figure 32-83. Write Isochronous TX FIFO Data Flowchart


32.3.26 Summary of USB Device Controller Interrupts

Table 32-56 lists the interrupt types by endpoint types.

Table 32-56. USB Device Controller Interrupt Type by Endpoint Type

Interrupt Type	General USB IRQs (VIM channel # 69 by default)				EP-Specific IRQs (Non-ISO Interrupt on VIM channel # 70 by default)		SOF ISO Interrupt on VIM channel # 68 by default)
	Setup (EP0)	Control (EP0) Out	Control (EP0) In	Other	Bulk or InterruptOut	Bulk or InterruptIn	(Isochronous) SOF
Transaction ACKed		X	X		X	X	
Transaction NAKed (if enabled)		X	X		X	X	
Transaction Stalled		X	X		X	X	
Setup	X						
SOF							X
Device state changed				X			
RX DMA EOT (non_ISO)				X			
RX DMA trans count (non_ISO)				X			
TX DMA done (non_ISO)				X			

32.3.26.1 USB Device Controller Clock Control

The device global clock module (GCM) provides a single 48-MHz clock to the USB device controller and USB host controller. This is the VCLKA3 clock domain. This clock domain can be stopped by software to reduce USB host and USB device controller power consumption when USB functionality is not needed. The GCM controls the 48-MHz clock to the USB device controller via the following memory-mapped registers:

- SYS.CDDIS
- SYS.CDDISSET
- SYS.CDDISCLR

32.3.26.2 USB Device Controller Hardware Reset

Reset of the USB device controller is provided by the SYS module. The system reset signal is used to reset the USB host controller and the USB device controller. When held in reset, the USB device controller does not recognize any USB activity, and its registers have no effect on USB functionality.

32.3.27 DMA Operation

The USB device controller provides support for six DMA channels. Three receive DMA channels are reserved to OUT transfers (ISO or non-ISO) and three transmit DMA channels are reserved to IN transfers (ISO or non-ISO). It is not possible to operate DMA transactions on control EP0.

NOTE: The CPU must not access an endpoint used in a DMA transfer through the EP_NUM, CTRL, and STAT_FLG registers (in DMA, this remark applies after the CPU has set the CTRL.SET_FIFO_EN bit to enable the RX DMA transfer). In particular, the CPU must not set the halt feature while the endpoint is selected in the RXDMA_CFG register.

To use the DMA channels properly, you must set the DMA configuration during the address state interrupt (DS_CHANGE).

The parameters used for DMA transactions (FIFO size, ISO endpoint, double-buffering, and pointers) are those defined for the associated endpoint.

32.3.27.1 Receive DMA Channels Overview

Receive DMA channels are programmed via the three RXDMA control registers. Each channel is assigned to a given endpoint number by assigning a non-zero value in RXDMA_CFG.RXDMA_n_EP fields (a 0 value means the DMA channel is deselected). Received OUT data must be read when an RX DMA request is active, through the register DATA_DMA. The RX FIFO accessed is that of the endpoint for which the DMA request is active (only one RX DMA request is active at a time).

The USB device controller receive DMA channels 0, 1, and 2 are connected to the device System DMA controller requests # 15, 19, and 29, respectively.

32.3.27.2 Non-Isochronous OUT (USB HOST -> CPU) DMA Transactions

During non-ISO transfers to a DMA operated OUT endpoint, a request to the CPU DMA controller is generated when data have been placed into endpoint FIFO and must be read. ACK and NAK interrupts are always disabled automatically by the core for DMA operated endpoints.

There are two dedicated maskable interrupts per DMA channel to control non-ISO OUT transfers.

32.3.27.3 End of Transfer Interrupt (IRQ_SRC.RX_n_EOT)

This interrupt signals that the core has detected an end-of-transfer (EOT). EOT occurs in the two following cases:

- When the last valid transaction to the endpoint is either an empty packet (ACK and buffer empty) or a packet whose size is less than the physical endpoint buffer size (ACK and buffer not full)
- When the number of received transactions has reached the programmed value in the RXDMA_n.RX_n_TC field, if the RXDMA_n.RX_n_STOP bit has been set by the CPU

After an end of transfer interrupt, the CPU must set CTRL.SET_FIFO_EN for the endpoint to reenable the channel.

The CPU must not initiate a new RX DMA transfer until it receives an end-of-transfer interrupt.

32.3.27.4 Transaction Count Interrupt (IRQ_SRC.RX_n_CNT)

The intent of this interrupt is watermark control. It can be used by the CPU to monitor the file size of incoming transfers and take appropriate actions if, for instance, the file being received exceeds an expected size.

A transaction count interrupt does not disable the ongoing DMA transfer.

A transaction count interrupt occurs each time the number of received transactions (and not bytes) has reached the programmed value in the receive transaction counter for the DMA channel. One transaction has a size equal to the buffer size of the selected non-ISO endpoint. RX_n_COUNT interrupt is asserted even if RXDMA_n.RX_n_STOP has been set; in that case, both RX_n_COUNT and RX_n_EOT interrupts are asserted (see [Figure 32-84](#) through [Figure 32-87](#)).

The transaction count watermark is programmed in the RXDMA_n register.

Figure 32-84. Non-ISO RX DMA Transaction Example (RX_TC=2)

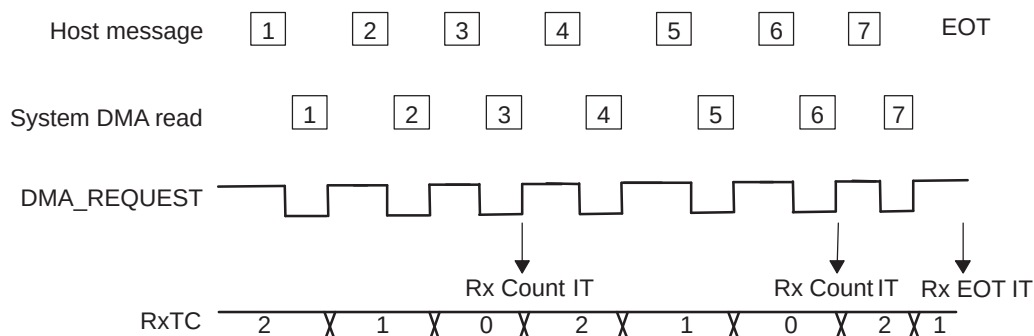


Figure 32-85. Non-ISO RX DMA Start Routine

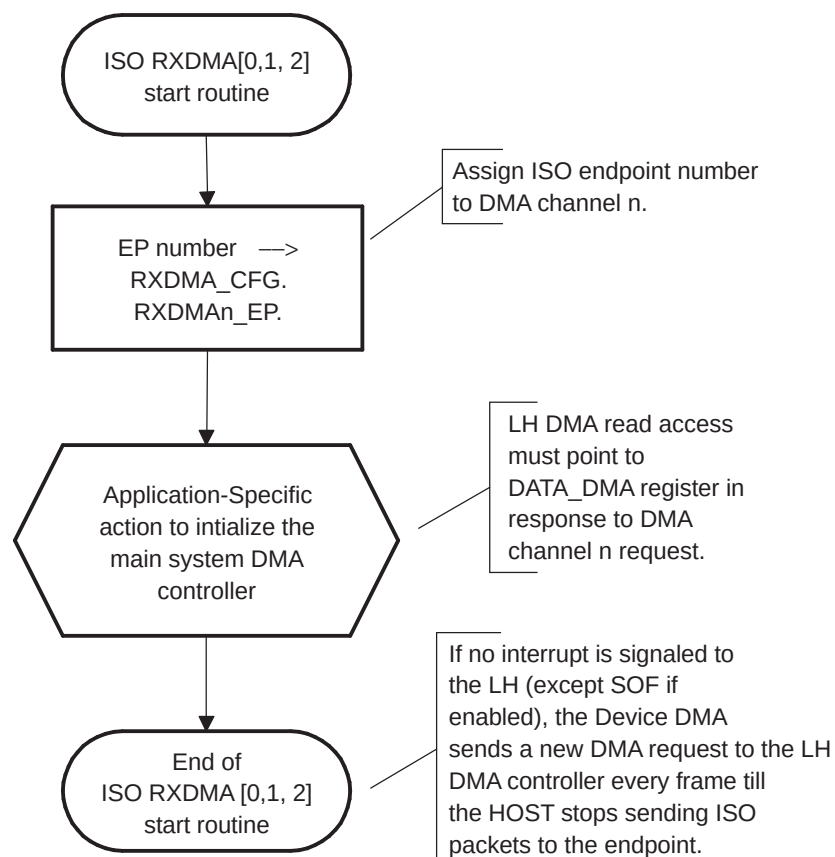


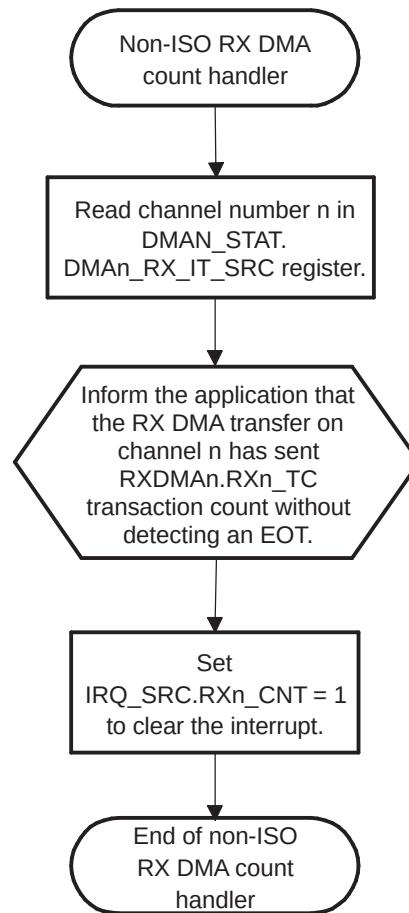
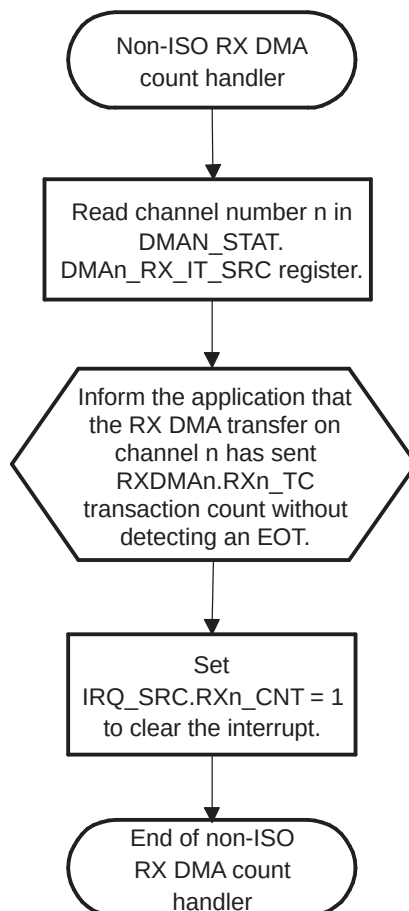
Figure 32-86. Non-ISO RX DMA EOT Interrupt Handler


Figure 32-87. Non-ISO RX DMA Transactions Count Interrupt Handler



32.3.27.5 Isochronous OUT (USB HOST -> CPU) DMA Transactions

During ISO transfers to a DMA operated OUT endpoint, a request to the CPU DMA controller is generated every 1-ms frame when an isochronous data packet is received with no error. There is no interrupt associated with DMA transfer to ISO OUT endpoints (see [Figure 32-88](#) and [Figure 32-89](#)).

Figure 32-88. ISO RX DMA Transaction

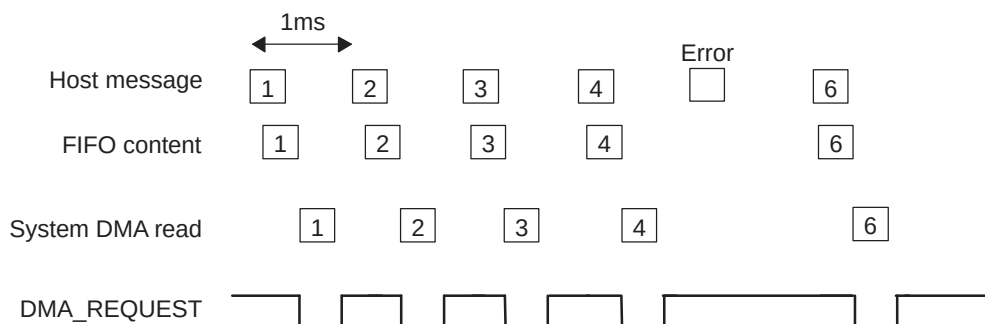
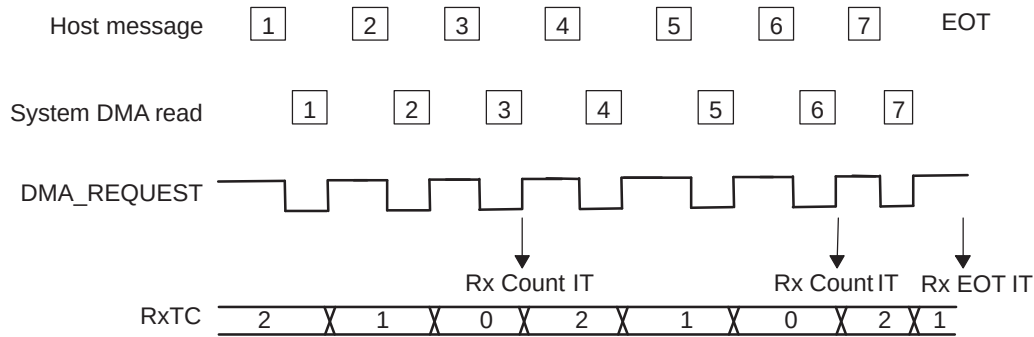


Figure 32-89. ISO RX DMA Start Routine


32.3.27.6 Transmit DMA Channels Overview

Transmit DMA channels are programmed via the three TXDMA control registers. Each channel can be assigned to a given endpoint number by assigning a non-zero value in TXDMA_CFG.TXDMA_n_EP (a 0 value means the DMA channel is deselected). The other three control registers (TXDMA0, TXDMA1, and TXDMA2) operate in a different manner for ISO and non-ISO endpoints. Transmitted data must be written into the DATA_DMA when a TX DMA request is active. They are written into the TX FIFO of the endpoint associated with active request (only one TX DMA request active at a given time).

The USB device controller transmit DMA channels 0, 1, and 2 are connected to the device system DMA controller requests # 14, 20, and 28, respectively.

32.3.27.7 Non-Isochronous IN (CPU -> USB HOST) DMA Transactions

Non-ISO (bulk) TX DMA file transfers are virtually unlimited in size. The flowcharts depicted in [Figure 32-90](#) and [Figure 32-91](#) show how to handle small, medium, or large file transfers.

TXDMA0, TXDMA1, and TXDMA2 registers operate for non-ISO endpoints in the following manner. The transfer size counter (TXDMA_n.TXN_TSC) corresponds to either the number of bytes to transmit (EOT bit set) or the number of buffers to transmit (EOT bit cleared). The buffer size corresponds to the programmed size of the TX endpoint.

A request to the CPU main DMA controller is generated when the endpoint buffer is empty initially after that the START bit is set and then each time there is space free in TX FIFO for other TX packet to be written, until TXDMA_n.TXN_TSC counts down to zero. The request is removed when the buffer is full or when there are no more bytes of data to be sent.

A DMA transmit transfer done interrupt is signaled to the CPU after the last IN transaction completes successfully. This is after START bit was set and after TXDMA_n.TXN_TSC equals 0 for the selected DMA channel.

The CPU must not initiate a new TX DMA transfer until it receives a TX_DONE interrupt.

A small file transfer less than 1024 bytes can be achieved in a single pass DMA signaled by a single interrupt completion. A file size equal to or greater than 1024 bytes needs two or more DMA passes, signaled by an interrupt completion after each pass.

[Figure 32-90](#) shows the necessary steps to prepare and permit a TX DMA transfer of any size. It also effectively starts the initial DMA transfer. The completion of this DMA task is signaled to the CPU via a DONE interrupt, whose handler is shown in [Figure 32-91](#). The start routine and the associated interrupt handler are tightly coupled.

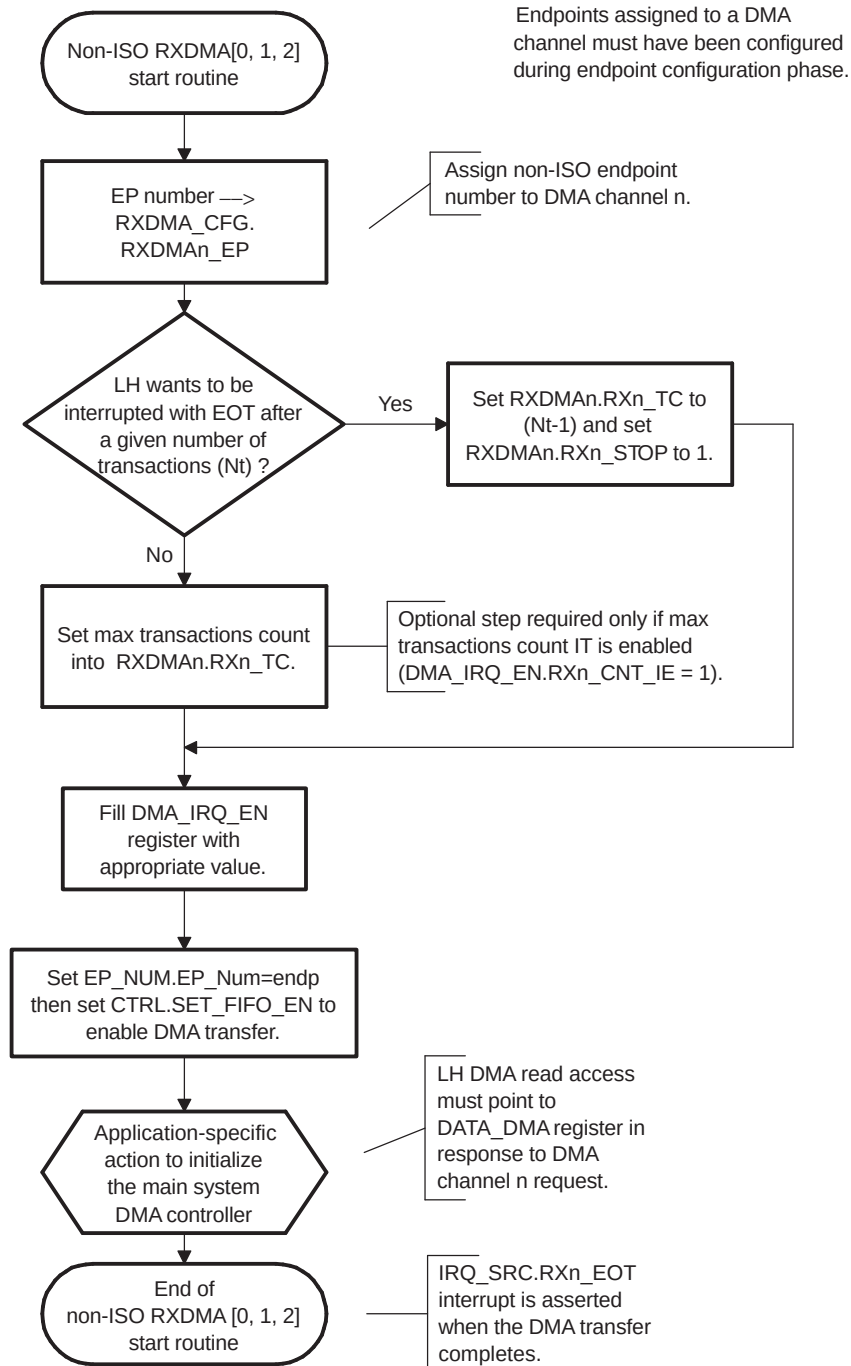
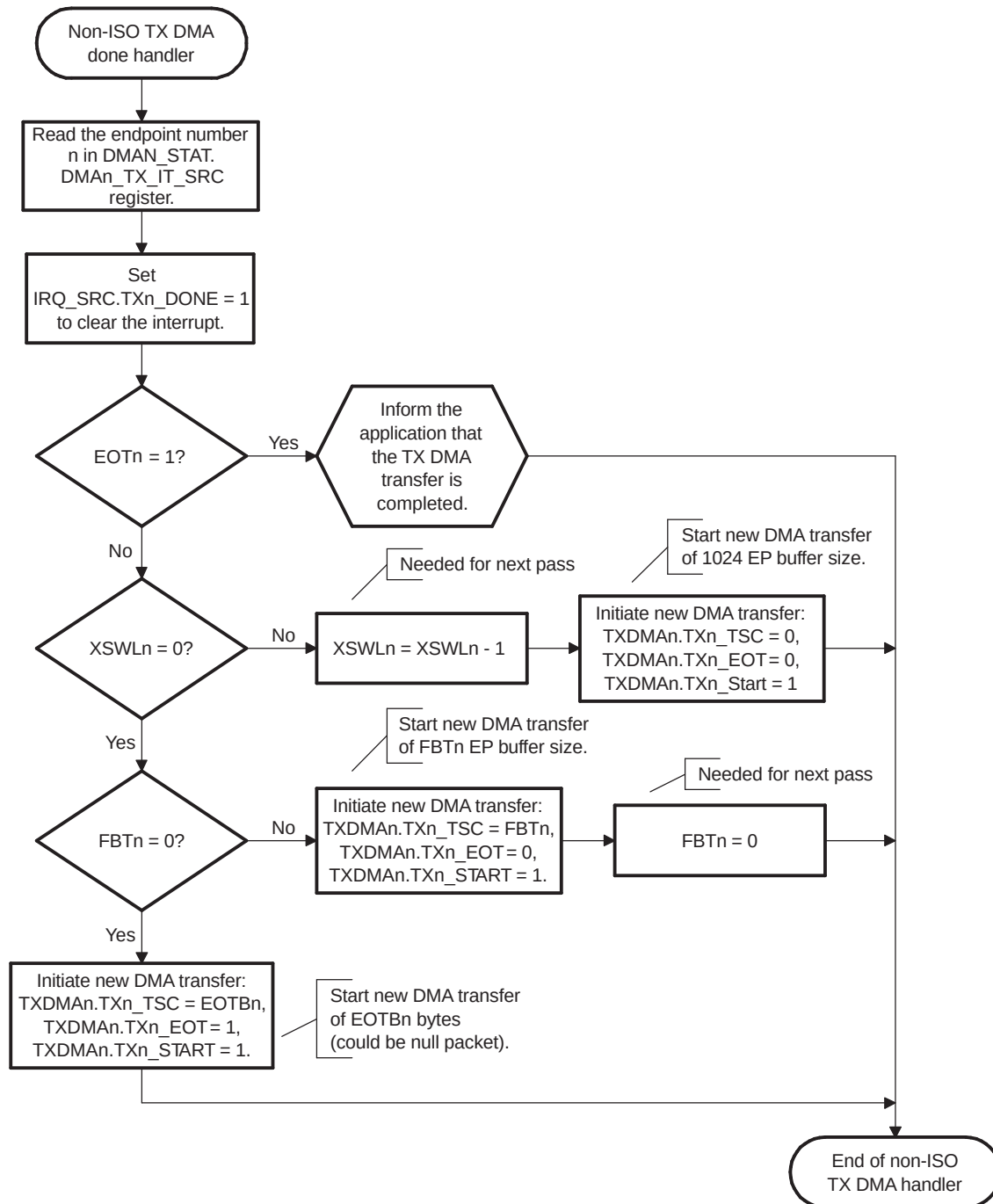
Figure 32-90. Non-ISO TX DMA Start Routine


Figure 32-91. Non-ISO TX DMA Done Interrupt Handler


Example 32-1. 100603 Bytes to Transfer via 32 Bytes IN Bulk Endpoint

This gives XSWL=0x3, FBT=0x47, EOTB=0x1b,

which means five passes of DMA transfer, signaled by 5 TXn_DONE interrupts, are required:

1. EOT=0, FBT=0, loop=3 32768 bytes transferred (1024 x 32 bytes)
2. EOT=0, FBT=0, loop=2 32768 bytes
3. EOT=0, FBT=0, loop=1 32768 bytes
4. EOT=0, FBT=0x47, loop=0 2272 bytes (71 x 32 bytes)
5. EOT=1, FBT=0x1B, loop=0 27 bytes

32.3.27.8 Isochronous IN (USB HOST -> CPU) DMA Transactions

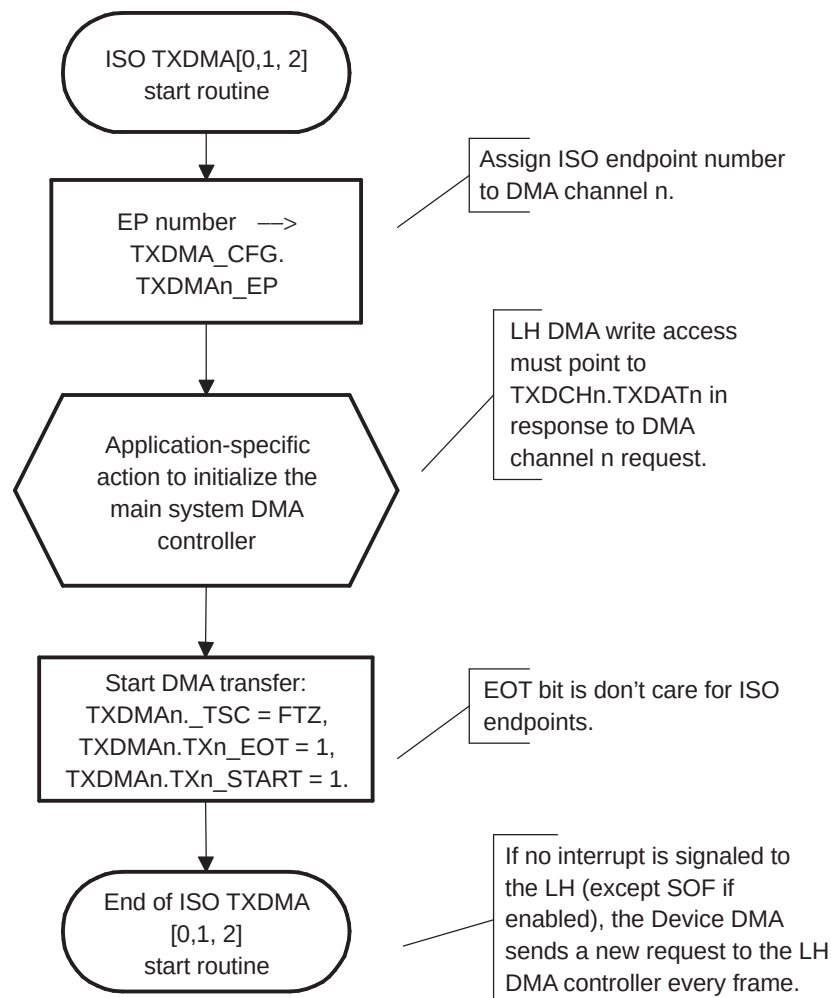
For ISO endpoints, the transfer size counter (TXDMA_n.TX_n_TSC) corresponds to the number of bytes to transmit. The programmed size must not exceed the programmed buffer size of the endpoint. Otherwise, the result is unpredictable (see [Figure 32-92](#)).

A request to the CPU main DMA controller is generated when the endpoint buffer is empty initially after the START bit is set, and then after each SOF (every 1 ms). The request is removed when the number of bytes written in the buffer matches the TXDMA_n.TX_n_TSC value.

During ISO transfers to a DMA operated IN endpoint, a request to the CPU system DMA controller is generated every 1-ms frame when an isochronous data packet is received with no error. There is no special interrupt associated with the DMA transfer.

No interrupt is signaled to the CPU during DMA operation to ISO IN endpoints.

Figure 32-92. ISO TX DMA Start Routine



32.3.27.9 Important Note on DMA Requests

For each direction, only one DMA request can be active at any time. A request must then be serviced to allow the next pending request on the same direction to be asserted. In particular, a TX DMA request is asserted at each start-of-frame if a TX DMA channel is configured for an isochronous endpoint; this request must be serviced imperatively.

32.3.27.10 Note on DMA Channels Deconfiguration

It is recommended that the CPU wait for an EOT (RX) or a DONE (TX) interrupt before disabling the channel by writing a value 0 in the TX/RXDMA_CFG register. However, if needed by the application, the CPU can unselect the endpoint number in the TX/RXDMA_CFG register during a DMA transfer. The resulting behavior is:

- For RX transfer:
 - If RX DMA request is active for the endpoint when the endpoint is unselected, deconfiguration is effective only at the end of the RX DMA request (that is, after all the DMA data have been read). When double-buffering is used, the deconfiguration is effective after both buffers have been read (if both buffers were not empty at unselection). An EOT interrupt is asserted if an end-of-transfer is detected.
 - If the RX DMA request is not active when unselection occurs, the effect is immediate.
- For TX transfer:
 - If the request is active when the endpoint is unselected, deconfiguration is effective after the TX DMA request has been handled and the TX data have been sent through an IN transaction (both buffers in case of double-buffering with both buffers full). No TX_DONE interrupt is asserted even if TXDMA_n.TSC bit value is 0 after the transaction.
 - If the TX DMA request is inactive when the endpoint is unselected, deconfiguration is effective when all data available in TX buffer(s) have been sent through IN transaction(s). If the TXDMA_n.TSC value is 0 at this point, no TX_DONE interrupt is asserted. If TX_DONE interrupt had already been asserted when the endpoint is deselected, configuration is effective only after the TX_DONE interrupt handling.

TX/RXDMA_CFG.TX/RXDMA_n_EP reflects the endpoint value until deconfiguration is effective. The CPU must read this register to know if the channel has been disabled yet or not. It must wait until the read value is 0 before performing other actions to the endpoint. After effective deconfiguration, all transactions to the endpoint generate an endpoint-specific interrupt (if non-transparent).

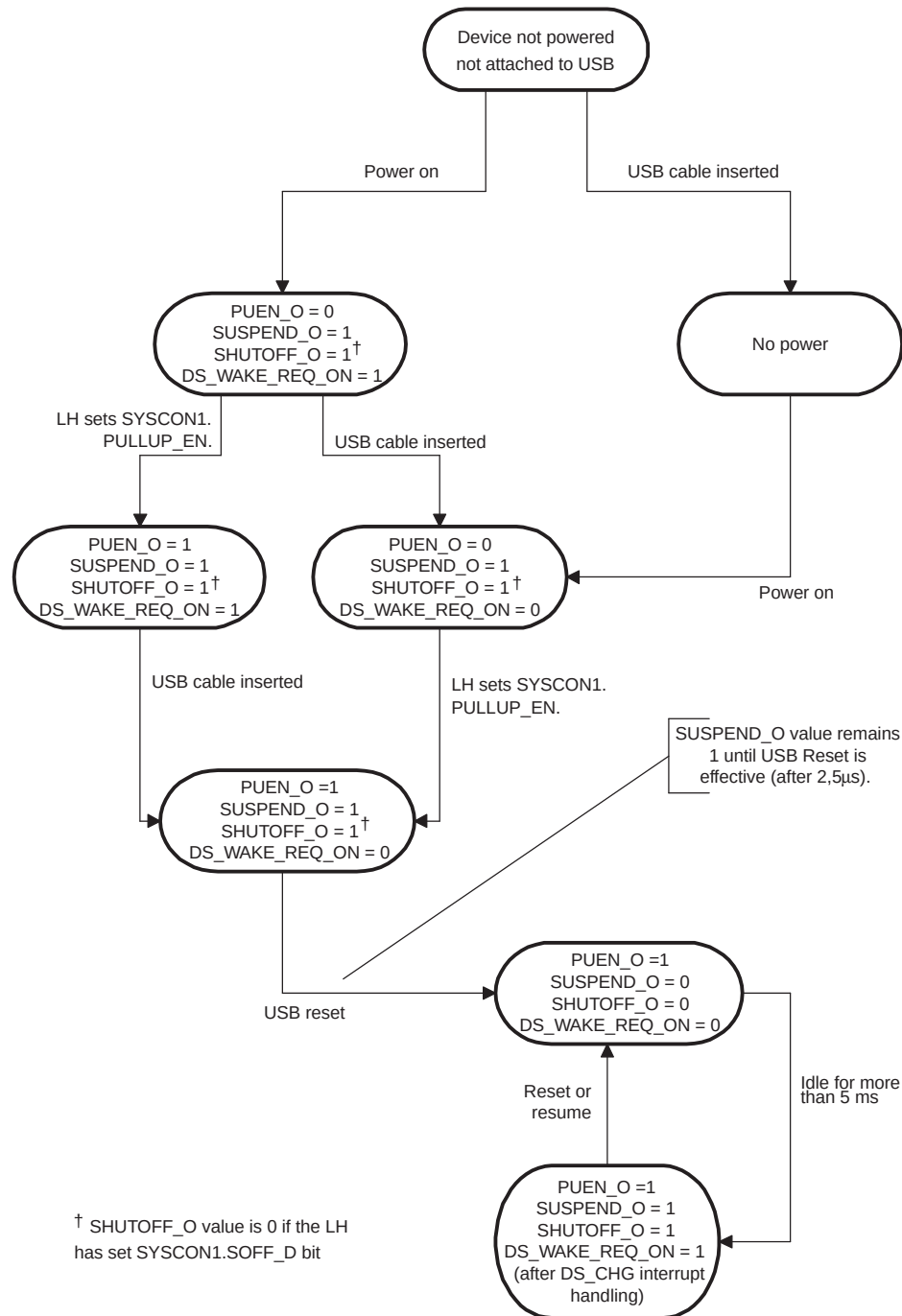
If the selected endpoint is of isochronous type, deconfiguration is effective after the TX/RX request has been serviced, and the subsequent isochronous transactions are handled at SOF interrupt through the endpoint registers (EP_NUM and STAT_FLG).

32.3.28 Power Management

Figure 32-93 shows the values assigned to the USB device controller signals concerned with power management, in the function of the device state. These signals are:

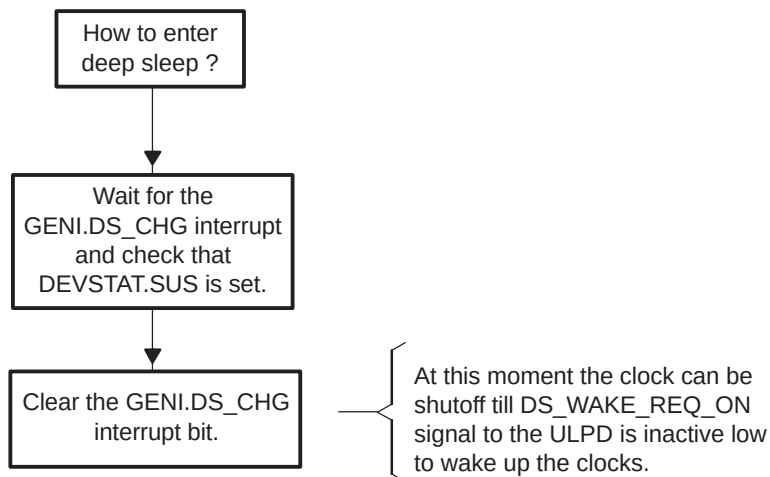
- PUEN_O: Pullup enable signal, always reflecting the SYSCON1.PULLUP_EN register bit
- SHUTOFF_O: Power circuitry shutoff signal, controlled by the core and the SYSCON1.SOFF_DIS bit
- DS_WAKE_REQ_ON: Deep-sleep wake request, asserted low when the interface clock is needed
- SUSPEND_O: Suspend signal, asserted high when the device is in suspend mode

From a software point of view, Figure 32-94 shows the reaction. This flowchart does not need to be implemented; it only reflects the way the module can enter deep sleep.

Figure 32-93. Power Management Signal Values


A SHUTOFF_O value is 0 is the LH has set SYSCON1.SOFF_D bit.

Figure 32-94. Power Management Flowchart



32.4 USB Connectivity

This section includes:

- Transceiver interface
- Selecting and configuring USB connectivity
- Transceiver signaling
- Host controller connectivity with USB transceivers
- USB function controller connectivity with USB transceivers
- USB hardware considerations

32.4.1 Transceiver Interface

The device provides three USB ports, two of which may be used simultaneously. In a USB system, a USB transceiver is needed for each USB port. It converts between signaling appropriate for the USB controllers and signaling appropriate for the USB cable. This device does not provide an integrated USB transceiver.

Several different types of external transceiver signaling are supported. Signaling between the USB controller and the external USB transceiver for monitoring and controlling the differential USB signal is done via a 6-wire signaling interface, with two or more additional control signals provided either by additional signals or via an I2C link.

32.4.2 Selecting and Configuring USB Connectivity

The process of selecting desired USB connectivity and configuring the device for that connectivity can be done using the steps listed below.

32.4.2.1 Choose the Desired USB Functionality

Choose the desired USB functionality. The USB module on this microcontroller supports the following configurations:

- 1 USB Host Port + 1 USB Device Port, OR
- 2 USB Host Ports

Interface signals for the USB host port #2 are multiplexed with interface signals for the USB device port. If the application requires the use of the USB device port, then the USB host port #2 must first be powered down.

32.4.2.2 Select How USB Functionality Is Multiplexed to Pins

Refer to [Table 32-1](#) for information on multiplexing and control of USB host ports and USB device port interface signals at the top level.

32.4.3 Transceiver Signaling

The USB module on this device uses a unidirectional (6-wire) signaling type for the transceiver. The signaling is shown in [Table 32-57](#).

Table 32-57. Signaling Between USB Controller and Unidirectional USB Transceiver Using DAT/SE0 Signaling

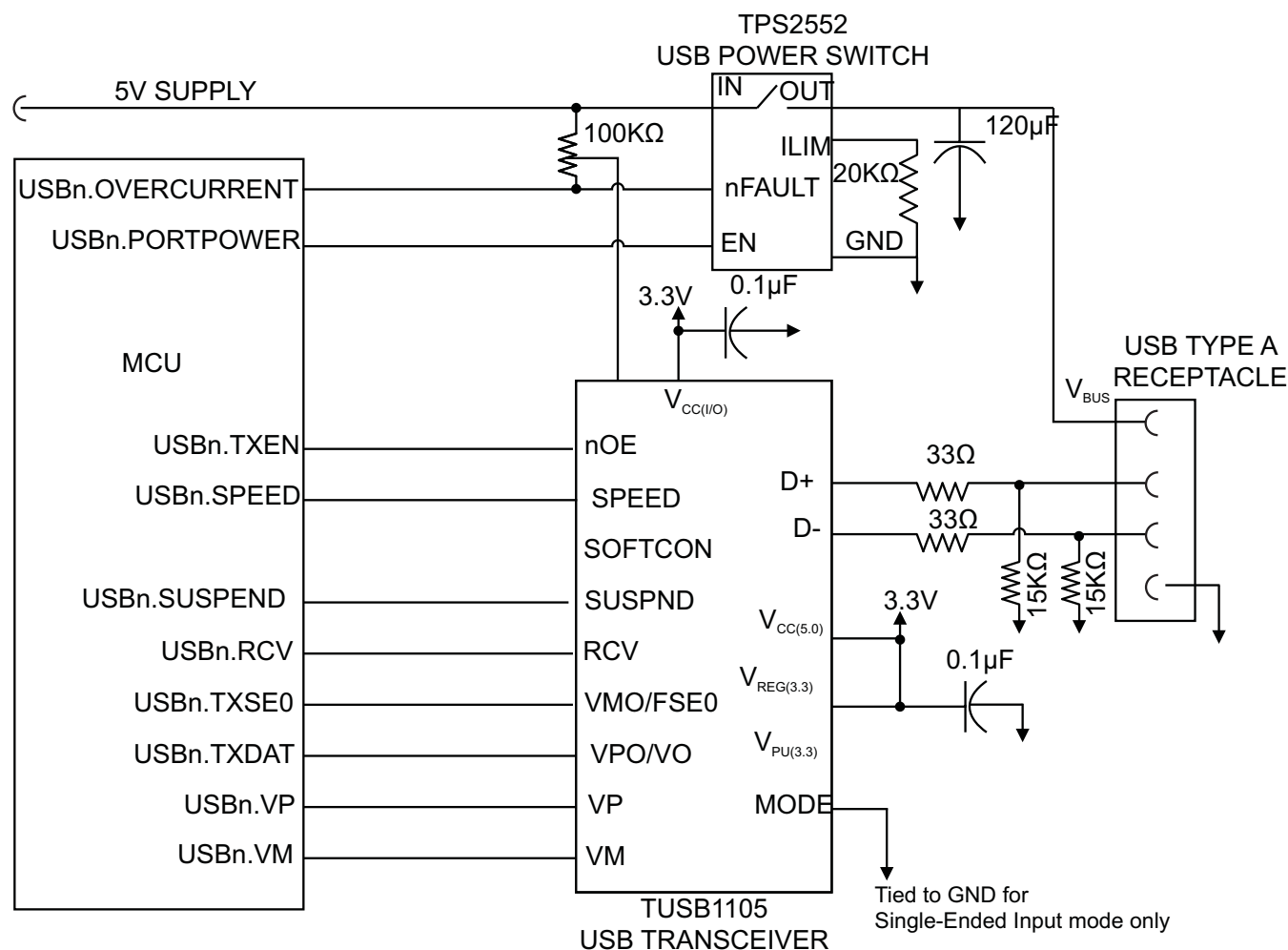
Logical Signal Name(s)	Pin Direction	Transceiver Pin Direction	Description				
GZO	Output	Input	When low, USB transceiver drives D+ and D-.				
DAT and SE0	Output	Input	Controls the values output by the USB transceiver on D+ and D- when GZO is low. Ignored when GZO is high.				
			GZO	DAT	SE0	D+	D-
			0	0	0	0	1
				1	0	1	0
				X	1	0	0
			1	X	X	Undriven	Undriven
RCV	Input	Output	Output from transceiver differential receiver.				
			D+		D-		RCV
			0		0		X
			0		1		0
			1		0		1
			1		1		X
VP	Input	Output	Output from transceiver single-ended D+ signal receiver .				
			D+				VP
			0				0
			1				1
VM	Input	Output	Output from transceiver single-ended D- signal receiver.				
			D-				VM
			0				0
			1				1
SPEED	Output	Input	Determines transceiver D+, D- output characteristics. 0 = Low speed. 1 = Full speed. Transceivers with I ² C interfaces can implement this functionality as a control bit rather than a pin.				
SUSPEND	Output	Input	Controls transceiver power down modes. 0 = Active mode. 1 = Low-power mode. Transceivers with I ² C interfaces can implement this functionality as a control bit rather than a pin.				

This device does not support unidirectional transceivers that implement VPO/VMO signaling.

32.4.4 Host Controller Connectivity With USB Transceivers

To provide a robust USB solution, a system that provides a USB host controller must implement certain features. These features include a type-A receptacle, power on the VBUS signal (can be switched or unswitched power), transient suppression, pull-down resistors, and USB-compatible downstream port transceiver.

Figure 32-95. Pin Group 0 USB Host Using 6-Wire USB Transceiver



Proper initialization of the device to support this mode of operation requires proper setting of the top-level pin multiplexing (see [Section 32.4.2.2](#)).

32.4.5 USB Function Controller Connectivity With USB Transceivers

To provide a robust USB solution, a system that provides a USB device controller port must implement certain features. These features include:

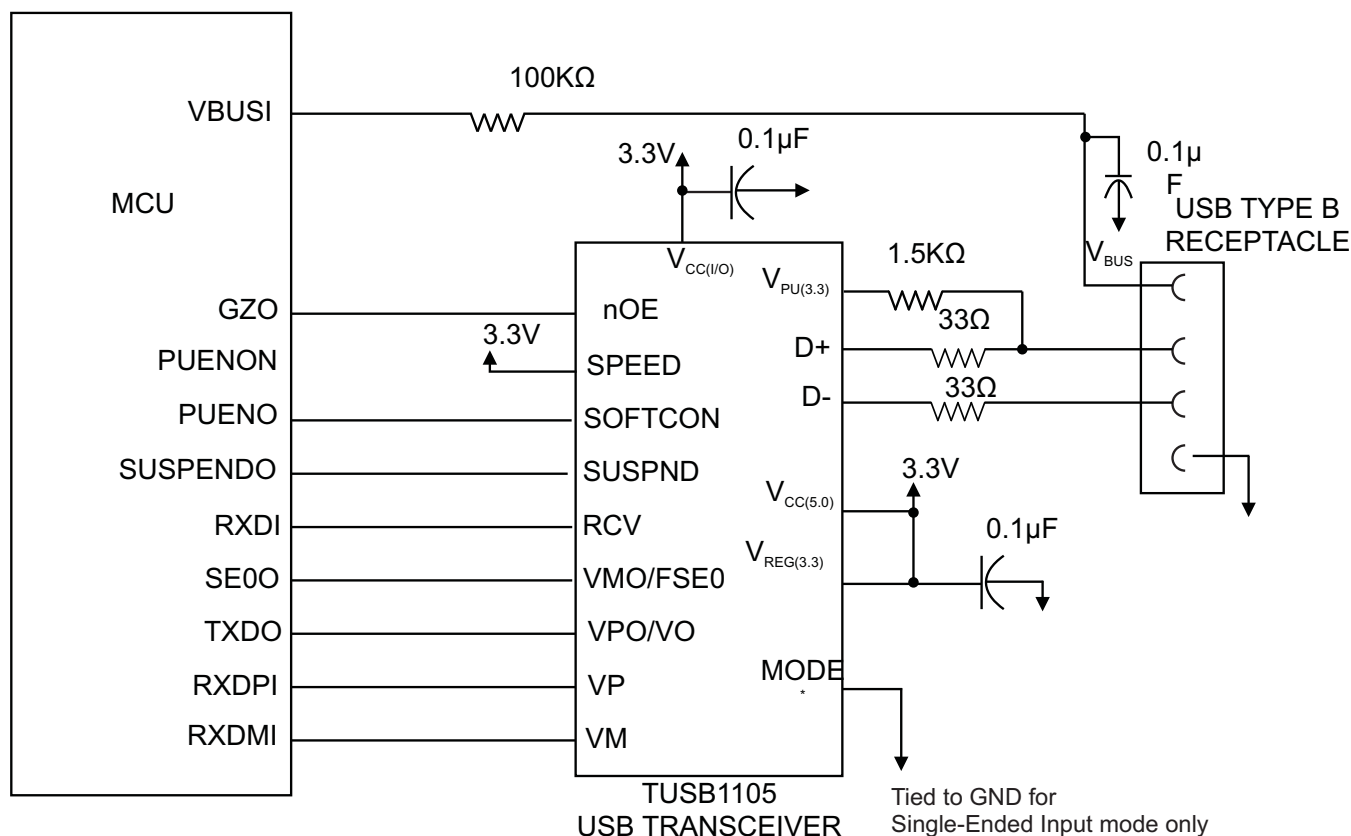
- A USB type-B or mini-B receptacle
- VBUS power detection
- Transient suppression
- A controllable pullup resistor to the D+ or D- line
- USB-compatible upstream port transceiver

Optional weak pulldown resistors can be used to hold the D+ and D- signals at voltages below the USB transceiver VIL level when there is no USB host connected to the USB Type B connector. By keeping D+ and D- voltages below VIL, the USB transceiver IDDQ can be reduced. Choice of value for the pulldown resistors must be made carefully so that the circuit meets the requirements of the *USB Specification*.

The USB function controller only supports implementation as a full-speed USB device. As such, the pullup resistor must be connected to the D+ signal to indicate implementation of a full-speed USB device.

This device supports a 6-wire connection to the USB transceiver.

Figure 32-96. USB Device Connections on USB Pin Group 0 Using 6-Wire Transceiver



Proper initialization of the device to support this mode of operation requires proper setting of the top-level pin multiplexing (see [Section 32.4.2.2](#)).

32.4.6 USB Hardware Considerations

32.4.6.1 VBUS Power Switching for USB Type A Host Receptacles

The USB specification places several VBUS requirements on USB hosts, including current capability, droop, and other important characteristics. Circuits that meet the USB specification requirements can be implemented using Texas Instruments devices such as the TPS2552 and TPS2553 power distribution switch devices. Further information, including data sheets and application notes, can be found at the Texas Instruments web site.

32.4.6.2 Transient Suppression for USB Connectors

It is important to provide transient suppression for USB connectors. Electrostatic discharge that occurs when a user connects or disconnects a USB cable can have a dramatic effect on a system if not suppressed. Texas Instruments offers several devices for transient suppression on USB connections, such as the SN65220, SN65240, and SN75240 universal serial bus port transient suppressor devices. Further information, including data sheets and application notes, can be found at the Texas Instruments web site.

32.4.6.3 VBUS Monitoring for USB Function Controller

A USB function controller must be capable of monitoring the VBUS voltage provided by the upstream USB host controller. The device provides the input pin USB_VBUS, which is provided to the USB function controller. This input is a CMOS input that is not rated for the full VBUS range defined by the USB specification. An external signal level shifter is required to convert the VBUS signal range to a range suitable for the USB_VBUS pin.

32.4.6.4 USB D+ Pullup Enable for USB Function Controller

The USB_PUEN pin is driven high when the pullup is active and is driven low when the pullup is inactive. In this mode of operation, the pullup resistor can be connected directly between the USB_PUEN and the D+ signal. Some transceivers such as the TUSB1105 from Texas Instruments have an additional input pin called SOFTCON. When this pin is driven High, the D+ gets pulled up through an external resistor to 3.3V. The USB_PUEN can be connected to the SOFTCON pin in this case.

32.4.6.5 USB D+, D- Pulldown for USB Function Controller

System implementations that use the USB function controller and are sensitive to supply current requirements can implement weak pulldown resistors on the USB D+ and D- signals associated with the USB Type-B receptacle. When there is no host controller attached upstream of the USB Type-B receptacle, the undriven D+ and D- wires can float to voltages that cause excessive current consumption by the USB transceiver. Weak pulldowns can help prevent this problem. Selection of pulldown resistors depends on transceiver characteristics, D+ pullup resistor implementation, and board layout, and must be designed to meet USB D+ and D- signal requirements.